claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_49c1018a-e67\agent-af0e103f24820bea1.jsonl`
- SHA-256: `2048f8d7e22627877738cf8168546893fa58233268fcee22ebf662126843b877`
- Source modified: `2026-06-18T01:37:03+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_49c1018a-e67`
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`

Transcript

1. user (2026-06-18T01:33:12.220Z)

```
Adversarial bug-hunter, mature Python repo (badminton-highlight-indexer). Read
READ-ONLY in this worktree (origin/master), DO NOT modify:
  C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/hunt-base
A prior precision pass found ZERO bugs; you cross-cut by BUG CLASS across ALL files to catch
what an area-by-area pass missed. Report a finding ONLY if you READ the code and a CONCRETE
trigger reaches it and a maintainer would call it a real defect. Include exact file+line, the
trigger, why it's wrong, severity, minimal fix, confidence. PRECISION over recall ? empty list
is a valid, respected answer; do NOT manufacture findings. EXCLUDE backend/eval/ablation.py,
backend/eval/fusion_compare.py, backend/eval/fusion_audio.py, scripts/nightly_regression.py.

CLASS: unguarded access on EXTERNAL or
PERSISTED data (the proven vein ? same family as a just-fixed KeyError). Grep/scan EVERY reader
of JSON/DB rows/config/API responses/env for c["x"], d["x"], obj.attr, lst[i] that lacks a guard
and can hit absent/renamed/typed-differently data. Trace each to a real input that triggers it.
```

2. assistant (2026-06-18T01:33:14.578Z)

```
I'll hunt for unguarded access on external/persisted data across the codebase. Let me start by
setting up and surveying the structure.
```

3. user (2026-06-18T01:33:16.278Z)

```
4bd9d2d fix(eval): ablation litmus tolerates a mixed-schema golden corpus (#209)
```

4. user (2026-06-18T01:33:16.560Z)

```
backend\eval\__init__.py
backend\eval\classifier.py
backend\eval\training.py
backend\providers\__init__.py
backend\sports\__init__.py
backend\sports\badminton.py
backend\sports\base.py
backend\utils\__init__.py
backend\utils\geometry.py
backend\annotations\base.py
backend\annotations\file_provider.py
backend\annotations\automated_provider.py
backend\annotations\__init__.py
backend\tools\rally_slicer.py
backend\tools\__init__.py
backend\eval\gemini_refine.py
backend\eval\audio\__init__.py
backend\eval\audio\e1_probe.py
```

backend\eval\export_wasb_segments.py
backend\pipeline\audio__init__.py
backend\pipeline\validators__init__.py
backend\results.py
backend\eval\metrics.py
backend\eval\regression.py
backend\eval\annotations.py
backend\features__init__.py
backend\reporting\harvest.py
backend\reporting\run_signals.py
backend\eval\calibrate_wasb.py
backend\eval\calibration.py
backend\eval\eval_partial_labels.py
backend\eval\rally_seq_proto.py
backend\features\rally_seq.py
backend\pipeline\detectors__init__.py
backend\pipeline\segmenters\base.py
backend\pipeline\segmenters\motion.py
backend\pipeline\validators\blur.py
backend\pipeline\validators\format.py
backend\pipeline\validators\pts_continuity.py
backend\pipeline\validators\relevance.py
backend\pipeline\validators\resolution_fps.py
backend\pipeline\validators\scene_cut.py
backend\providers\base.py
backend\reporting__init__.py
backend\eval\calibrate_local.py
backend\eval\gt_loader.py
backend\eval\harness.py
backend\pipeline\audio\hit_detector.py
backend\pipeline\validators\base.py
backend\utils\paths.py
backend\utils\video.py
backend\pipeline\segmenters\tracknet_hybrid.py
backend\pipeline\segmenters\wasb_hybrid.py
backend\config__init__.py
backend\pipeline\segmenters\yolo_hybrid.py
backend\pipeline\detectors\fusion_features.py
backend\pipeline\detectors\rally_gate.py
backend\pipeline\segmenters__init__.py
backend\tools\export.py
backend\utils\logging_setup.py
backend\pipeline\detectors\audio_features.py
backend\eval\eval_stats.py
backend\eval\score_calibration.py
backend\pipeline\detectors\person_detector.py
backend\eval\fusion_audio.py
backend\eval\fusion_compare.py
backend\eval\experiment.py
backend\infrastructure\execution_policy.py
backend\providers\gemini.py
backend\utils\run_telemetry.py
backend\eval\batch.py
backend\utils\hashing.py
backend\eval\fusion_golden.py
backend\eval\per_user_gate.py
backend\eval\per_user_eval.py
backend\eval\per_user_regression.py
backend\eval\ablation.py
backend\pipeline\personalization\serving.py
backend\pipeline\personalization__init__.py
backend\eval\served_gate_a.py
backend\eval\distill_local.py
backend\eval\promotion.py
backend\eval\serve_contrast.py

```
backend\config\loader.py
backend\eval\splits.py
backend\infrastructure\database.py
backend\main.py
backend\pipeline\detectors\base.py
backend\pipeline\stitcher\compiler.py
backend\pipeline\segmenters\gemini.py
backend\pipeline\detectors\wasb_infer.py
backend\pipeline\detectors\wasb_runner.py
backend\eval\segmentation_metrics.py
backend\config\models.py
backend\eval\golden_fixtures.py
backend\eval\rally_seg_eval.py
backend\eval\tolerance_metrics.py
backend\eval\windowing.py
backend\pipeline\detectors\tracknet_runner.py
backend\pipeline\segmenters\fusion_hybrid.py
(Results are truncated. Consider using a more specific path or pattern.)
```

5. assistant (2026-06-18T01:33:19.135Z)

Let me focus on the bug class. I'll survey the key external/persisted data readers: JSON loaders, DB row readers, config/API response parsers.

6. user (2026-06-18T01:33:21.717Z)

```
__pycache__
annotations
api
assets
config
eval
features
infrastructure
main.py
pipeline
providers
reporting
results.py
sports
tools
utils
---API---
  871 backend/main.py
   60 backend/results.py
  931 total
```

7. user (2026-06-18T01:33:22.885Z)

```
Found 23 files
backend\eval\rally_seg_eval.py
backend\eval\golden_fixtures.py
backend\pipeline\detectors\wasb_infer.py
backend\config\loader.py
backend\eval\splits.py
backend\infrastructure\database.py
backend\eval\serve_contrast.py
backend\eval\distill_local.py
backend\eval\served_gate_a.py
backend\eval\ablation.py
backend\eval\per_user_regression.py
backend\eval\fusion_golden.py
backend\providers\gemini.py
backend\eval\experiment.py
```

```
backend\eval\fusion_compare.py
backend\eval\fusion_audio.py
backend\tools\export.py
backend\api\static\app.js
backend\eval\calibrate_local.py
backend\pipeline\validators\format.py
backend\eval\rally_seq_proto.py
backend\eval\regression.py
backend\eval\classifier.py
```

8. assistant (2026-06-18T01:33:26.567Z)

```
Let me look at the API layer (main.py + api/) and the provider response parsers first ? these
handle external API responses and request bodies, the highest-value targets for this bug class.
```

9. user (2026-06-18T01:33:27.296Z)

```
1       import os
2       import sys
3       import argparse
4       import logging
5       import threading
6       import traceback
7       import uuid
8       from concurrent.futures import ThreadPoolExecutor
9       import uvicorn
10      from dotenv import load_dotenv
11
12      load_dotenv()
13
14      # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
15      logging.basicConfig(
16          level=logging.INFO,
17          format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
18          datefmt='%H:%M:%S'
19      )
20      logger = logging.getLogger(__name__)
21      from fastapi import FastAPI, HTTPException, Request
22      from fastapi.middleware.cors import CORSMiddleware
23      from pydantic import BaseModel
24      from typing import List, Dict, Any, Optional
25
26      from fastapi.staticfiles import StaticFiles
27      from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
28      from backend.pipeline.validators.base import registry
29      from backend.infrastructure.database import Database
30      from backend.pipeline.segmenters.base import segmenter_registry
31      from backend.pipeline.stitcher.compiler import VideoCompiler
32      from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
33      from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path
34      from backend.config import load_config as load_app_config, AppConfig, StitchingConfig  #
new typed config
35      from backend.results import Failure  # standardized error/result contract
36      from backend.reporting import emit_indexer_report
37
38      app = FastAPI(title="Sports Highlight Indexer API")
39
40      # Enable CORS for the local web interface. allow_origins='*' with allow_credentials=True
is
41      # rejected by browsers per the fetch spec and is an unsafe default to carry into the
42      # auth/tenancy work; this tool uses no cookies, so credentials stay off.
43      app.add_middleware(
44          CORSMiddleware,
```

```
45          allow_origins=["*"],
46          allow_credentials=False,
47          allow_methods=["*"],
48          allow_headers=["*"],
49      )
50
51      # Serves static frontend files directly (now under api/ per approved structure)
52      os.makedirs("backend/api/static", exist_ok=True)
53      app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")
54
55      @app.get("/", response_class=HTMLResponse)
56      def serve_index():
57          index_path = "backend/api/static/index.html"
58          if os.path.exists(index_path):
59              with open(index_path, "r", encoding="utf-8") as f:
60                  return f.read()
61          return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"
62
63      # Global variables initialized at startup
64      db_instance: Optional[Database] = None
65      global_config: Optional[AppConfig] = None  # now typed (Pydantic model)
66
67      # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 on purpose: a single
68      # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
69      # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
70      # already parallelize their AI handoff internally via their own pools.
71      _job_executor: Optional[ThreadPoolExecutor] = None
72
73      def _get_executor() -> ThreadPoolExecutor:
74          """Lazy module-global executor; tests monkeypatch this for inline execution."""
75          global _job_executor
76          if _job_executor is None:
77              _job_executor = ThreadPoolExecutor(max_workers=1,
thread_name_prefix="jobworker")
78          return _job_executor
79
80      # Legacy thin wrapper for any remaining direct calls during transition.
81      # New code should import load_app_config / AppConfig from backend.config directly.
82      def load_config(config_path: str = "config.json") -> Dict[str, Any]:
83          cfg = load_app_config(config_path)
84          return cfg.model_dump(mode="json")
85
86      # --- API Models ---
87      class ProcessRequest(BaseModel):
88          video_path: str
89          player_pool: Optional[List[str]] = None
90          max_frames: Optional[int] = None
91          segmenter: Optional[str] = None
92
93      class QueryRequest(BaseModel):
94          video_id: str
95          server: Optional[str] = None
96          receiver: Optional[str] = None
97          duration_min: Optional[float] = None
98          event_type: Optional[str] = None
99
100     class CompileRequest(BaseModel):
101         video_id: str
102         segment_ids: List[int]
103         output_filename: str
104
105     def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
106         """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
107
```

```
        On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
        fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
and
        keep the prior good results intact ? a failed/empty re-run never destroys prior
data.
        """
        if segments:
            db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
        else:
            db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)


    # --- API Endpoints ---
    @app.get("/api/config")
    def get_config():
        return global_config

    def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
        """The full hash ? validate ? segment ? report pipeline for one video.

        This is the former synchronous /api/process body, unchanged in behavior, now run on
        the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
        the sync endpoint used to return (UI compatibility); the per-video observability
        report is still emitted in `finally:` and grafted as response["reports"].

        `progress` is an optional callable(stage: str) used to surface coarse job progress.
        Raises on unexpected internal errors ? the caller records those as a job failure
        (after this function has already restored the pre-run segments, see below).
        """
        notify = progress or (lambda stage: None)

        notify("hashing")
        video_id = compute_video_id(req.video_path)
        was_reingest = db_instance.get_video(video_id) is not None

        # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
        notify("validating")
        logger.info(f"Running validations for {req.video_path}...")
        val_results, val_context = registry.run_all_with_context(req.video_path,
global_config)
        failures = [r for r in val_results if not r.passed]

        # typed AppConfig: attribute access for the default segmenter (with safe fallback)
        default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
        seg_name = req.segmenter or default_seg
        segmenter_actual = None
        segmentation_ran = False
        response: Optional[Dict[str, Any]] = None
        try:
            if failures:
                # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
                # status; it no longer destroys the video's prior good segments.
                db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": "Video failed validation checks.",
                    "results": [r.model_dump() for r in val_results],
                }
                return response

            # 2. Run segmenter & indexer
```

```
            logger.info("[API] Video passed checks. Segmenting and indexing...")
            db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])

            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                # Submit-time validation already 400s unknown names; this is the defensive
                # in-worker path (e.g. config default pointing at an unregistered
segmenter).
                available = list(segmenter_registry.list_available().keys())
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
                    "results": [r.model_dump() for r in val_results],
                    "play_segments": [],
                }
                return response

            logger.info(f"[API] Using segmenter: {seg_name}")
            notify(f"segmenting ({seg_name})")
            segmenter_actual = seg_name
            # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
            # if the run yields segments; otherwise drop the new partial rows and keep the
old.
            play_wm, cand_wm = db_instance.segment_watermarks(video_id)
            segmenter = segmenter_cls(db_instance, global_config)
            try:
                result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
            except Exception:
                # A raising segmenter must not leave half-written rows: restore the pre-run
                # state (same destroy-after-confirm contract as the Failure branch), then
let
                # the job worker record the failure.
                db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
                raise
            segmentation_ran = True

            if isinstance(result, Failure):
                logger.error(f"[API] Segmenter failed: {result.message}")
                db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": f"Segmenter '{seg_name}' failed: {result.message}",
                    "results": [r.model_dump() for r in val_results],
                    "play_segments": [],
                }
                return response

            segments = result.value if hasattr(result, "value") else result
            notify("finalizing")
            _finalize_reingest(video_id, segments, play_wm, cand_wm)

            response = {
                "video_id": video_id,
                "status": "success",
                "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
                "results": [r.model_dump() for r in val_results],
                "play_segments": segments,
            }
```

```
            return response
        finally:
            # Always emit the per-video observability report (next to the video, or to
            # report.output_dir). Never raises. Surface the paths in the response.
            paths = emit_indexer_report(
                db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
                val_results=val_results, val_context=val_context,
                segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
                was_reingest=was_reingest, segmentation_ran=segmentation_ran,
            )
            if paths and isinstance(response, dict):
                response["reports"] = paths


    class JobWaiting(Exception):
        """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
        WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`",
        NOT a failure. The worker persists `next_poll_at` + `progress` (via
`set_job_waiting`),
        releases the thread, and re-claims the job when due (`claim_due_job`). No work to
date
        is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
contract.

        The wiring is additive: the production segmenter path never raises this today (zero
        behaviour change), so a job runs exactly as before. The hook is what a later slice
        (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
        """

        def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
            super().__init__(f"job parked for {delay_sec:.0f}s")
            self.delay_sec = max(0.0, float(delay_sec))
            self.progress = progress


    def _job_to_request(job: Dict[str, Any]) -> ProcessRequest:
        """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
        re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
        the original submit. The row stores exactly the ProcessRequest fields."""
        return ProcessRequest(
            video_path=job["video_path"],
            player_pool=job.get("player_pool"),
            max_frames=job.get("max_frames"),
            segmenter=job.get("segmenter"),
        )


    def _run_claimed_job(job: Dict[str, Any]) -> None:
        """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
its
        terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).

        Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
means
        the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
        result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
the
        job self-scheduled and will be re-claimed when `next_poll_at` is due.
        """
        job_id = job["id"]
        req = _job_to_request(job)
        try:
            result = _execute_processing(
                req, progress=lambda stage: db_instance.set_job_progress(job_id, stage))
```

```
282             if result.get("video_id"):
283                 db_instance.set_job_video_id(job_id, result["video_id"])
284             db_instance.mark_job_done(job_id, result)
285         except JobWaiting as w:
286             # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
287             logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
288             db_instance.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
289         except Exception as e:
290             logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
291             db_instance.mark_job_failed(job_id, f"{type(e).__name__}: {e}")
292
293
294     def _drain_due_jobs() -> int:
295         """One worker tick: claim and run every currently-runnable job (queued, or waiting
whose
296         `next_poll_at` is due), earliest-due first, until the table has nothing runnable.
297
298         Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is
picked
299         up by a FUTURE drain once due; we schedule that follow-up drain via
`_schedule_drain` so
300         a single-worker box keeps making progress with no central timer (the DB is the
clock).
301         Returns the number of jobs that reached a terminal/parked state this tick.
302         """
303         ran = 0
304         while True:
305             job = db_instance.claim_due_job()
306             if job is None:
307                 break
308             ran += 1
309             _run_claimed_job(job)
310         # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a
new
311         # submit. On the single-box executor this is a delayed re-drain; multi-worker just
relies
312         # on the next submit/drain. Best-effort ? never raises into the worker.
313         _schedule_next_drain_if_waiting()
314         return ran
315
316
317     def _schedule_next_drain_if_waiting() -> None:
318         """If any job is parked in 'waiting', arm a re-drain for when the soonest one is
due, so a
319         self-scheduled job resumes with no further client submit. The drain itself re-checks
320         due-ness, so an early wake is harmless and a clamped wake for a far-future job is
fine."""
321         try:
322             delay = db_instance.seconds_until_next_due()
323         except Exception as e:  # pragma: no cover - defensive; never wedge the worker
324             logger.error(f"Could not compute next-due delay: {e}")
325             return
326         if delay is None:
327             return  # nothing waiting
328         # Clamp so a far-future job doesn't pin a thread forever; the drain re-checks due-
ness.
329         _arm_wake_timer(max(0.0, min(delay, _MAX_DRAIN_WAKE_SEC)))
330
331
332     def _arm_wake_timer(delay_sec: float) -> None:
333         """Arm a single daemon timer that re-submits a drain after ``delay_sec`` (no central
334         scheduler ? the DB is the clock). Isolated as the sole timer-creating seam so tests
can
335         monkeypatch it to a no-op and assert the *scheduling decision* without real
waiting."""
```

```
        timer = threading.Timer(delay_sec, lambda: _get_executor().submit(_drain_due_jobs))
        timer.daemon = True
        timer.start()


    #: A parked job further out than this is re-checked by a capped wake rather than one
long
    #: sleeping timer ? keeps the wake cadence bounded without a central scheduler.
    _MAX_DRAIN_WAKE_SEC = 60.0


    @app.post("/api/process", status_code=202)
    def process_video(req: ProcessRequest):
        """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.

        Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
        client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
        runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
        worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
        """
        allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
        try:
            validate_input_path(req.video_path, allowed_root=allowed_root)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        if not os.path.exists(req.video_path):
            raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
        if req.segmenter and not segmenter_registry.get(req.segmenter):
            available = list(segmenter_registry.list_available().keys())
            raise HTTPException(
                status_code=400,
                detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}"
            )

        job_id = uuid.uuid4().hex
        if not db_instance.create_job(job_id, req.video_path, req.segmenter,
                                      req.player_pool, req.max_frames):
            raise HTTPException(status_code=500, detail="Failed to persist job record.")
        _get_executor().submit(_drain_due_jobs)
        return JSONResponse(
            status_code=202,
            content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
        )


    @app.get("/api/jobs/{job_id}")
    def get_job(job_id: str):
        """Job state: {status, progress, result, reports}. `status` = execution state
        (queued|running|done|failed); the pipeline verdict is result["status"]."""
        job = db_instance.get_job(job_id)
        if not job:
            raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
        result = job.get("result")
        return {
            "job_id": job["id"],
            "status": job["status"],
            "progress": job.get("progress"),
            "video_id": job.get("video_id"),
            "error": job.get("error"),
            "result": result,
            "reports": (result or {}).get("reports"),
            "created_at": job.get("created_at"),
```

```
            "started_at": job.get("started_at"),
            "finished_at": job.get("finished_at"),
        }

    # --- Chunked upload (B2, PR-2) ----------------------------------------------------
    # Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
    # proxies cap request bodies at ~100 MB ? HOSTING_PLAN §2), are assembled server-side
    # under security.upload_dir, then submitted to /api/process by the returned path.
    # Upload state is in-process by design (this is the single-box alpha): a server restart
    # aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
    # arrives with PR-3 identity scoping.

    _uploads: Dict[str, Dict[str, Any]] = {}
    _uploads_lock = threading.Lock()


    class UploadInitRequest(BaseModel):
        filename: str
        total_size_bytes: Optional[int] = None


    class UploadCompleteRequest(BaseModel):
        total_parts: int


    def _upload_cfg():
        sec = getattr(global_config, "security", None)
        upload_dir = getattr(sec, "upload_dir", None) or "output/uploads"
        max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
        part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
        exts = [e.lower().lstrip(".") for e in (getattr(sec, "allowed_upload_extensions",
None) or ["mp4", "mov"])]
        return upload_dir, max_bytes, part_bytes, exts


    def _abort_upload(upload_id: str) -> None:
        with _uploads_lock:
            st = _uploads.pop(upload_id, None)
        if st:
            try:
                os.remove(st["tmp_path"])
            except OSError:
                pass


    @app.post("/api/upload/init")
    def upload_init(req: UploadInitRequest):
        """Start a chunked upload. Returns upload_id; PUT sequential parts next."""
        upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
        try:
            name = safe_output_filename(req.filename)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        ext = os.path.splitext(name)[1].lower().lstrip(".")
        if ext not in exts:
            raise HTTPException(status_code=400,
                                detail=f"Extension '.{ext}' not allowed. Allowed:
{sorted(exts)}")
        if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
            raise HTTPException(status_code=413,
                                detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB
upload limit.")
        os.makedirs(upload_dir, exist_ok=True)
        upload_id = uuid.uuid4().hex
        tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
```

```
        open(tmp_path, "wb").close()
        with _uploads_lock:
            _uploads[upload_id] = {"filename": name, "tmp_path": tmp_path, "parts": 0,
"bytes": 0}
        return {"upload_id": upload_id, "max_part_mb": part_bytes // (1024 * 1024),
                "next_part": f"/api/upload/{upload_id}/part/0"}


    @app.put("/api/upload/{upload_id}/part/{index}")
    async def upload_part(upload_id: str, index: int, request: Request):
        """Append one part (raw bytes body). Parts are strictly sequential (0,1,2,...)."""
        _, max_bytes, part_bytes, _ = _upload_cfg()
        with _uploads_lock:
            st = _uploads.get(upload_id)
        if st is None:
            raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
        if index != st["parts"]:
            raise HTTPException(status_code=409,
                                detail=f"Out-of-order part: expected {st['parts']}, got
{index} (parts are sequential)")

        received = 0
        overflow = None
        with open(st["tmp_path"], "ab") as fh:
            async for chunk in request.stream():
                received += len(chunk)
                if received > part_bytes:
                    overflow = f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part
limit."
                    break
                if st["bytes"] + received > max_bytes:
                    overflow = f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total
limit."
                    break
                fh.write(chunk)
        if overflow:
            _abort_upload(upload_id)
            raise HTTPException(status_code=413, detail=overflow + " Upload aborted ? re-
init to retry.")

        with _uploads_lock:
            st["parts"] += 1
            st["bytes"] += received
        return {"upload_id": upload_id, "received_parts": st["parts"], "received_bytes":
st["bytes"]}


    @app.post("/api/upload/{upload_id}/complete")
    def upload_complete(upload_id: str, req: UploadCompleteRequest):
        """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
        upload_dir, _, _, _ = _upload_cfg()
        with _uploads_lock:
            st = _uploads.get(upload_id)
        if st is None:
            raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
        if st["parts"] == 0 or st["parts"] != req.total_parts:
            raise HTTPException(status_code=409,
                                detail=f"Expected {req.total_parts} part(s), received
{st['parts']}.")
        final_path = os.path.join(upload_dir, st["filename"])
        if os.path.exists(final_path):  # never overwrite an earlier upload with the same
name
            stem, ext = os.path.splitext(st["filename"])
            final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
```

```
        os.replace(st["tmp_path"], final_path)
        with _uploads_lock:
            _uploads.pop(upload_id, None)
        video_id = compute_video_id(final_path)
        return {"path": os.path.abspath(final_path), "video_id": video_id,
                "size_bytes": os.path.getsize(final_path),
                "next": "POST /api/process with this path"}


    @app.delete("/api/upload/{upload_id}")
    def upload_abort(upload_id: str):
        """Abort an in-flight upload and delete its partial file."""
        with _uploads_lock:
            known = upload_id in _uploads
        if not known:
            raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
        _abort_upload(upload_id)
        return {"status": "aborted", "upload_id": upload_id}


    # --- Highlight download (B3, PR-2) ------------------------------------------------

    @app.get("/api/highlights/{video_id}/{filename}")
    def download_highlight(video_id: str, filename: str):
        """Stream a compiled highlight reel ? `filename` is the bare name given to
/api/compile
        (which only writes into output_dir; the browser previously got back an unreachable
        server-local path)."""
        try:
            name = safe_output_filename(filename)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        if not db_instance.get_video(video_id):
            raise HTTPException(status_code=404, detail="Indexed video record not found.")
        output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
        path = os.path.join(output_dir, name)
        try:
            path = resolve_under_root(path, output_dir)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        if not os.path.isfile(path):
            raise HTTPException(status_code=404,
                                detail=f"No compiled file named '{name}'. Compile first via
/api/compile.")
        media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
        return FileResponse(path, media_type=media, filename=name)


    @app.post("/api/query")
    def query_play_segments(req: QueryRequest):
        filters = {
            "server": req.server,
            "receiver": req.receiver,
            "duration_min": req.duration_min,
            "event_type": req.event_type
        }
        segments = db_instance.query_play_segments(req.video_id, filters)
        return {"play_segments": segments}

    @app.post("/api/compile")
    def compile_highlights(req: CompileRequest):
        video = db_instance.get_video(req.video_id)
        if not video:
            raise HTTPException(status_code=404, detail="Indexed video record not found.")
```

```
577
578         # Retrieve matching play segments from db
579         all_segments = db_instance.query_play_segments(req.video_id, {})
580         matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]
581
582         if not matched_segments:
583             raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")
584
585         # Reject path traversal / absolute output names (os.path.join would otherwise let an
586         # absolute or '..'-laden output_filename write anywhere on disk).
587         try:
588             out_name = safe_output_filename(req.output_filename)
589         except ValueError as e:
590             raise HTTPException(status_code=400, detail=str(e)) from e
591
592         # The configured shot-count floor (stitching.min_shot_count) applies on the API
593         # path too ? parity with scripts/run_process_and_stitch.py. Segments WITHOUT
shot_count
594         # metadata are exempt: the floor filters known counts only, so explicitly
595         # requested manual/legacy segments can't silently vanish under the default floor.
596         stitching = getattr(global_config, "stitching", None) or StitchingConfig()
597         kept = []
598         for s in matched_segments:
599             meta = s.get("metadata") or {}
600             sc = meta.get("shot_count") if isinstance(meta, dict) else None
601             try:
602                 if sc is not None and int(sc) < stitching.min_shot_count:
603                     continue
604             except (TypeError, ValueError):
605                 pass  # unparseable count -> treat as unknown, keep
606             kept.append(s)
607         if len(kept) < len(matched_segments):
608             logger.info(f"[compile] stitching.min_shot_count={stitching.min_shot_count}:
dropped "
609                         f"{len(matched_segments) - len(kept)}/{len(matched_segments)}
segments below the floor.")
610         if not kept:
611             raise HTTPException(status_code=400,
612                                 detail=f"All {len(matched_segments)} matching segments fall
below "
613
f"stitching.min_shot_count={stitching.min_shot_count}.")
614
615         # Extract start/end time pairs
616         time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
617
618         # Run compiler with the configured stitching paddings + optional branding watermark
619         output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
620         compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
621                                  end_padding=stitching.end_padding,
watermark=stitching.watermark)
622         try:
623             out_path = compiler.compile_highlights(video["filepath"], time_pairs, out_name)
624             return {"status": "success", "filepath": out_path}
625         except Exception as e:
626             raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}") from e
627
628     # --- CLI Debug Utility & Orchestration ---
629     def run_cli_diagnose_clip(video_path: str, timestamp: float):
630         """CLI debugging utility to examine segment properties around a timestamp."""
631         print("=" * 60)
632         print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
```

```
633         print("=" * 60)
634
635         cfg = load_config()  # legacy wrapper still works, returns dict for now
636
637         # 1. Validation check diagnostics
638         print("[DIAGNOSTIC] Running validation framework...")
639         results = registry.run_all(video_path, cfg)
640         for r in results:
641             status_symbol = "[PASS]" if r.passed else "[FAIL]"
642             print(f"  {status_symbol} {r.validator_name}: {r.message}")
643             if r.details:
644                 print(f"         Details: {r.details}")
645
646         # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
647         print("-" * 60)
648         print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
649         import cv2
650         cap = cv2.VideoCapture(video_path)
651         if not cap.isOpened():
652             print("[FAIL] OpenCV could not open video.")
653             return
654
655         fps = cap.get(cv2.CAP_PROP_FPS)
656         total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)
657
658         start_frame = max(0, int((timestamp - 3.0) * fps))
659         end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))
660
661         # Measure frame-by-frame changes in sample region
662         cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
663         prev_gray = None
664         motions = []
665
666         for _ in range(start_frame, end_frame):
667             ret, frame = cap.read()
668             if not ret:
669                 break
670             gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
671             gray = cv2.GaussianBlur(gray, (21, 21), 0)
672
673             if prev_gray is not None:
674                 diff = cv2.absdiff(prev_gray, gray)
675                 _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
676                 motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
677                 motions.append(motion_ratio)
678             prev_gray = gray
679
680         cap.release()
681
682         if motions:
683             avg_motion = sum(motions) / len(motions)
684             # Support both legacy dict (from wrapper) and future direct model
685             if hasattr(cfg, "indexing"):
686                 threshold = getattr(cfg.indexing, "motion_threshold", 0.08)
687             else:
688                 threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08)  # type:
ignore[attr-defined]
689             print(f"  Sample Frame Count: {len(motions)}")
690             print(f"  Average motion index around timestamp: {round(avg_motion, 4)}
(threshold: {threshold})")
691             if avg_motion > threshold:
692                 print("  Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play.")
693             else:
```

```
                print("  Result: [DEAD TIME] Idle movement. Segment classified as dead time.")
        else:
            print("  [ERROR] No visual frames were extracted in the sample range.")

        print("=" * 60)

    def main():
        parser = argparse.ArgumentParser(description="Sports Highlight Indexer Orchestrator")
        parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
        parser.add_argument("--output-dir", default="output", help="Directory where processed clips/highlights are saved")
        parser.add_argument("--setup", action="store_true", help="Launch dependency checker and setup utility")
        parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a video clip to run detailed diagnostics on")
        parser.add_argument("--server", action="store_true", help="Run the FastAPI local API server")
        parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI server on")
        parser.add_argument("--max-frames", type=int, help="Limit processing to the first N sub-sampled frames for debugging")

        available_segmenters = list(segmenter_registry.list_available().keys())
        parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally detection segmenter. Available: {available_segmenters}")
        parser.add_argument("--trim-start", type=float, help="Start time in seconds for the debug trim segment feature")
        parser.add_argument("--trim-end", type=float, help="End time in seconds for the debug trim segment feature")
        parser.add_argument("--trim-output", help="Output filename for the debug trimmed segment (saves in output-dir unless absolute path)")

        args = parser.parse_args()

        if args.setup:
            # Invoke the diagnostics/bootstrap script (renamed from the old root
            # setup.py to scripts/doctor.py so it no longer shadows the PEP 517
            # build system). Resolve its path from this file so `indexer --setup`
            # works regardless of the caller's cwd.
            import subprocess
            doctor_path = os.path.join(
                os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
                "scripts", "doctor.py",
            )
            print(f"[INFO] Invoking diagnostics: {doctor_path} ...")
            subprocess.run([sys.executable, doctor_path])
            return

        if args.trim_start is not None and args.trim_end is not None:
            if not args.video_path:
                print("[ERROR] Please provide --video-path to extract a segment.")
                return

            # Determine output path
            out_filename = args.trim_output or "trimmed_segment.mp4"
            if os.path.isabs(out_filename):
                out_path = out_filename
                out_dir = os.path.dirname(out_path)
                out_name = os.path.basename(out_path)
            else:
                out_dir = args.output_dir
                out_path = os.path.join(out_dir, out_filename)
                out_name = out_filename
```

```
747
748             os.makedirs(out_dir, exist_ok=True)
749             print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")
750
751             # global_config isn't initialized this early; load the typed config so the trim
752             # clip carries the same configured stitching paddings + watermark as a real
753             # highlight (watermark default-on).
754             stitching = load_app_config().stitching
755             compiler = VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
756                                      end_padding=stitching.end_padding,
watermark=stitching.watermark)
757             try:
758                 res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
759                 print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
760             except Exception as e:
761                 print(f"[ERROR] Failed to compile trimmed segment: {e}")
762             return
763
764         if args.debug_clip is not None:
765             if not args.video_path:
766                 print("[ERROR] Please provide --video-path to debug a specific clip.")
767                 return
768             run_cli_diagnose_clip(args.video_path, args.debug_clip)
769             return
770
771         # Initialize Global Config and DB
772         global global_config, db_instance
773         global_config = load_app_config()  # returns typed AppConfig
774
775         # The CLI --output-dir overrides the configured output directory. It now lives
776         # on the typed model (OutputConfig.output_dir), so every consumer ? including
777         # /api/compile ? reads the same value instead of a write-only runtime hack.
778         global_config.output.output_dir = args.output_dir
779         os.makedirs(global_config.output.output_dir, exist_ok=True)
780
781         db_path = os.path.join(global_config.output.output_dir,
global_config.output.db_name)
782         db_instance = Database(db_path)
783
784         # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
785         # the executor is in-process. Mark them failed so pollers see a terminal state.
786         swept = db_instance.fail_stale_jobs()
787         if swept:
788             logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
failed.")
789
790         # CLI processing mode (no web UI needed)
791         if args.video_path and not args.server:
792             print(f"[CLI] Processing video file: {args.video_path}")
793             if not os.path.exists(args.video_path):
794                 print(f"[FAIL] Video file not found: {args.video_path}")
795                 return
796
797             video_id = compute_video_id(args.video_path)
798             was_reingest = db_instance.get_video(video_id) is not None
799
800             # Validate (with-context variant surfaces ffprobe_meta for the report)
801             print("[CLI] Validating...")
802             val_results, val_context = registry.run_all_with_context(args.video_path,
global_config)
803             failures = [r for r in val_results if not r.passed]
804
805             # Save video status
```

```
806             status = "failed" if failures else "processed"
807             db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r
in val_results])
808
809             seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
810             segmenter_actual = None
811             segmentation_ran = False
812             try:
813                 print("\n" + "="*40)
814                 print("VALIDATION SUMMARY")
815                 print("="*40)
816                 for r in val_results:
817                     status_symbol = "[PASS]" if r.passed else "[FAIL]"
818                     print(f"{status_symbol} {r.validator_name}: {r.message}")
819                 print("="*40 + "\n")
820
821                 if failures:
822                     print("[FAIL] Video failed checks. Indexing halted.")
823                     return
824
825                 # Index
826                 segmenter_cls = segmenter_registry.get(seg_name)
827                 if not segmenter_cls:
828                     print(f"[FAIL] Segmenter '{seg_name}' not found.")
829                     return
830
831                 print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
832                 segmenter_actual = seg_name
833                 play_wm, cand_wm = db_instance.segment_watermarks(video_id)
834                 segmenter = segmenter_cls(db_instance, global_config)
835                 result = segmenter.process_video(video_id, args.video_path,
max_frames=args.max_frames)
836                 segmentation_ran = True
837
838                 if isinstance(result, Failure):
839                     print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
840                     print("[FAIL] Indexing halted for video.")
841                     segments = []
842                 else:
843                     segments = result.value if hasattr(result, "value") else result
844                     print(f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}")
845                 # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
846                 _finalize_reingest(video_id, segments, play_wm, cand_wm)
847             finally:
848                 # Always emit the per-video observability report. Never raises.
849                 paths = emit_indexer_report(
850                     db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
851                     val_results=val_results, val_context=val_context,
852                     segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
853                     was_reingest=was_reingest, segmentation_ran=segmentation_ran,
854                 )
855                 if paths:
856                     print(f"[REPORT] Technical report : {paths.get('technical_md')}")
857                     print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
858                     print(f"[REPORT] Machine JSON     : {paths.get('json')}")
859             return
860
861         # Start Local Server Mode
862         if args.server or not args.video_path:
863             print(f"[SERVER] Starting local FastAPI server on
http://localhost:{args.port}...")
```

```
            # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3):
the
            # in-process wake-timer for any parked 'waiting' job died with the old process,
so
            # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
            _get_executor().submit(_drain_due_jobs)
            uvicorn.run(app, host="127.0.0.1", port=args.port)

    if __name__ == "__main__":
        main()

```

10. assistant (2026-06-18T01:33:30.185Z)

Now let me look at the Gemini provider (parses external API responses) and the database row readers, which are prime spots for this bug class.

11. user (2026-06-18T01:33:31.017Z)

```
1       import os
2       import time
3       import json
4       import logging
5       from typing import Tuple
6       from google import genai
7       from google.genai import types
8       from backend.providers.base import AIVideoProvider
9       from backend.infrastructure.execution_policy import (
10          ExecutionPolicy, PolicyTimeout, poll_until, run_bounded,
11      )
12
13      logger = logging.getLogger(__name__)
14
15      # The Gemini File API caps uploads at 2GB; stay well under it. Callers must chunk so
16      # each uploaded clip is below this. Oversize uploads are refused (not silently
attempted).
17      MAX_UPLOAD_MB = 500
18
19      class GeminiProvider(AIVideoProvider):
20          def __init__(self, api_key: str, model_name: str):
21              super().__init__(api_key, model_name)
22              # Initialize client lazily to avoid connection/auth issues during class
instantiation
23              self._client = None
24
25          @property
26          def client(self):
27              if self._client is None:
28                  self._client = genai.Client(api_key=self.api_key)
29              return self._client
30
31          def _safe_delete(self, video_file) -> None:
32              """Best-effort cleanup of an uploaded File (never raises)."""
33              try:
34                  self.client.files.delete(name=video_file.name)
35              except Exception:
36                  pass
37
38          def analyze_video(self, video_path: str, prompt: str,
39                            chunk_duration: float = 0.0,
40                            label: str = "") -> Tuple[list, dict]:
41              log_prefix = f"Gemini {label}" if label else "Gemini"
42
43              metrics = {
44                  "upload_time": 0.0,
```

```
                "process_time": 0.0,
                "gen_time": 0.0
            }

            # 0. Refuse oversize uploads (Gemini File API limit is 2GB; we cap lower for
safety/speed)
            try:
                size_mb = os.path.getsize(video_path) / (1024 * 1024)
            except OSError:
                size_mb = 0.0
            if size_mb > MAX_UPLOAD_MB:
                logger.error(f"[{log_prefix}] {video_path} is {size_mb:.0f}MB >
{MAX_UPLOAD_MB}MB cap ? "
                             f"SKIPPED WITHOUT ANALYSIS (no segments for this time range). "
                             f"Re-encode/downscale chunks (indexing.ai_chunk_scale_width) or
chunk smaller.")
                metrics["oversize_skipped"] = True  # callers aggregate this into the run
report
                return [], metrics

            # 1. Upload (retry transient network failures, e.g. WinError 10053 / reset
connections)
            logger.info(f"[{log_prefix}] Uploading {video_path}...")
            t_upload_start = time.time()
            video_file = None
            for attempt in range(3):
                try:
                    video_file = self.client.files.upload(file=video_path)
                    metrics["upload_time"] = time.time() - t_upload_start
                    logger.info(f"[{log_prefix}] Upload complete in
{metrics['upload_time']:.1f}s. URI: {video_file.uri}")
                    break
                except Exception as e:
                    logger.warning(f"[{log_prefix}] Upload attempt {attempt + 1}/3 failed:
{e}")
                    if attempt < 2:
                        time.sleep(2 * (attempt + 1))
            if video_file is None:
                logger.error(f"[{log_prefix}] Upload failed after 3 attempts.")
                return [], metrics

            # 2. Wait for processing on Google servers ? a BOUNDED poll (a stuck file must
not hang
            # the worker forever). Per-poll logs at DEBUG (no INFO spam); see
EXECUTION_POLICY.md.
            t_process_start = time.time()
            process_policy = ExecutionPolicy(poll_every_n_sec=10.0, max_wait_sec=1200.0)  #
20 min cap

            def _processed():
                nonlocal video_file
                video_file = self.client.files.get(name=video_file.name)
                return None if video_file.state.name == "PROCESSING" else video_file

            try:
                video_file = poll_until(_processed, process_policy,
                                        label=f"{log_prefix} processing", logger=logger)
            except PolicyTimeout:
                logger.error(f"[{log_prefix}] Processing exceeded
{process_policy.max_wait_sec:.0f}s ? giving up.")
                self._safe_delete(video_file)
                return [], metrics
            except Exception as e:
                logger.error(f"[{log_prefix}] Status check failed: {e}")
                self._safe_delete(video_file)
```

```
99                  return [], metrics
100             if video_file.state.name == "FAILED":
101                 logger.error(f"[{log_prefix}] Video processing failed:
{video_file.error.message}")
102                 self._safe_delete(video_file)
103                 return [], metrics
104
105             metrics["process_time"] = time.time() - t_process_start
106             logger.info(f"[{log_prefix}] Processing finished in
{metrics['process_time']:.1f}s.")
107
108             # 3. Call Gemini under a HARD per-attempt timeout (op_timeout_sec ? the hang-
killer that was
109             # missing: a generate() that never returns previously blocked forever) + retry
the 503/network
110             # FAILURE waves (capacity 503s on the flagship flash come in bursts; ~6
attempts).
111             logger.info(f"[{log_prefix}] Submitting multimodal request to
{self.model_name}...")
112             t_gen_start = time.time()
113             gen_policy = ExecutionPolicy(op_timeout_sec=180.0, max_retries=5,
backoff_base_sec=3.0)
114             raw_text = None
115
116             def _generate():
117                 return self.client.models.generate_content(
118                     model=self.model_name,
119                     contents=[video_file, prompt],
120                     config=types.GenerateContentConfig(
121                         response_mime_type="application/json", temperature=0.0),
122                 )
123
124             try:
125                 response = run_bounded(_generate, gen_policy, label=f"{log_prefix}
generate", logger=logger)
126                 raw_text = response.text
127                 metrics["gen_time"] = time.time() - t_gen_start
128                 logger.info(f"[{log_prefix}] Response received in
{metrics['gen_time']:.1f}s:\n{raw_text}")
129             except Exception as e:                  # PolicyTimeout (hang) or the last 503 ?
both end here
130                 logger.error(f"[{log_prefix}] generate gave up after retries: {e}")
131             self._safe_delete(video_file)
132
133             if not raw_text:
134                 return [], metrics
135
136             # 4. Parse JSON
137             try:
138                 segments = json.loads(raw_text)
139                 if not isinstance(segments, list):
140                     logger.error(f"[{log_prefix}] Response is not a JSON array.")
141                     return [], metrics
142                 return segments, metrics
143             except Exception as e:
144                 logger.error(f"[{log_prefix}] JSON parse failed: {e}")
145                 return [], metrics
146
```

12. assistant (2026-06-18T01:33:34.304Z)

The Gemini provider returns segments as a raw list from `json.loads` ? let me trace how consumers access fields in those segment dicts. That's a classic spot for `seg["start"]` on a renamed/missing key. Let me check the segmenter that consumes these.

13. user (2026-06-18T01:33:35.295Z)

```
1       import os
2       import time
3       import logging
4       import subprocess
5       import concurrent.futures
6       from typing import List, Dict, Any, Tuple
7       from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
8       from backend.utils.video import get_video_duration, extract_video_chunk
9       from backend.sports import sport_registry
10      from backend.providers import provider_registry
11      from backend.reporting import run_signals
12
13      logger = logging.getLogger(__name__)
14
15
16      def _chunk_workdir(video_id: str) -> str:
17          """Per-video chunk staging dir under output/chunks/<video_id>.
18
19          Concurrency isolation: two runs for DIFFERENT videos must not share one chunk dir ?
20          otherwise one run finishing (and rmdir-ing the shared dir) can undo the other's
21          in-flight work. A per-video subdir means each run only ever creates/cleans its own.
22          """
23          return os.path.join("output", "chunks", video_id)
24
25
26      class GeminiPlaySegmenter(BasePlaySegmenter):
27          @property
28          def name(self) -> str:
29              return "gemini"
30
31          @property
32          def description(self) -> str:
33              return "High-precision AI-powered semantic play segment detection using Google
Gemini Multimodal API."
34
35          def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
36              """
37              Processes a video to detect play segments using Google Gemini Multimodal Video
API,
38              populates SQLite, and indexes detailed events.
39              """
40              # Get sport profile config
41              sport_name = self.config.get("sport", "badminton")
42              try:
43                  sport_profile = sport_registry.get(sport_name)
44              except KeyError as e:
45                  logger.error(str(e))
46                  return []
47
48              # Get AI provider and model config
49              idx_cfg = self.config.get("indexing", {})
50              provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
51              model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
52
53              # Get appropriate API key based on the provider
54              api_key_env_var = f"{provider_name.upper()}_API_KEY"
55              api_key = os.environ.get(api_key_env_var)
56              if not api_key:
57                  logger.error(
58                      f"{api_key_env_var} environment variable is not set! "
```

```
                    f"To run the {provider_name} segmenter, set your API key. "
                    f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
                )
                return []

            try:
                provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
            except KeyError as e:
                logger.error(str(e))
                return []

            # A: Check for debug_duration to trim the video first
            debug_duration = self.config.get("indexing", {}).get("debug_duration")
            video_to_upload = video_path
            is_trimmed = False
            temp_trimmed_path = None

            if debug_duration:
                logger.info(f"DEBUG FEATURE: Trimming video to first {debug_duration}
seconds for fast processing...")
                temp_trimmed_path = os.path.join("output", f"temp_trimmed_{video_id}.mp4")
                os.makedirs("output", exist_ok=True)
                if os.path.exists(temp_trimmed_path):
                    try:
                        os.remove(temp_trimmed_path)
                    except Exception:
                        pass

                cmd = [
                    "ffmpeg", "-y",
                    "-ss", "0",
                    "-t", str(debug_duration),
                    "-i", video_path,
                    "-c", "copy",
                    temp_trimmed_path
                ]
                logger.debug(f"Running command: {' '.join(cmd)}")
                try:
                    subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
                    logger.info(f"Trimmed video successfully created at:
{temp_trimmed_path}")
                    video_to_upload = temp_trimmed_path
                    is_trimmed = True
                except Exception as e:
                    logger.warning(f"Failed to trim video using ffmpeg: {e}. Falling back to
full video.")


            # Helper to clean up local temp files
            def cleanup_temp_files():
                if is_trimmed and temp_trimmed_path and os.path.exists(temp_trimmed_path):
                    try:
                        os.remove(temp_trimmed_path)
                    except Exception:
                        pass

            # 1. Determine actual video duration
            video_duration = get_video_duration(video_to_upload)
            if video_duration > 0:
                logger.info(f"Detected video duration: {video_duration:.1f}s")
            else:
                logger.warning("Could not detect video duration. Prompt will not include
```

```
duration context.")
119
120             # 2. Decide whether to use chunked processing
121             idx_cfg = self.config.get("indexing", {})
122             chunk_duration = idx_cfg.get("chunk_duration", 300)    # 5 minutes default
123             chunk_overlap = idx_cfg.get("chunk_overlap", 30)       # 30 seconds overlap
124             use_chunking = video_duration > 0 and video_duration > chunk_duration
125             # Re-encode + downscale chunks for upload (0 = legacy stream-copy). Stream-
copied
126             # 4K chunks (~1.3GB/90s) exceed the provider's MAX_UPLOAD_MB and are refused, so
127             # every chunk of a 4K source gets skipped; Gemini analyzes at low res anyway.
128             ai_scale_width = int(idx_cfg.get("ai_chunk_scale_width", 1280) or 0)
129
130             ai_segments = []
131             # Per-video chunk workdir (concurrency isolation): a concurrent run for another
132             # video keeps its own dir, so cleanup here never removes another run's chunks.
133             chunks_dir = _chunk_workdir(video_id)
134
135             if use_chunking:
136                 # --- CHUNKED PROCESSING ---
137                 step = chunk_duration - chunk_overlap
138                 chunk_starts = []
139                 t = 0.0
140                 while t < video_duration:
141                     chunk_starts.append(t)
142                     t += step
143                 num_chunks = len(chunk_starts)
144                 logger.info(
145                     f"Video is {video_duration:.1f}s ? splitting into {num_chunks}
overlapping chunks "
146                     f"({chunk_duration}s each, {chunk_overlap}s overlap)."
147                 )
148
149                 os.makedirs(chunks_dir, exist_ok=True)
150
151                 def process_single_chunk(ci: int, chunk_start: float) -> Tuple[str, list,
dict]:
152                     chunk_end = min(chunk_start + chunk_duration, video_duration)
153                     actual_chunk_dur = chunk_end - chunk_start
154                     chunk_label = f"Chunk {ci+1}/{num_chunks}"
155                     chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
156
157                     # Extract chunk with ffmpeg
158                     logger.info(f"Extracting {chunk_label}: {chunk_start:.1f}s -
{chunk_end:.1f}s ({actual_chunk_dur:.1f}s)")
159                     success = extract_video_chunk(
160                         video_to_upload, chunk_start, actual_chunk_dur, chunk_path,
161                         reencode=ai_scale_width > 0,
162                         scale_width=ai_scale_width if ai_scale_width > 0 else None,
163                     )
164                     if not success:
165                         logger.error(f"Failed to extract {chunk_label}. Skipping.")
166                         return chunk_label, [], {}
167
168                     # Measure chunk size
169                     try:
170                         chunk_size_mb = os.path.getsize(chunk_path) / (1024 * 1024)
171                     except Exception:
172                         chunk_size_mb = 0.0
173
174                     # Get prompt from sport profile
175                     prompt = sport_profile.get_prompt(chunk_duration=actual_chunk_dur)
176
177                     # Process this chunk through the provider
178                     chunk_segments, metrics = provider.analyze_video(
```

```
                        chunk_path, prompt, chunk_duration=actual_chunk_dur,
label=chunk_label
                    )
                    metrics["file_size_mb"] = chunk_size_mb
                    metrics["segments_found"] = len(chunk_segments)

                    # Offset timestamps back to full-video coordinates
                    for segment in chunk_segments:
                        if "start_time" in segment:
                            try:
                                segment["start_time"] = float(segment["start_time"]) +
chunk_start
                            except (ValueError, TypeError):
                                pass
                        if "end_time" in segment:
                            try:
                                segment["end_time"] = float(segment["end_time"]) +
chunk_start
                            except (ValueError, TypeError):
                                pass

                    logger.info(f"{chunk_label} returned {len(chunk_segments)} play
segments.")

                    # Clean up chunk file
                    try:
                        os.remove(chunk_path)
                    except Exception:
                        pass

                    return chunk_label, chunk_segments, metrics

                # Run chunks in parallel. Workers come from indexing.ai_max_workers (same
knob
                # the hybrids honor; was hardcoded 5). ai_max_workers=1 = serial ? needed to
                # gently ramp a cold/prepaid Gemini project, whose burst throttling returns
a
                # misleading "credits depleted" 429 under 5 simultaneous uploads.
                all_metrics = {}
                max_workers = max(1, int(self.config.get("indexing",
{}).get("ai_max_workers", 5)))
                logger.info("Starting parallel processing with up to %d concurrent
workers...", max_workers)
                t_chunking_start = time.time()
                with concurrent.futures.ThreadPoolExecutor(max_workers=max_workers) as
executor:
                    futures = [executor.submit(process_single_chunk, ci, start) for ci,
start in enumerate(chunk_starts)]
                    for future in concurrent.futures.as_completed(futures):
                        label, segments, metrics = future.result()
                        ai_segments.extend(segments)
                        if metrics:
                            all_metrics[label] = metrics

                # Oversize refusals must be LOUD: each one is a time range with NO analysis,
                # so "0 segments / success" here would be a silent failure, not "no
rallies".
                oversize_skips = sum(1 for m in all_metrics.values() if
m.get("oversize_skipped"))
                if oversize_skips:
                    logger.error(
                        f"{oversize_skips}/{num_chunks} chunks exceeded the provider upload
cap and were "
                        f"SKIPPED WITHOUT ANALYSIS ? segments are missing for those time
ranges. "
```

```
                        f"Set indexing.ai_chunk_scale_width (default 1280) so chunks are re-
encoded/"
                        f"downscaled before upload."
                    )
                    run_signals.incr(video_id, "oversize_clips_skipped", oversize_skips)

                # Print Summary Log
                summary_lines = [
                    "",
                    "=" * 60,
                    "               GEMINI CHUNKING SUMMARY",
                    "=" * 60,
                    f" Total time: {time.time() - t_chunking_start:.1f}s",
                    f" Total chunks: {num_chunks}",
                    f" Total play segments found: {len(ai_segments)}",
                    "-" * 60,
                    f" {'Chunk':<12} | {'Size':<8} | {'Upload':<8} | {'Process':<8} |
{'Gen.':<8} | {'Segments'}",
                    "-" * 60,
                ]
                for i in range(num_chunks):
                    lbl = f"Chunk {i+1}/{num_chunks}"
                    m = all_metrics.get(lbl, {})
                    sz = f"{m.get('file_size_mb', 0):.1f}MB" if m else "N/A"
                    upl = f"{m.get('upload_time', 0):.1f}s" if m else "N/A"
                    prc = f"{m.get('process_time', 0):.1f}s" if m else "N/A"
                    gen = f"{m.get('gen_time', 0):.1f}s" if m else "N/A"
                    seg = m.get('segments_found', 0) if m else 0
                    summary_lines.append(f" {lbl:<12} | {sz:<8} | {upl:<8} | {prc:<8} |
{gen:<8} | {seg}")
                summary_lines.append("=" * 60)
                logger.info("\n".join(summary_lines))

                # Clean up local temp files
                cleanup_temp_files()
                try:
                    os.rmdir(chunks_dir)
                except Exception:
                    pass

                logger.info(f"All chunks processed. {len(ai_segments)} total play segments
before deduplication.")

            else:
                # --- SINGLE-FILE PROCESSING (short videos) ---
                try:
                    file_sz = os.path.getsize(video_to_upload) / (1024 * 1024)
                except Exception:
                    file_sz = 0.0

                # Get prompt from sport profile
                prompt = sport_profile.get_prompt(chunk_duration=video_duration)
                ai_segments, metrics = provider.analyze_video(
                    video_to_upload, prompt, chunk_duration=video_duration,
label="SingleFile"
                )
                if metrics.get("oversize_skipped"):
                    logger.error(
                        f"The video ({file_sz:.0f}MB) exceeded the provider upload cap and
was "
                        f"SKIPPED WITHOUT ANALYSIS ? 0 segments here does NOT mean 'no
rallies'."
                    )
                    run_signals.incr(video_id, "oversize_clips_skipped", 1)
                summary_lines = [
```

```
                    "",
                    "=" * 50,
                    "        GEMINI SINGLE FILE SUMMARY",
                    "=" * 50,
                    f" File Size: {file_sz:.1f} MB",
                    f" Upload time: {metrics.get('upload_time', 0):.1f}s",
                    f" API Processing time: {metrics.get('process_time', 0):.1f}s",
                    f" Prompt Generation time: {metrics.get('gen_time', 0):.1f}s",
                    f" Play segments found: {len(ai_segments)}",
                    "=" * 50,
                ]
                logger.info("\n".join(summary_lines))
                cleanup_temp_files()

            if not ai_segments:
                return []

            # Populate SQLite DB with parsed play segments
            segments_data = []
            logger.info(f"Processing {len(ai_segments)} play segments through
validation...")

            def parse_time(val) -> float:
                """Converts a timestamp value to total float seconds.
                Handles float/int pass-through, plain numeric strings, and
                colon-separated formats (MM:SS, HH:MM:SS). Logs warnings
                when unexpected formats are encountered."""
                if isinstance(val, (int, float)):
                    return float(val)
                if isinstance(val, str):
                    val = val.strip()
                    if ":" in val:
                        # Gemini returned MM:SS or HH:MM:SS despite being asked for decimal
seconds.
                        # Convert it, but warn so the issue is visible in logs.
                        parts = val.split(":")
                        parts.reverse()
                        total = 0.0
                        for i, p in enumerate(parts):
                            try:
                                total += float(p) * (60 ** i)
                            except ValueError:
                                pass
                        logger.warning(f"Timestamp '{val}' was in colon-separated format
(MM:SS/HH:MM:SS) instead of decimal seconds. Converted to {total:.2f}s.")
                        return total
                    try:
                        return float(val)
                    except ValueError:
                        logger.warning(f"Could not parse timestamp string '{val}' as a
number. Defaulting to 0.0.")
                        return 0.0
                logger.warning(f"Unexpected timestamp type {type(val).__name__} for value
'{val}'. Defaulting to 0.0.")
                return 0.0

            # 7. Parse timestamps and build candidate segment list
            idx_cfg = self.config.get("indexing", {})
            min_segment_duration = idx_cfg.get("min_segment_duration", 3.0)
            max_segment_duration = 90.0  # Singles play segments almost never exceed this

            candidates = []
            for idx, segment in enumerate(ai_segments):
                start = parse_time(segment.get("start_time", 0.0))
                end = parse_time(segment.get("end_time", 0.0))
```

```
348                 if start >= end:
349                     logger.info(f"Skipping segment #{idx+1}: start_time ({start:.2f}s) >=
end_time ({end:.2f}s)")
350                     continue
351                 candidates.append({
352                     "idx": idx,
353                     "start": start,
354                     "end": end,
355                     "raw": segment
356                 })
357 
358             # 8. Post-hoc validation pass
359             logger.info(f"Running validation on {len(candidates)} candidate play
segments...")
360             validated = []
361             rejected = []
362 
363             for c in candidates:
364                 label = f"Segment #{c['idx']+1} [{c['start']:.2f}s - {c['end']:.2f}s]"
365 
366                 # A: Reject segments that start beyond the video
367                 if video_duration > 0 and c["start"] >= video_duration:
368                     rejected.append((label, f"start_time ({c['start']:.2f}s) is beyond video
duration ({video_duration:.1f}s)"))
369                     continue
370 
371                 # B: Clamp end_time to video duration
372                 if video_duration > 0 and c["end"] > video_duration:
373                     logger.info(f"{label}: end_time ({c['end']:.2f}s) exceeds video duration
({video_duration:.1f}s). Clamping.")
374                     c["end"] = video_duration
375 
376                 dur = c["end"] - c["start"]
377 
378                 # C: Reject segments shorter than min_segment_duration
379                 if dur < min_segment_duration:
380                     rejected.append((label, f"duration ({dur:.2f}s) is below minimum
({min_segment_duration}s)"))
381                     continue
382 
383                 # D: Flag suspiciously long segments
384                 if dur > max_segment_duration:
385                     logger.warning(
386                         f"{label}: duration ({dur:.1f}s) is unusually long
(>{max_segment_duration}s). "
387                         f"This may be a detection error. Keeping but flagging."
388                     )
389 
390                 validated.append(c)
391 
392             # E: Resolve overlapping segments (keep the longer one)
393             if len(validated) > 1:
394                 # Sort by start time
395                 validated.sort(key=lambda r: r["start"])
396                 non_overlapping = [validated[0]]
397                 for curr in validated[1:]:
398                     prev = non_overlapping[-1]
399                     # Check if current segment overlaps with the previous one
400                     if curr["start"] < prev["end"]:
401                         prev_dur = prev["end"] - prev["start"]
402                         curr_dur = curr["end"] - curr["start"]
403                         kept, dropped = (prev, curr) if prev_dur >= curr_dur else (curr,
prev)
404                         kept_label = f"Segment #{kept['idx']+1} [{kept['start']:.2f}s -
{kept['end']:.2f}s]"
```

```
                        dropped_label = f"Segment #{dropped['idx']+1}
[{dropped['start']:.2f}s - {dropped['end']:.2f}s]"
                        rejected.append((dropped_label, f"overlaps with {kept_label} ? kept
the longer segment"))
                        if kept == curr:
                            non_overlapping[-1] = curr  # Replace previous with current
                    else:
                        non_overlapping.append(curr)
                validated = non_overlapping

            # Log validation summary
            if rejected:
                reject_lines = [f"  ? {label}: {reason}" for label, reason in rejected]
                logger.info(f"{len(rejected)} play segments were REJECTED:\n" +
"\n".join(reject_lines))
            logger.info(f"{len(validated)} play segments passed validation (out of
{len(ai_segments)} from Gemini).")

            stitch_cfg = self.config.get("stitching", {})
            begin_pad = stitch_cfg.get("begin_padding", 0.5)
            end_pad = stitch_cfg.get("end_padding", 0.5)

            total_highlight_dur = 0.0
            for c in validated:
                total_highlight_dur += (c["end"] - c["start"]) + begin_pad + end_pad

            mins = int(total_highlight_dur // 60)
            secs = int(total_highlight_dur % 60)
            logger.info(
                f"Estimated final highlight video duration: {mins:02d}:{secs:02d} "
                f"({total_highlight_dur:.1f}s total; includes {begin_pad}s begin +
{end_pad}s end padding per segment)"
            )

            # 9. Populate SQLite DB with validated play segments
            for c in validated:
                start = c["start"]
                end = c["end"]
                segment = c["raw"]
                dur = end - start

                # Extract shot_count from Gemini or compute estimate if not present
                shot_count = segment.get("shuttle_exchange_count")
                if shot_count is None:
                    shot_count = max(3, int(dur / 0.8))  # 1 shot every 0.8 seconds as
fallback estimate
                else:
                    try:
                        shot_count = int(shot_count)
                    except Exception:
                        shot_count = max(3, int(dur / 0.8))

                # Only store metrics we actually know ? no fabricated data
                metadata = {
                    "shot_count": shot_count,
                    "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
                    "detector": model_name
                }

                segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)

                if segment_id:
                    segments_data.append({
```

```
462                         "id": segment_id,
463                         "start_time": start,
464                         "end_time": end,
465                         "duration": dur,
466                         "metadata": metadata
467                     })
468
469             return segments_data
470
471     # Register the segmenter
472     segmenter_registry.register(GeminiPlaySegmenter)
473
```

14. assistant (2026-06-18T01:33:39.064Z)

This file uses `.get()` consistently. Let me now look at the database layer (row readers) and the config loader, which are persisted-data readers. Then I'll check the eval/stitcher consumers of DB rows and JSON fixtures.

15. user (2026-06-18T01:33:40.530Z)

```
1       import sqlite3
2       import json
3       import logging
4       from contextlib import contextmanager
5       from typing import List, Dict, Any, Optional, Tuple
6
7       logger = logging.getLogger(__name__)
8
9       class Database:
10          def __init__(self, db_path: str):
11              self.db_path = db_path
12              self._init_db()
13
14          @contextmanager
15          def _connect(self):
16              """Transaction scope that also CLOSES the connection.
17
18              ``with sqlite3.connect(...)`` alone is only a commit/rollback scope ? it
19              never closes, so every call leaked its connection to GC (lingering file
20              handles on the WAL sidecars under Windows). Internal methods use this.
21              """
22              conn = self._get_connection()
23              try:
24                  with conn:
25                      yield conn
26              finally:
27                  conn.close()
28
29          def _get_connection(self):
30              # busy_timeout: wait (not fail) on a locked DB ? needed once the async job model
31              # adds concurrent writers (a swallowed 'database is locked' silently drops
segments).
32              conn = sqlite3.connect(self.db_path, timeout=30.0)
33              conn.row_factory = sqlite3.Row
34              # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
35              conn.execute("PRAGMA foreign_keys = ON")
36              # WAL allows concurrent readers during a write; busy_timeout bounds lock waits.
37              conn.execute("PRAGMA busy_timeout = 30000")
38              try:
39                  conn.execute("PRAGMA journal_mode = WAL")
40              except sqlite3.OperationalError:
41                  pass  # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
42              return conn
43
```

```
        def _init_db(self):
            """Initializes tables for videos, play_segments, and events."""
            with self._connect() as conn:
                cursor = conn.cursor()

                # Videos Table
                cursor.execute("""
                    CREATE TABLE IF NOT EXISTS videos (
                        id TEXT PRIMARY KEY,
                        filepath TEXT NOT NULL,
                        processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                        status TEXT NOT NULL,
                        validation_results TEXT
                    )
                """)

                # Play Segments Table (Extensible metadata JSON)
                cursor.execute("""
                    CREATE TABLE IF NOT EXISTS play_segments (
                        id INTEGER PRIMARY KEY AUTOINCREMENT,
                        video_id TEXT NOT NULL,
                        start_time REAL NOT NULL,
                        end_time REAL NOT NULL,
                        duration REAL NOT NULL,
                        server TEXT,
                        receiver TEXT,
                        metadata TEXT,
                        FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
                    )
                """)

                # Events Table
                cursor.execute("""
                    CREATE TABLE IF NOT EXISTS events (
                        id INTEGER PRIMARY KEY AUTOINCREMENT,
                        segment_id INTEGER NOT NULL,
                        timestamp REAL NOT NULL,
                        type TEXT NOT NULL,
                        player TEXT,
                        details TEXT,
                        FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE
CASCADE
                    )
                """)

                # Versioned migrations live here. PRAGMA user_version is the schema
                # contract ? bump it for every change and add an ordered `if version < N`
                # block below. Tests assert the expected version, so accidental drift fails
CI.
                version = cursor.execute("PRAGMA user_version").fetchone()[0]

                if version < 1:
                    # v1: raw candidate detections (pre-confirmation), detector-agnostic.
                    # Common fields are columns; detector-specific metrics go in `stats`
JSON,
                    # so a new segmenter reuses this table by setting its own
`detector`/`stats`.
                    cursor.execute("""
                        CREATE TABLE IF NOT EXISTS candidate_segments (
                            id INTEGER PRIMARY KEY AUTOINCREMENT,
                            video_id TEXT NOT NULL,
                            detector TEXT NOT NULL,
                            chunk_index INTEGER,
                            start_time REAL NOT NULL,
                            end_time REAL NOT NULL,
```

```
                            duration REAL NOT NULL,
                            confidence REAL,
                            stats TEXT,
                            detection_params TEXT,
                            confirmed_segment_id INTEGER,
                            human_label TEXT,
                            notes TEXT,
                            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                            FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
                            FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id)
ON DELETE SET NULL
                        )
                    """)
                    cursor.execute("""
                        CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
                            ON candidate_segments(video_id, detector)
                    """)
                    cursor.execute("PRAGMA user_version = 1")

                if version < 2:
                    # v2: async job model (DEFERRED_HARDENING_PLAN #16). One row per
submitted
                    # /api/process request. `status` is the EXECUTION state of the job
                    # (queued|running|done|failed); the pipeline's own verdict (e.g. a video
                    # that failed validation) lives inside `result` JSON as
result["status"].
                    # `result` is the full sync-era response dict (incl. report paths), so
the
                    # observability report is the job's artifact, per the plan.
                    cursor.execute("""
                        CREATE TABLE IF NOT EXISTS jobs (
                            id TEXT PRIMARY KEY,
                            video_path TEXT NOT NULL,
                            video_id TEXT,
                            segmenter TEXT,
                            player_pool TEXT,
                            max_frames INTEGER,
                            status TEXT NOT NULL DEFAULT 'queued',
                            progress TEXT,
                            error TEXT,
                            result TEXT,
                            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                            started_at TIMESTAMP,
                            finished_at TIMESTAMP
                        )
                    """)
                    cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_status ON
jobs(status)")
                    cursor.execute("PRAGMA user_version = 2")

                if version < 3:
                    # v3: self-scheduling execution policy (EXECUTION_POLICY.md P2).
`policy` = the
                    # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting'
job is due
                    # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ?
any worker
                    # re-claims it via `claim_due_job` when due ? with NO master node: the
table IS
                    # the coordinator. ALTER (not recreate) so existing v2 job rows survive.
                    cursor.execute("ALTER TABLE jobs ADD COLUMN policy TEXT")
                    cursor.execute("ALTER TABLE jobs ADD COLUMN next_poll_at TIMESTAMP")
                    cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,
next_poll_at)")
                    cursor.execute("PRAGMA user_version = 3")
```

```
160
161                 if version < 4:
162                     # v4: personalization groundwork (PERSONALIZATION_PLAN.md §2). Add a
NULLABLE
163                     # `user_id` to videos + candidate_segments so a future per-user / multi-
tenant
164                     # path can scope rows to an owner. **NULL = local install = byte-
identical to
165                     # today** ? every existing row stays NULL and every existing query
(which never
166                     # filters on user_id) is unaffected. ALTER (not recreate) so v1-v3 rows
survive.
167                     # Additive ONLY: no served-behavior change rides on this slice.
168                     cursor.execute("ALTER TABLE videos ADD COLUMN user_id TEXT")
169                     cursor.execute("ALTER TABLE candidate_segments ADD COLUMN user_id TEXT")
170                     # Partial indexes (WHERE user_id IS NOT NULL) keep the NULL=local fast
path free
171                     # of index churn while making the eventual per-user lookups cheap.
172                     cursor.execute(
173                         "CREATE INDEX IF NOT EXISTS idx_videos_user ON videos(user_id) "
174                         "WHERE user_id IS NOT NULL")
175                     cursor.execute(
176                         "CREATE INDEX IF NOT EXISTS idx_candidates_user "
177                         "ON candidate_segments(user_id) WHERE user_id IS NOT NULL")
178                     cursor.execute("PRAGMA user_version = 4")
179
180                 if version < 5:
181                     # v5: per-user model store (PERSONALIZATION_PLAN.md §2). One row per
(user_id,
182                     # version): the resolved per-user `fusion_config` overlay + an INLINE
183                     # `classifier_json` (tenant-safe ? no file artifact), plus the promotion
evidence
184                     # and provenance that earned it. `is_active` pins the one row the
serving bridge
185                     # resolves (the FUTURE owner-gated wiring into `_select_for_handoff`).
This slice
186                     # only persists/loads it ? NOTHING reads it on the served path yet.
187                     #
188                     #   user_id              ? owner of the model (NULL allowed = the local
install)
189                     #   version              ? monotonic per-user model version
190                     #   baseline_version     ? the shared baseline this was fit against
(upgrade switch)
191                     #   feature_schema_hash  ? fingerprint of the feature set; MINOR-vs-
MAJOR re-fit gate
192                     #   fusion_config        ? JSON per-user FusionConfig overlay (cue_flags
/ thresholds)
193                     #   classifier_json      ? JSON LogisticRegression.to_dict() (None =
config-only model)
194                     #   promotion_evidence   ? JSON per_user_gate PromotionDecision (why it
was promoted)
195                     #   n_videos_at_fit      ? held-out-user corpus size when fit (min_n
trust floor input)
196                     #   is_active            ? the one resolved row for this user (partial-
unique below)
197                     cursor.execute("""
198                         CREATE TABLE IF NOT EXISTS user_models (
199                             id INTEGER PRIMARY KEY AUTOINCREMENT,
200                             user_id TEXT,
201                             version INTEGER NOT NULL DEFAULT 1,
202                             baseline_version TEXT,
203                             feature_schema_hash TEXT,
204                             fusion_config TEXT,
205                             classifier_json TEXT,
206                             promotion_evidence TEXT,
```

```
                            n_videos_at_fit INTEGER NOT NULL DEFAULT 0,
                            is_active INTEGER NOT NULL DEFAULT 0,
                            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
                        )
                    """)
                    # One model version per user (NULLs compare distinct in SQLite, so the
local
                    # install can still carry multiple versioned rows; the active-row guard
below
                    # is what enforces a single served model).
                    cursor.execute("""
                        CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_user_version
                            ON user_models(user_id, version)
                    """)
                    # At most ONE active model per user: a partial-unique index over the
active rows.
                    # (SQLite treats NULL user_id rows as distinct, so the local install's
single
                    # active row is still guarded ? there is one local user.)
                    cursor.execute("""
                        CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_one_active
                            ON user_models(user_id) WHERE is_active = 1
                    """)
                    cursor.execute("PRAGMA user_version = 5")
                # future: if version < 6: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 6

                conn.commit()

        def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
            """Register or update a video row WITHOUT touching its child segments.

            Uses UPSERT, not INSERT OR REPLACE: REPLACE deletes the conflicting row, which ?
            with foreign_keys ON ? cascade-deletes every
play_segment/event/candidate_segment.
            That made a status update (e.g. marking 'failed' on re-validation, or
'processed'
            before the segmenter runs) silently destroy prior good results. UPSERT mutates
the
            row in place; clearing segments is now an explicit, deliberate operation
            (clear_video_data / prune_segments).
            """
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "INSERT INTO videos (id, filepath, status, validation_results)
VALUES (?, ?, ?, ?) "
                        "ON CONFLICT(id) DO UPDATE SET "
                        "filepath=excluded.filepath, status=excluded.status, "
                        "validation_results=excluded.validation_results",
                        (video_id, filepath, status, json.dumps(validation_results))
                    )
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to add video: {e}")
                return False

        def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
            """Return (max play_segment id, max candidate_segment id) for a video (0 if
none).

            Captured before a (re)segmentation so the run's new rows (id > watermark) can be
            told apart from prior rows (id <= watermark), enabling destroy-AFTER-confirm.
```

```
            """
            with self._connect() as conn:
                cur = conn.cursor()
                p = cur.execute("SELECT COALESCE(MAX(id), 0) FROM play_segments WHERE
video_id = ?",
                                (video_id,)).fetchone()[0]
                c = cur.execute("SELECT COALESCE(MAX(id), 0) FROM candidate_segments WHERE
video_id = ?",
                                (video_id,)).fetchone()[0]
                return int(p), int(c)

        def prune_segments(self, video_id: str, *, play_upto: Optional[int] = None,
                           play_after: Optional[int] = None, cand_upto: Optional[int] =
None,
                           cand_after: Optional[int] = None) -> None:
            """Delete play/candidate segments by id range (events cascade from
play_segments).

            Used to finish a (re)segmentation: keep the NEW rows and drop the OLD ones on a
            successful run (``*_upto`` = the pre-run watermark), or drop the NEW partial
rows
            and keep the OLD ones when the run failed/produced nothing (``*_after``).
            """
            with self._connect() as conn:
                cur = conn.cursor()
                if play_upto is not None:
                    cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id <= ?",
(video_id, play_upto))
                if play_after is not None:
                    cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id > ?",
(video_id, play_after))
                if cand_upto is not None:
                    cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id <=
?", (video_id, cand_upto))
                if cand_after is not None:
                    cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id >
?", (video_id, cand_after))
                conn.commit()

        def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
            try:
                with self._connect() as conn:
                    cursor = conn.cursor()
                    cursor.execute(
                        "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
                        (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {}))
                    )
                    conn.commit()
                    return cursor.lastrowid
            except Exception as e:
                logger.error(f"Failed to add play segment: {e}")
                return None

        def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "INSERT INTO events (segment_id, timestamp, type, player, details)
VALUES (?, ?, ?, ?, ?)",
                        (segment_id, timestamp, event_type, player, json.dumps(details or
```

```
{}))
                    )
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to add event: {e}")
                return False

        def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
                                  chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
                                  stats: Optional[Dict[str, Any]] = None,
                                  detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
            """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
            detector-specific dicts stored as JSON. Returns the new candidate id, or
None."""
            try:
                with self._connect() as conn:
                    cursor = conn.cursor()
                    cursor.execute(
                        """INSERT INTO candidate_segments
                           (video_id, detector, chunk_index, start_time, end_time, duration,
                            confidence, stats, detection_params)
                           VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
                        (video_id, detector, chunk_index, start_time, end_time, end_time -
start_time,
                         confidence, json.dumps(stats or {}), json.dumps(detection_params or
{}))
                    )
                    conn.commit()
                    return cursor.lastrowid
            except Exception as e:
                logger.error(f"Failed to add candidate segment: {e}")
                return None

        def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
            """Record that a candidate detection was confirmed as a given play_segment."""
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id =
?",
                        (segment_id, candidate_id)
                    )
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to link candidate {candidate_id} to segment
{segment_id}: {e}")
                return False

        def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
                                     only_unlabeled: bool = False) -> List[Dict[str, Any]]:
            """Fetch candidate detections for the annotation / fine-tuning workflow.
            `stats` and `detection_params` are deserialized from JSON."""
            query = "SELECT * FROM candidate_segments WHERE video_id = ?"
            params: List[Any] = [video_id]

            if detector:
                query += " AND detector = ?"
                params.append(detector)
            if only_unlabeled:
                query += " AND human_label IS NULL"
```

```
367
368             query += " ORDER BY start_time"
369
370             candidates = []
371             with self._connect() as conn:
372                 rows = conn.cursor().execute(query, params).fetchall()
373                 for row in rows:
374                     d = dict(row)
375                     d["stats"] = json.loads(d["stats"] or "{}")
376                     d["detection_params"] = json.loads(d["detection_params"] or "{}")
377                     candidates.append(d)
378             return candidates
379
380         def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
381             with self._connect() as conn:
382                 row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?",
(video_id,)).fetchone()
383                 if row:
384                     d = dict(row)
385                     d["validation_results"] = json.loads(d["validation_results"] or "[]")
386                     return d
387             return None
388
389         def query_play_segments(self, video_id: str, filters: Dict[str, Any]) ->
List[Dict[str, Any]]:
390             """
391             Dynamically queries play segments based on filters:
392             - duration_min: float
393             - server: string
394             - receiver: string
395             - event_type: string (e.g. smash, foul)
396             """
397             query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
398             params = [video_id]
399
400             if filters.get("duration_min"):
401                 query += " AND r.duration >= ?"
402                 params.append(filters["duration_min"])
403
404             if filters.get("server"):
405                 query += " AND r.server = ?"
406                 params.append(filters["server"])
407
408             if filters.get("receiver"):
409                 query += " AND r.receiver = ?"
410                 params.append(filters["receiver"])
411
412             if filters.get("event_type"):
413                 # Check if there is an event of a specific type inside this segment
414                 query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
415                 params.append(filters["event_type"])
416
417             segments_list = []
418             with self._connect() as conn:
419                 cursor = conn.cursor()
420                 rows = cursor.execute(query, params).fetchall()
421
422                 for row in rows:
423                     r_dict = dict(row)
424                     r_id = r_dict["id"]
425                     r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")
426
427                     # Fetch associated events
428                     event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
```

```
(r_id,)).fetchall()
                    r_dict["events"] = []
                    for er in event_rows:
                        e_dict = dict(er)
                        e_dict["details"] = json.loads(e_dict["details"] or "{}")
                        r_dict["events"].append(e_dict)

                    segments_list.append(r_dict)

            return segments_list

        def clear_video_data(self, video_id: str):
            with self._connect() as conn:
                conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
                conn.commit()

        # --- Async job store (DEFERRED_HARDENING_PLAN #16) ------------------------------
        # Writers follow the repo convention (swallow + logger.error, return bool); the
        # worker logs loudly on a failed transition so a job can't silently wedge.

        def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
                       player_pool: Optional[List[str]] = None,
                       max_frames: Optional[int] = None,
                       policy: Optional[Dict[str, Any]] = None) -> bool:
            """Insert a new job in state 'queued'. ``policy`` is an optional JSON
ExecutionPolicy
            (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy."""
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "INSERT INTO jobs (id, video_path, segmenter, player_pool,
max_frames, status, policy) "
                        "VALUES (?, ?, ?, ?, ?, 'queued', ?)",
                        (job_id, video_path, segmenter,
                         json.dumps(player_pool) if player_pool is not None else None,
                         max_frames, json.dumps(policy) if policy is not None else None)
                    )
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to create job {job_id}: {e}")
                return False

        def mark_job_running(self, job_id: str) -> bool:
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP, "
                        "progress='starting' WHERE id = ?", (job_id,))
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to mark job {job_id} running: {e}")
                return False

        def set_job_progress(self, job_id: str, progress: str) -> bool:
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "UPDATE jobs SET progress=? WHERE id = ?", (progress, job_id))
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to set progress for job {job_id}: {e}")
                return False
```

```
491
492         def set_job_video_id(self, job_id: str, video_id: str) -> bool:
493             try:
494                 with self._connect() as conn:
495                     conn.cursor().execute(
496                         "UPDATE jobs SET video_id=? WHERE id = ?", (video_id, job_id))
497                     conn.commit()
498                 return True
499             except Exception as e:
500                 logger.error(f"Failed to set video_id for job {job_id}: {e}")
501                 return False
502
503         def mark_job_done(self, job_id: str, result: Dict[str, Any]) -> bool:
504             """Job EXECUTED to completion. The pipeline's own verdict (success/failed
505             validation) is inside `result` ? job status 'done' means 'the worker
finished'."""
506             try:
507                 with self._connect() as conn:
508                     conn.cursor().execute(
509                         "UPDATE jobs SET status='done', progress='finished', result=?, "
510                         "finished_at=CURRENT_TIMESTAMP WHERE id = ?",
511                         (json.dumps(result), job_id))
512                     conn.commit()
513                 return True
514             except Exception as e:
515                 logger.error(f"Failed to mark job {job_id} done: {e}")
516                 return False
517
518         def mark_job_failed(self, job_id: str, error: str) -> bool:
519             """Worker crashed / raised ? infrastructure failure, not a pipeline verdict."""
520             try:
521                 with self._connect() as conn:
522                     conn.cursor().execute(
523                         "UPDATE jobs SET status='failed', error=?, "
524                         "finished_at=CURRENT_TIMESTAMP WHERE id = ?", (error, job_id))
525                     conn.commit()
526                 return True
527             except Exception as e:
528                 logger.error(f"Failed to mark job {job_id} failed: {e}")
529                 return False
530
531         def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
532             """Fetch one job; `result`/`player_pool` JSON deserialized."""
533             with self._connect() as conn:
534                 row = conn.cursor().execute("SELECT * FROM jobs WHERE id = ?",
(job_id,)).fetchone()
535                 if not row:
536                     return None
537                 d = dict(row)
538                 d["result"] = json.loads(d["result"]) if d["result"] else None
539                 d["player_pool"] = json.loads(d["player_pool"]) if d["player_pool"] else
None
540                 d["policy"] = json.loads(d["policy"]) if d.get("policy") else None
541                 return d
542
543         # --- Self-scheduling execution policy (EXECUTION_POLICY.md P2)
------------------------
544         def set_job_policy(self, job_id: str, policy: Optional[Dict[str, Any]]) -> bool:
545             """Attach/replace a job's JSON ExecutionPolicy."""
546             try:
547                 with self._connect() as conn:
548                     conn.cursor().execute(
549                         "UPDATE jobs SET policy=? WHERE id = ?",
550                         (json.dumps(policy) if policy is not None else None, job_id))
551                     conn.commit()
```

```
552                 return True
553             except Exception as e:
554                 logger.error(f"Failed to set policy for job {job_id}: {e}")
555                 return False
556 
557         def set_job_waiting(self, job_id: str, delay_sec: float, progress: Optional[str] =
None) -> bool:
558             """Self-schedule: park a job in 'waiting' until ``delay_sec`` from now,
releasing the
559             worker. `claim_due_job` re-picks it when due ? cooperative, no master.
``next_poll_at`` is
560             computed in SQLite's own UTC format so it compares directly to
CURRENT_TIMESTAMP."""
561             try:
562                 with self._connect() as conn:
563                     conn.cursor().execute(
564                         "UPDATE jobs SET status='waiting', "
565                         "next_poll_at=datetime('now', ?), "
566                         "progress=COALESCE(?, progress) WHERE id = ?",
567                         (f"+{max(0, int(delay_sec))} seconds", progress, job_id))
568                     conn.commit()
569                 return True
570             except Exception as e:
571                 logger.error(f"Failed to set job {job_id} waiting: {e}")
572                 return False
573 
574         def seconds_until_next_due(self) -> Optional[float]:
575             """Seconds until the soonest parked ('waiting') job becomes due, so the worker
can
576             schedule a wake-up (EXECUTION_POLICY.md P3 self-scheduling). Returns 0.0 if one
is
577             already due, or None if nothing is waiting. A NULL ``next_poll_at`` ? due now
(0.0)."""
578             try:
579                 with self._connect() as conn:
580                     row = conn.cursor().execute(
581                         "SELECT MIN(CASE WHEN next_poll_at IS NULL THEN 0 ELSE "
582                         "  (julianday(next_poll_at) - julianday('now')) * 86400.0 END) AS
secs "
583                         "FROM jobs WHERE status='waiting'").fetchone()
584                 if row is None or row["secs"] is None:
585                     return None
586                 return max(0.0, float(row["secs"]))
587             except Exception as e:
588                 logger.error(f"Failed to compute next-due delay: {e}")
589                 return None
590 
591         def claim_due_job(self) -> Optional[Dict[str, Any]]:
592             """Atomically claim ONE runnable job ? 'queued', or 'waiting' whose
``next_poll_at`` is due ?
593             flipping it to 'running'. Compare-and-set on (id, status) so concurrent workers
never
594             double-claim (the DB row-claim is the only coordination needed; no master).
Earliest-due
595             first. Returns the claimed job dict, or None if nothing is runnable."""
596             try:
597                 with self._connect() as conn:
598                     cur = conn.cursor()
599                     row = cur.execute(
600                         "SELECT id, status FROM jobs WHERE status='queued' "
601                         "OR (status='waiting' AND (next_poll_at IS NULL OR next_poll_at <=
CURRENT_TIMESTAMP)) "
602                         "ORDER BY COALESCE(next_poll_at, created_at) ASC LIMIT
1").fetchone()
603                     if not row:
```

```
                        return None
                    cur.execute(
                        "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP "
                        "WHERE id=? AND status=?", (row["id"], row["status"]))
                    conn.commit()
                    if cur.rowcount != 1:
                        return None                    # lost the race to a concurrent
worker
                return self.get_job(row["id"])
            except Exception as e:
                logger.error(f"Failed to claim a due job: {e}")
                return None

        def fail_stale_jobs(self, reason: str = "interrupted by server restart") -> int:
            """Crash recovery: any job still queued/running from a previous process is dead
            (the executor is in-process). Mark failed so clients see a terminal state, not a
            hang. Called once at startup. Returns the number of jobs swept."""
            try:
                with self._connect() as conn:
                    cur = conn.cursor()
                    cur.execute(
                        "UPDATE jobs SET status='failed', error=?,
finished_at=CURRENT_TIMESTAMP "
                        "WHERE status IN ('queued', 'running')", (reason,))
                    conn.commit()
                    return cur.rowcount
            except Exception as e:
                logger.error(f"Failed to sweep stale jobs: {e}")
                return 0

        # --- Per-user model store (PERSONALIZATION_PLAN.md §2, v5) ------------------------
        # Groundwork ONLY: these persist/load the per-user model row. NOTHING on the served
        # pipeline reads them yet ? the serving bridge (backend/pipeline/personalization/
        # serving.py) resolves them, and wiring that bridge into the production decision
seam
        # (`fusion_hybrid._select_for_handoff`) is the NEXT, owner-gated PR. NULL user_id =
        # the local install. `fusion_config` / `classifier_json` / `promotion_evidence` are
        # JSON-serialised dicts; the rest are scalar columns.

        def upsert_user_model(self, user_id: Optional[str], *, version: int = 1,
                              baseline_version: Optional[str] = None,
                              feature_schema_hash: Optional[str] = None,
                              fusion_config: Optional[Dict[str, Any]] = None,
                              classifier_json: Optional[Dict[str, Any]] = None,
                              promotion_evidence: Optional[Dict[str, Any]] = None,
                              n_videos_at_fit: int = 0,
                              is_active: bool = False) -> Optional[int]:
            """Insert or replace ONE per-user model row keyed by ``(user_id, version)``.

            Idempotent on that key (UPSERT, not INSERT-OR-REPLACE: REPLACE would re-allocate
the
            rowid). When ``is_active`` is True this row becomes the user's single active
model ?
            any previously-active row for the SAME user is first cleared, so the
            ``idx_user_models_one_active`` partial-unique invariant always holds (one active
model
            per user). Returns the row id, or None on failure.
            """
            try:
                with self._connect() as conn:
                    cur = conn.cursor()
                    if is_active:
                        # Demote the current active row first (NULL user_id = local: IS NULL
match).
                        if user_id is None:
```

```
                            cur.execute(
                                "UPDATE user_models SET is_active = 0 "
                                "WHERE user_id IS NULL AND is_active = 1 AND version != ?",
                                (version,))
                        else:
                            cur.execute(
                                "UPDATE user_models SET is_active = 0 "
                                "WHERE user_id = ? AND is_active = 1 AND version != ?",
                                (user_id, version))
                    cur.execute(
                        """INSERT INTO user_models
                           (user_id, version, baseline_version, feature_schema_hash,
                            fusion_config, classifier_json, promotion_evidence,
                            n_videos_at_fit, is_active)
                           VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
                           ON CONFLICT(user_id, version) DO UPDATE SET
                             baseline_version=excluded.baseline_version,
                             feature_schema_hash=excluded.feature_schema_hash,
                             fusion_config=excluded.fusion_config,
                             classifier_json=excluded.classifier_json,
                             promotion_evidence=excluded.promotion_evidence,
                             n_videos_at_fit=excluded.n_videos_at_fit,
                             is_active=excluded.is_active""",
                        (user_id, version, baseline_version, feature_schema_hash,
                         json.dumps(fusion_config) if fusion_config is not None else None,
                         json.dumps(classifier_json) if classifier_json is not None else
None,
                         json.dumps(promotion_evidence) if promotion_evidence is not None
else None,
                         n_videos_at_fit, 1 if is_active else 0))
                    conn.commit()
                    rid = cur.execute(
                        "SELECT id FROM user_models WHERE "
                        + ("user_id IS NULL" if user_id is None else "user_id = ?")
                        + " AND version = ?",
                        ((version,) if user_id is None else (user_id, version))).fetchone()
                    return int(rid[0]) if rid else None
            except Exception as e:
                logger.error(f"Failed to upsert user_model (user={user_id}, v={version}):
{e}")
                return None

        def get_active_user_model(self, user_id: Optional[str]) -> Optional[Dict[str, Any]]:
            """Return the single active per-user model row for ``user_id`` (NULL = local
install),
            with ``fusion_config`` / ``classifier_json`` / ``promotion_evidence`` JSON
deserialised,
            or None if the user has no active model. Does not touch the served pipeline."""
            with self._connect() as conn:
                if user_id is None:
                    row = conn.cursor().execute(
                        "SELECT * FROM user_models WHERE user_id IS NULL AND is_active =
1").fetchone()
                else:
                    row = conn.cursor().execute(
                        "SELECT * FROM user_models WHERE user_id = ? AND is_active = 1",
                        (user_id,)).fetchone()
                if not row:
                    return None
                d = dict(row)
                for col in ("fusion_config", "classifier_json", "promotion_evidence"):
                    d[col] = json.loads(d[col]) if d[col] else None
                return d

```

16. assistant (2026-06-18T01:33:44.599Z)

The DB layer is defensively coded. Now let me check the config loader (parses persisted config.json) and the segment-consuming code paths in the stitcher/compiler. Let me also look at how `query_play_segments` results flow ? in `compile_highlights`, `s["start_time"]` and `s["end_time"]` are accessed (these come from DB columns, always present, fine).

Let me look at the config loader and the eval JSON loaders, which read external label/fixture files.

17. user (2026-06-18T01:33:45.659Z)

```
1       """Configuration loader and saver.
2
3       Provides a single place to load validated AppConfig with built-in defaults
4       merged with optional file overrides (config.json).
5
6       This replaces scattered json.load + .get() patterns throughout the codebase.
7       """
8
9       from __future__ import annotations
10
11      import json
12      import logging
13      from pathlib import Path
14      from typing import Any
15
16      from pydantic import BaseModel
17
18      from .models import AppConfig
19
20      logger = logging.getLogger(__name__)
21
22      DEFAULT_CONFIG_PATH = Path("config.json")
23
24
25      def _unknown_key_paths(data: dict[str, Any], model_cls: type[BaseModel], prefix: str =
"") -> list[str]:
26          """Dotted paths in `data` that the typed config will not honor.
27
28          Two silent failure modes exist without this check: a typo'd TOP-LEVEL key is
29          kept by AppConfig's ``extra="allow"`` but never read by typed code, and a
30          typo'd key INSIDE a section is silently dropped by Pydantic. Either way the
31          user's intent is lost ? collect every such key so load_config can warn.
32          """
33          unknown: list[str] = []
34          fields = model_cls.model_fields
35          for key, val in data.items():
36              if key not in fields:
37                  unknown.append(prefix + key)
38                  continue
39              ann = fields[key].annotation
40              if isinstance(val, dict) and isinstance(ann, type) and issubclass(ann,
BaseModel):
41                  unknown.extend(_unknown_key_paths(val, ann, f"{prefix}{key}."))
42          return unknown
43
44
45      def _get_builtin_defaults() -> dict[str, Any]:
46          """Return a plain dict of the current built-in defaults.
47
48          The AppConfig model is the single source of truth for defaults.
49          """
50          return AppConfig().model_dump(mode="json")
51
```

```
52
53      def load_config(path: str | Path | None = None) -> AppConfig:
54          """Load configuration with the following precedence:
55
56          1. Built-in defaults defined in the Pydantic models.
57          2. Values from the JSON file (if present and readable).
58          3. (Future) environment / CLI overrides.
59
60          Missing file or unreadable file ? returns a fully defaulted AppConfig.
61          Unknown keys in the file are tolerated (extra="allow" on the model).
62          """
63          cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
64
65          file_data: dict[str, Any] = {}
66          if cfg_path.exists():
67              try:
68                  with open(cfg_path, "r", encoding="utf-8") as f:
69                      file_data = json.load(f)
70              except Exception as exc:
71                  # Non-fatal: continue with defaults + whatever we could parse.
72                  # In production we might log here.
73                  logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial
data.")
74
75          if file_data:
76              unknown = _unknown_key_paths(file_data, AppConfig)
77              if unknown:
78                  logger.warning(
79                      "[CONFIG WARNING] %s contains keys the typed config does not "
80                      "recognize (typo'd or removed knobs ? top-level extras are kept "
81                      "but unread; unknown section keys are silently dropped): %s",
82                      cfg_path, ", ".join(sorted(unknown)),
83                  )
84
85          # Merge: start from defaults, overlay file data (Pydantic will validate/coerce).
86          merged = {**_get_builtin_defaults(), **file_data}
87
88          return AppConfig.model_validate(merged)
89
90
91      def save_default_config(path: str | Path | None = None) -> Path:
92          """Write a fresh default config.json.
93
94          Used by scripts/doctor.py diagnostics and the --setup CLI path.
95          Overwrites if the file exists.
96          """
97          cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
98          cfg = AppConfig()
99          cfg_path.parent.mkdir(parents=True, exist_ok=True)
100         cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
101         return cfg_path
102
103
104     # Backwards-compatible alias used by some older call sites during transition.
105     def get_default_config() -> dict[str, Any]:
106         """Return defaults as a plain dict (for transitional use only)."""
107         return _get_builtin_defaults()
108
```

18. assistant (2026-06-18T01:33:49.052Z)

Config loader is solid. Now let me focus on the eval modules that read JSON ground-truth/label files and DB rows ? these are the persisted-data readers. Let me look at the gt_loader, annotations, and golden_fixtures which parse external label files. Also the stitcher compiler which consumes segment data.

19. user (2026-06-18T01:33:49.708Z)

```
1       """Canonical ground-truth interval loader ? THE one GT/golden-CSV reader (ADR-008
prereq).
2
3       The repo historically carried two incompatible loaders:
4       - ``annotations.load_annotations``: positional ``start,end`` (the collector's
5         ``curate export`` format), raises on bad rows;
6       - ``calibrate_wasb.load_golden_gts``: DictReader requiring ``start_time/end_time``
7         (the rally-annotator golden format), silently skipping bad rows ? fed 7 modules.
8
9       A file in one convention fed to the other loader yielded zero/garbage intervals with no
10      error. This module accepts BOTH conventions (header-mapped, else positional), so every
11      quantity the ship-gate depends on (gt_hash, LOVO scores, the future consent fingerprint)
12      flows through one reader. Both legacy entry points now delegate here.
13
14      Accepted shapes (one rally per row; blank lines and ``#`` comments ignored):
15      - header with ``start``/``end`` OR ``start_time``/``end_time`` columns (any other
16        columns ? rally_number, ending_reason, sport, ? ? are ignored);
17      - no header: columns 0,1 positionally.
18      Timestamps: decimal seconds or ``MM:SS`` / ``HH:MM:SS``
(``annotations.parse_timestamp``).
19      """
20      import csv
21      import logging
22      from typing import List, Optional, Tuple
23
24      from backend.eval.annotations import parse_timestamp
25
26      logger = logging.getLogger(__name__)
27
28      Interval = Tuple[float, float]
29
30      _START_NAMES = ("start", "start_time")
31      _END_NAMES = ("end", "end_time")
32
33
34      def _header_columns(row: List[str]) -> Optional[Tuple[int, int]]:
35          """If `row` is a header naming start/end columns, return their indices, else
None."""
36          cells = [(c or "").strip().lower() for c in row]
37          start_idx = next((i for i, c in enumerate(cells) if c in _START_NAMES), None)
38          end_idx = next((i for i, c in enumerate(cells) if c in _END_NAMES), None)
39          if start_idx is not None and end_idx is not None:
40              return start_idx, end_idx
41          return None
42
43
44      def load_gt_intervals(csv_path: str, *, strict: bool = True) -> List[Interval]:
45          """Load GT rally intervals from either golden-CSV convention.
46
47          strict=True (corpus/gate use): raise ValueError with file:line on any malformed row
48          (bad timestamp, end <= start, negative start) so labeling mistakes fail loudly.
49          strict=False (legacy tuning-driver behavior): skip bad rows, but LOG how many were
50          skipped ? a silently shrinking GT set flatters precision.
51
52          Returns intervals sorted by start; overlapping GT intervals are logged as a warning
53          (they break one-to-one matching and usually indicate a labeling error).
54          """
55          intervals: List[Interval] = []
56          skipped = 0
57          cols: Optional[Tuple[int, int]] = None   # (start_idx, end_idx); None until detected
58          saw_header = False
59
60          with open(csv_path, newline="", encoding="utf-8-sig") as f:
```

```
61              for lineno, row in enumerate(csv.reader(f), start=1):
62                  if not row or not (row[0] or "").strip() or row[0].strip().startswith("#"):
63                      continue
64                  if cols is None and not saw_header:
65                      header = _header_columns(row)
66                      if header is not None:
67                          cols = header
68                          saw_header = True
69                          continue            # consume the header row
70                      cols = (0, 1)           # headerless: positional start,end
71                  assert cols is not None     # every branch above assigns it
72                  si, ei = cols
73                  try:
74                      if len(row) <= max(si, ei):
75                          raise ValueError(f"expected at least {max(si, ei) + 1} columns, got
{row!r}")
76                      start = parse_timestamp(row[si])
77                      end = parse_timestamp(row[ei])
78                      if start < 0:
79                          raise ValueError(f"negative start ({start})")
80                      if start >= end:
81                          raise ValueError(f"start ({start}) >= end ({end})")
82                  except ValueError as e:
83                      if strict:
84                          raise ValueError(f"{csv_path}:{lineno}: {e}") from e
85                      skipped += 1
86                      continue
87                  intervals.append((start, end))
88
89          if skipped:
90              logger.warning("%s: skipped %d malformed GT row(s) (strict=False)", csv_path,
skipped)
91
92          intervals.sort(key=lambda iv: iv[0])
93          for (s1, e1), (s2, e2) in zip(intervals, intervals[1:]):
94              if s2 < e1:
95                  logger.warning("%s: overlapping GT intervals (%.3f-%.3f vs %.3f-%.3f) ? "
96                                 "labeling error? Overlaps break one-to-one matching.",
97                                 csv_path, s1, e1, s2, e2)
98          return intervals
99
```

20. assistant (2026-06-18T01:33:52.948Z)

Well-guarded. Let me look at the annotations file_provider and automated_provider which read annotation files, and the golden_fixtures loader. Let me also examine the stitcher compiler which receives time_pairs.

21. user (2026-06-18T01:33:54.131Z)

```
1       """Golden Regression Fixtures (M1) ? video-free, CONTROL-only characterization harness.
2
3       `docs/GOLDEN_REGRESSION_FIXTURES.md` (§4, §6, §7). This module is the **single shared
4       replay code path** used by both the freeze tool and the characterization gate, and it
5       deliberately stays on the **CONTROL path** (D5 in that doc): the learned
6       variants train a numpy/OpenBLAS logistic regression whose near-threshold decisions flip
7       across CPUs, so they can never anchor a cross-machine byte/parse snapshot. Here we
replay
8       only `experiment.CONTROL` (`is_learned() == False`), which keeps all candidate
intervals,
9       merges overlaps, and **avoids numpy EXECUTION on the replay path** ? i.e. no numpy array
/
10      LogisticRegression math runs while replaying. (numpy is still *imported* transitively
via
11      `experiment` ? the learned-variant classifier module and MUST be installed; the CONTROL
```

```
    path simply never calls into it.)

    The replay reuses the already-shipped eval stack VERBATIM::

        parse_trajectory_csv ? distill_local.build_candidates(proposal=pinned)
            ? VideoCandidates(name, intervals, features={5 shuttle cols},
gts=load_gt_intervals)
            ? experiment.predict(CONTROL, vc) ? metrics.score(preds, gts, iou)

    Tier-0 / privacy invariants (D3, §4c, §6 of the doc), enforced here by construction:

    * Fixtures are **synthetic** ? deterministic formulae, NO randomness ? so they carry
zero
      provenance/licensing risk and are safe to commit publicly (owner Q7 "synthetic-only").
    * Every fixture is tagged ``kind="synthetic"``, ``minors_free=true``,
``license="synthetic"``.
    * ``expected.json`` carries **only the 5 shuttle feature columns** ? there is, by
design, no
      person / pose / gait / face / audio field anywhere in the schema (positive-assertion
test).
    * The windowing params ``(vt, mg, mwd)`` are **pinned inside each fixture's provenance**
and
      passed explicitly to ``build_candidates`` as a raw 3-tuple ? we do NOT import or rely
on the
      named ``windowing.SERVED`` constant / ``WindowingPreset``, so a future SERVED retune
cannot
      silently churn fixtures.
    * ``max_win`` (W1 bounded-windowing) is **RECORDED** in provenance/manifest but NOT
applied by
      the replay: ``distill_local.build_candidates`` (shared code, intentionally not
modified here)
      takes only the 3-tuple ``(vt, mg, mwd)`` and
``TrackNetRunner.trajectory_to_action_windows``
      has no max-window arg, so these fixtures characterize the **W1-OFF** path by design.
The pinned
      ``max_win`` is a forward-looking annotation, NOT a knob the gate exercises ? do not
assume it
      tracks a future SERVED W1 flip (it would need new wiring + a re-freeze to take
effect).

    §6 honest-limits caveat (carried in the CLI / test messages, not only the docs): a green
    suite proves only that the deterministic post-trajectory transforms are unchanged for
the
    frozen synthetic cases ? NOT that the detector/decoder/AI handoff/DB are correct, and
NOT
    that rally detection is good. Synthetic-tier numbers measure plumbing correctness, never
    field quality (never cite them in QUALITY_ITERATIONS.md / the paper).

    Offline + deterministic. The replay imports `tracknet_runner` (stdlib-only for the
    parse/window math ? no TF) and reaches no numpy/cv2 EXECUTION on the CONTROL path, but
it is
    NOT a zero-third-party-import module: importing `experiment` transitively imports numpy
(via
    the learned-variant classifier), so numpy must be installed even though the CONTROL
replay
    never calls into it.
    """
    from __future__ import annotations

    import argparse
    import json
    import math
    import os
    import re
    import shutil
```

```
58      import sys
59      import tempfile
60      from dataclasses import dataclass
61      from pathlib import Path
62      from typing import Any, Dict, List, Optional, Sequence, Tuple
63
64      # --- shipped eval stack, reused verbatim (CONTROL / pure-stdlib path only)
------------
65      from backend.eval import distill_local
66      from backend.eval.experiment import CONTROL, VideoCandidates, predict
67      from backend.eval.gt_loader import load_gt_intervals
68      from backend.eval.metrics import score
69      from backend.eval.regression import gt_hash
70      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
71
72      Interval = Tuple[float, float]
73
74      #: Bump when the fixture/expected schema changes shape. Loaders refuse a fixture whose
75      #: ``schema_version`` exceeds this (forward-incompatible), per the schema-drift guard.
76      CURRENT_SCHEMA = 1
77
78      #: Repo-root-relative fixtures dir, resolved from THIS file (never cwd) so the harness
works
79      #: from any working directory / a fresh clone. ``parents[2]`` = backend/eval ? backend ?
root.
80      FIXTURES_DIR: Path = Path(__file__).resolve().parents[2] / "eval_baselines" / "fixtures"
81
82      #: The pinned SERVED windowing the synthetic fixtures freeze at: (velocity_thresh,
merge_gap,
83      #: min_window_duration). Mirrors the historical SERVED values (0.02, 2.0, 2.0) but is a
LITERAL
84      #: here on purpose ? the spec forbids depending on the named ``windowing.SERVED``
constant so a
85      #: future retune of SERVED can never silently re-baseline a committed fixture. Each
fixture also
86      #: re-states these in its provenance; the replay reads them from there.
87      PINNED_VT = 0.02
88      PINNED_MERGE_GAP = 2.0
89      PINNED_MIN_WINDOW = 2.0
90      #: W1 (bounded-windowing) max-window seconds. 0 = OFF, matching the SERVED default.
NOTE: this is
91      #: only RECORDED (provenance/manifest) ? it is NOT applied by the replay.
``_pinned_proposal``
92      #: returns just ``(vt, mg, mwd)`` and ``distill_local.build_candidates`` (shared code,
deliberately
93      #: NOT modified by this harness) takes no max-window arg, so the fixtures characterize
the W1-OFF
94      #: path. Do NOT assume this value tracks a future SERVED W1 flip ? that would need new
wiring.
95      PINNED_MAX_WINDOW = 0.0
96
97      #: The 5 fps/resolution-invariant shuttle feature columns produced by distill_local. Re-
stated
98      #: here (not imported as a mutable alias) so a fixture's expected.json is pinned to
exactly these.
99      SHUTTLE_FEATURES: Tuple[str, ...] = (
100         "duration", "peak_velocity", "mean_velocity", "active_ratio", "net_crossings",
101     )
102     assert tuple(distill_local.FEATURE_NAMES) == SHUTTLE_FEATURES, (
103         "distill_local.FEATURE_NAMES drifted from the pinned shuttle feature set"
104     )
105
106     #: Forbidden key substrings ? biometric / identity / non-shuttle streams that MUST NOT
appear
107     #: anywhere in a Tier-0 fixture (D3 / §4c). The privacy assertion scans for these.
```

```
    FORBIDDEN_KEYS: Tuple[str, ...] = ("person", "pose", "gait", "face", "audio", "player")

    _FLOAT_ROUND_DP = 6  # all written floats rounded to this many decimals (determinism)
    _FLOAT_TOL = 1e-6    # abs tolerance when parse-comparing recomputed vs committed floats

    #: A 32-hex-char MD5 (the ``video_id`` shape, ``compute_video_id``) ? the leak-scan
refuses to
    #: emit a fixture whose written text contains one (it would be an attribution back-
channel, §4c/§5).
    _MD5_RE = re.compile(r"\b[0-9a-fA-F]{32}\b")

    #: Strict slug charset for a real-fixture ``--label`` ? lowercase alnum, ``_``/``-`` (no
spaces,
    #: no punctuation, no uppercase). A free-text label ("Lin Dan vs Lee, India Open 2024")
is an
    #: attribution back-channel that ``_scan_forbidden`` does NOT catch (it isn't a
FORBIDDEN_KEY / MD5
    #: / source-path token), so it is rejected up-front by construction (red-team: free-text
id leak).
    _LABEL_SLUG_RE = re.compile(r"^[a-z0-9][a-z0-9_\-]*$")


    # ============================================================================
    # Synthetic trajectory generation (deterministic ? NO random)
    # ============================================================================


    @dataclass(frozen=True)
    class RallySpec:
        """One in-flight rally segment: the shuttle oscillates across the net midline.

        ``start_frame`` ? first frame index; ``n_frames`` ? length; ``period`` ? oscillation
        period in frames (governs net-crossing count); ``amplitude`` ? px excursion from the
        net midline along the play axis. Deterministic sine motion ? fixed
velocity/crossings.
        """

        start_frame: int
        n_frames: int
        period: float = 30.0
        amplitude: float = 400.0


    @dataclass(frozen=True)
    class GapSpec:
        """Dead-time between rallies ? what separates one window from the next.

        ``kind="still"`` ? shuttle visible but resting (velocity ? 0 ? no active frames);
        ``kind="occluded"`` ? shuttle not visible (Visibility=0 ? trajectory broken). Both
        end an active run so the next rally forms a distinct window.
        """

        start_frame: int
        n_frames: int
        kind: str = "still"  # "still" | "occluded"


    @dataclass(frozen=True)
    class FixtureSpec:
        """A full synthetic scenario: a frame-indexed sequence of rallies and gaps + its GT.

        ``gts`` are the hand-authored ground-truth rally intervals (seconds) ? the synthetic
        "labels.csv". For a synthetic fixture they line up with the rallies by construction,
        which is exactly what makes the quality-floor a *plumbing-correctness* check, not a
        field-quality one (§6).
```

```
        """
        slug: str
        description: str
        fps: float
        frame_width: float
        frame_height: float
        segments: Tuple[Any, ...]  # ordered (RallySpec | GapSpec)
        gts: Tuple[Interval, ...]
        net_midline: float = 640.0


    def _rally_points(spec: RallySpec, net_midline: float, frame_height: float
                      ) -> List[Tuple[int, int, float, float]]:
        """Deterministic in-flight shuttle: sine oscillation across the net midline (play
axis = x)."""
        rows: List[Tuple[int, int, float, float]] = []
        y_mid = frame_height / 2.0
        for i in range(spec.n_frames):
            x = net_midline + spec.amplitude * math.sin(2.0 * math.pi * i / spec.period)
            # A small orthogonal wobble keeps y-spread < x-spread so the play axis is
unambiguously x.
            y = y_mid + 30.0 * math.sin(2.0 * math.pi * i / (spec.period / 2.0))
            rows.append((spec.start_frame + i, 1, x, y))
        return rows


    def _gap_points(spec: GapSpec, net_midline: float, frame_height: float
                    ) -> List[Tuple[int, int, float, float]]:
        """Dead-time rows: resting-visible (still) or absent (occluded)."""
        rows: List[Tuple[int, int, float, float]] = []
        y_mid = frame_height / 2.0
        for i in range(spec.n_frames):
            if spec.kind == "occluded":
                rows.append((spec.start_frame + i, 0, 0.0, 0.0))
            elif spec.kind == "still":
                rows.append((spec.start_frame + i, 1, net_midline, y_mid))
            else:  # pragma: no cover - guarded by builder
                raise ValueError(f"unknown gap kind {spec.kind!r} (use 'still'|'occluded')")
        return rows


    def make_synthetic_trajectory(spec: FixtureSpec) -> List[Tuple[int, int, float, float]]:
        """Build the deterministic ``(Frame, Visibility, X, Y)`` rows for a synthetic
scenario."""
        rows: List[Tuple[int, int, float, float]] = []
        for seg in spec.segments:
            if isinstance(seg, RallySpec):
                rows.extend(_rally_points(seg, spec.net_midline, spec.frame_height))
            elif isinstance(seg, GapSpec):
                rows.extend(_gap_points(seg, spec.net_midline, spec.frame_height))
            else:  # pragma: no cover - guarded by dataclass shape
                raise TypeError(f"unknown segment type {type(seg).__name__}")
        rows.sort(key=lambda r: r[0])
        return rows


    def write_trajectory_csv(rows: Sequence[Tuple[int, int, float, float]], path: Path) ->
None:
        """Write trajectory rows as the TrackNet CSV (``Frame,Visibility,X,Y``), LF + 6dp
floats."""
        lines = ["Frame,Visibility,X,Y"]
        for frame, vis, x, y in rows:
            lines.append(f"{int(frame)},{int(vis)},{_fmt(x)},{_fmt(y)}")
        path.write_text("\n".join(lines) + "\n", encoding="utf-8", newline="\n")
```

```
226
227
228     def write_labels_csv(gts: Sequence[Interval], path: Path) -> None:
229         """Write GT intervals as a ``start,end`` labels CSV (the strict gt_loader format),
LF."""
230         lines = ["start,end"]
231         for s, e in gts:
232             lines.append(f"{_fmt(s)},{_fmt(e)}")
233         path.write_text("\n".join(lines) + "\n", encoding="utf-8", newline="\n")
234
235
236     # ============================================================================================
237     # Built-in synthetic scenarios (the 3 committed slugs)
238     # ============================================================================================
239
240
241     def _fr(seconds: float, fps: float) -> int:
242         return int(round(seconds * fps))
243
244
245     def builtin_specs() -> List[FixtureSpec]:
246         """The committed synthetic scenarios. Deterministic; chosen so each exercises a
distinct
247         windowing behaviour (one window / dead-time split / occlusion split)."""
248         fps = 30.0
249         fw, fh = 1280.0, 720.0
250         net = 640.0
251
252         # 1) clean_single_rally: one ~4s in-flight rally ? exactly 1 candidate window,
crossings>0.
253         single = FixtureSpec(
254             slug="clean_single_rally",
255             description="One continuous ~4s rally; shuttle oscillates across the net ? 1
window.",
256             fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
257             segments=(RallySpec(start_frame=0, n_frames=120),),
258             # Window is [0.033, 3.967]; GT is the rally span the synthetic motion fills.
259             gts=((0.0, 4.0),),
260         )
261
262         # 2) multi_rally_with_deadtime: rally, 3s STILL dead-time, rally ? 2 windows split
by the
263         #    low-velocity gap (> merge_gap=2.0s) ? proves dead-time separates windows.
264         r0 = RallySpec(start_frame=0, n_frames=120)
265         gap_still = GapSpec(start_frame=120, n_frames=_fr(3.0, fps), kind="still")
266         r1 = RallySpec(start_frame=120 + _fr(3.0, fps), n_frames=120)
267         multi = FixtureSpec(
268             slug="multi_rally_with_deadtime",
269             description="Two ~4s rallies separated by 3s of still (resting-shuttle) dead-
time ? 2 windows.",
270             fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
271             segments=(r0, gap_still, r1),
272             gts=((0.0, 4.0), (7.0, 11.0)),
273         )
274
275         # 3) occlusion_break: rally, 3s OCCLUDED (Visibility=0), rally ? 2 windows split by
the
276         #    visibility break ? proves occlusion (not just low velocity) separates windows.
277         o0 = RallySpec(start_frame=0, n_frames=120)
278         gap_occ = GapSpec(start_frame=120, n_frames=_fr(3.0, fps), kind="occluded")
279         o1 = RallySpec(start_frame=120 + _fr(3.0, fps), n_frames=120)
280         occ = FixtureSpec(
281             slug="occlusion_break",
282             description="Two ~4s rallies separated by 3s of shuttle occlusion (Visibility=0)
? 2 windows.",
```

```
            fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
            segments=(o0, gap_occ, o1),
            gts=((0.0, 4.0), (7.0, 11.0)),
        )

        return [single, multi, occ]


    def _spec_by_slug() -> Dict[str, FixtureSpec]:
        return {s.slug: s for s in builtin_specs()}


    # ============================================================================
    # The shared replay (CONTROL / pure-stdlib path)
    # ============================================================================


    def _fmt(x: float) -> str:
        """Render a float with up to 6dp, no scientific notation, ``-0.0`` normalised to
``0.0``."""
        r = round(float(x) + 0.0, _FLOAT_ROUND_DP)
        if r == 0.0:
            r = 0.0
        return f"{r:.6f}"


    def _round(x: float) -> float:
        r = round(float(x) + 0.0, _FLOAT_ROUND_DP)
        return 0.0 if r == 0.0 else r


    def _pinned_proposal(prov: Optional[Dict[str, Any]]) -> Tuple[float, float, float]:
        """The windowing proposal tuple ``(vt, mg, mwd)``, read from the fixture provenance
when present
        (the pinned params travel with the fixture), else the module-level pinned defaults.
NEVER
        ``windowing.SERVED``. NOTE: the provenance ``max_win`` (W1) is intentionally NOT
read here ?
        ``build_candidates`` takes only this 3-tuple, so the replay characterizes the W1-OFF
path; the
        pinned ``max_win`` is a recorded annotation, not a knob the gate applies (see
PINNED_MAX_WINDOW)."""
        if prov and "windowing" in prov:
            w = prov["windowing"]
            return (float(w["vt"]), float(w["mg"]), float(w["mwd"]))
        return (PINNED_VT, PINNED_MERGE_GAP, PINNED_MIN_WINDOW)


    def replay_fixture(slug_or_dir: Any) -> Dict[str, Any]:
        """Replay one fixture through the shipped CONTROL path; return its recomputed
snapshot.

        Resolves ``slug_or_dir`` (a slug string or a fixture directory ``Path``) to its
committed
        ``trajectory.csv`` + ``labels.csv`` + ``provenance.json``, then:

            parse_trajectory_csv ? build_candidates(proposal=pinned) ? VideoCandidates(5
shuttle
            cols only) ? predict(CONTROL, vc) ? score(preds, gts, iou).

        Returns ``{slug, schema_version, iou, candidates:[{start,end,shuttle:{5 names}}],
        control_segments:[[s,e]], score:{...}, gt_hash}``. CONTROL keeps every candidate
then
        merges overlaps; NO numpy is touched.
        """
```

```
        fdir = _resolve_dir(slug_or_dir)
        slug = fdir.name
        traj_csv = fdir / "trajectory.csv"
        labels_csv = fdir / "labels.csv"
        prov = _read_json(fdir / "provenance.json") if (fdir / "provenance.json").is_file()
else None

        fps, fw = _fps_fw(prov, slug)
        iou = float(prov["iou"]) if (prov and "iou" in prov) else 0.5
        proposal = _pinned_proposal(prov)

        intervals, feature_rows = distill_local.build_candidates(
            str(traj_csv), fps=fps, frame_width=fw, proposal=proposal)

        # Build VideoCandidates DIRECTLY with ONLY the 5 shuttle feature columns. We never
call
        # fusion_compare.load_videos / ablation.load_videos ? those read c['person']
unconditionally
        # and would KeyError on these person-free fixtures (HARD CONSTRAINT 1).
        features: Dict[str, List[float]] = {
            name: [row[i] for row in feature_rows] for i, name in
enumerate(SHUTTLE_FEATURES)
        }
        gts = load_gt_intervals(str(labels_csv), strict=True)
        vc = VideoCandidates(name=slug, intervals=intervals, features=features, gts=gts)

        preds = predict(CONTROL, vc)  # CONTROL: keep all ? merge_overlaps; pure-stdlib, no
numpy
        sc = score(preds, gts, iou)

        candidates: List[Dict[str, Any]] = []
        for j, (s, e) in enumerate(intervals):
            shuttle = {name: _round(feature_rows[j][i]) for i, name in
enumerate(SHUTTLE_FEATURES)}
            # net_crossings is conceptually an integer count ? keep it exact (int), per
constraint 2.
            shuttle["net_crossings"] =
int(round(feature_rows[j][SHUTTLE_FEATURES.index("net_crossings")]))
            candidates.append({"start": _round(s), "end": _round(e), "shuttle": shuttle})

        return {
            "slug": slug,
            "schema_version": CURRENT_SCHEMA,
            "iou": _round(iou),
            "candidates": candidates,
            "control_segments": [[_round(s), _round(e)] for s, e in preds],
            "score": {
                "iou_threshold": _round(sc.iou_threshold),
                "num_pred": int(sc.num_pred),
                "num_gt": int(sc.num_gt),
                "true_positives": int(sc.true_positives),
                "false_positives": int(sc.false_positives),
                "false_negatives": int(sc.false_negatives),
                "precision": _round(sc.precision),
                "recall": _round(sc.recall),
                "f1": _round(sc.f1),
                "mean_iou": _round(sc.mean_iou),
                "mean_start_error": _round(sc.mean_start_error),
                "mean_end_error": _round(sc.mean_end_error),
            },
            "gt_hash": gt_hash(gts),
        }


    # ================================================================================
```

```
    # Freeze / refresh (expected.json + provenance.json are correct-by-construction)
    # ============================================================================================


    def _provenance(spec_or_dir: Any, slug: str, fps: float, fw: float, iou: float = 0.5
                    ) -> Dict[str, Any]:
        """The committed provenance block ? pins the windowing + privacy/licensing tags
(§4c, §5)."""
        desc = ""
        if isinstance(spec_or_dir, FixtureSpec):
            desc = spec_or_dir.description
        return {
            "slug": slug,
            "schema_version": CURRENT_SCHEMA,
            "kind": "synthetic",
            "license": "synthetic",
            "minors_free": True,
            "description": desc,
            "fps": _round(fps),
            "frame_width": _round(fw),
            "iou": _round(iou),
            "windowing": {
                "vt": PINNED_VT,
                "mg": PINNED_MERGE_GAP,
                "mwd": PINNED_MIN_WINDOW,
                "max_win": PINNED_MAX_WINDOW,
            },
            "paths": {
                "trajectory": "trajectory.csv",
                "labels": "labels.csv",
                "expected": "expected.json",
            },
            "note": (
                "Synthetic, deterministic, owned (no real footage). Tier-0 / CONTROL-only. "
                "Plumbing-correctness fixture ? NOT a measure of rally-detection quality
(§6)."
            ),
        }


    def freeze_fixture(slug: str, fixtures_dir: Path = FIXTURES_DIR,
                       iou: float = 0.5) -> Dict[str, Any]:
        """Compute ``expected.json`` + ``provenance.json`` for one fixture from its
COMMITTED
        ``trajectory.csv`` + ``labels.csv`` via the replay (so expected is correct-by-
construction).

        Idempotent: rewrites the same bytes given the same committed inputs. Requires the
trajectory
        + labels CSVs to already exist (use ``--freeze-synthetic`` to generate them first).
        """
        fdir = fixtures_dir / slug
        traj_csv, labels_csv = fdir / "trajectory.csv", fdir / "labels.csv"
        if not traj_csv.is_file() or not labels_csv.is_file():
            raise FileNotFoundError(
                f"{slug}: trajectory.csv/labels.csv missing ? run --freeze-synthetic to
generate them")

        # Provenance first (it pins the windowing the replay reads), then replay ? expected.
        spec = _spec_by_slug().get(slug)
        fps, fw = (spec.fps, spec.frame_width) if spec is not None else
_fps_fw_from_prov(fdir, slug)
        prov = _provenance(spec if spec is not None else fdir, slug, fps, fw, iou)
        _write_json(fdir / "provenance.json", prov)

```

```
453         expected = replay_fixture(fdir)
454         _write_json(fdir / "expected.json", expected)
455         return expected
456 
457 
458     def freeze_synthetic(fixtures_dir: Path = FIXTURES_DIR) -> List[str]:
459         """Generate trajectory.csv + labels.csv for every built-in scenario, freeze
expected.json +
460         provenance.json, and (re)write the manifest. Returns the slugs frozen."""
461         specs = builtin_specs()
462         fixtures_dir.mkdir(parents=True, exist_ok=True)
463         slugs: List[str] = []
464         for spec in specs:
465             fdir = fixtures_dir / spec.slug
466             fdir.mkdir(parents=True, exist_ok=True)
467             rows = make_synthetic_trajectory(spec)
468             write_trajectory_csv(rows, fdir / "trajectory.csv")
469             write_labels_csv(spec.gts, fdir / "labels.csv")
470             freeze_fixture(spec.slug, fixtures_dir=fixtures_dir)
471             slugs.append(spec.slug)
472         _write_manifest(specs, fixtures_dir)
473         return slugs
474 
475 
476     def refresh_snapshot(slug: Optional[str] = None, all_slugs: bool = False,
477                          fixtures_dir: Path = FIXTURES_DIR) -> List[str]:
478         """Re-freeze expected.json (+ provenance.json) from the COMMITTED trajectory.csv +
labels.csv ?
479         NO regeneration of the trajectory/labels. ? §6: this can rubber-stamp a real
regression; the
480         mitigation is reviewing the visible diff + recording a reason (the CLI requires
one)."""
481         if all_slugs:
482             targets = [d.name for d in sorted(fixtures_dir.iterdir())
483                        if d.is_dir() and (d / "trajectory.csv").is_file()]
484         elif slug:
485             targets = [slug]
486         else:
487             raise ValueError("refresh_snapshot needs --slug S or --all")
488         for s in targets:
489             freeze_fixture(s, fixtures_dir=fixtures_dir)
490         return targets
491 
492 
493     # ============================================================================
494     # P1 ? De-attribution + minimization: OWNED real trajectory+labels ? committable Tier-0
495     # fixture, reusing the SAME replay_fixture snapshot. (docs/GOLDEN_REGRESSION_FIXTURES.md
496     # §4c anti-attribution, §5 threat model, §7 row P1, §6 honest limits.)
497     # ============================================================================
498 
499 
500     class RefusalError(ValueError):
501         """The provenance/biometric refusal gate (M2) rejected an unsafe freeze.
502 
503         A subclass of ``ValueError`` (so existing ``except ValueError`` paths still catch
it) that
504         names *why* the unsafe public-fixture path was refused. The refusal is the code-
enforced
505         provenance gate (§4c "provenance hard-fail gate", §7 ?): the owner-recorded /
minors-excluded
506         / license flags are human-asserted, but the gate makes the unsafe path impossible to
reach
507         *silently* ? a fixture cannot be written unless all three are affirmatively set.
508         """
509 
```

```
510
511     def _new_surrogate_slug() -> str:
512         """An opaque RANDOM surrogate slug ? ``fx_<16 hex>`` from ``os.urandom`` (§4c).
513
514         Deliberately NOT the MD5 ``video_id`` and NOT ``HMAC(salt, video_id?name)`` ? a
random,
515         NON-reversible id with no algebraic link to the source, so it can't be brute-forced
over a
516         tiny known roster even if a salt leaked. The surrogate?source map (if kept) lives
off-git.
517         """
518         return f"fx_{os.urandom(8).hex()}"
519
520
521     #: The fixed schema filenames the writers ALWAYS emit (as ``paths`` literals). A source-
path token
522     #: that EXACTLY equals one of these is NOT a leak (the bytes would be written
regardless), so it is
523     #: excluded. Matched by exact membership, NEVER substring containment ? a substring rule
would let
524     #: a real source named ``label.csv`` (stem ``label`` ? ``labels.csv``) leak unnoticed
(red-team #4).
525     _SCHEMA_FILENAME_LITERALS: Tuple[str, ...] = (
526         "trajectory", "trajectory.csv", "labels", "labels.csv", "expected", "expected.json",
527         "provenance", "provenance.json",
528     )
529
530
531     def _scan_forbidden(text: str, *, where: str, source_paths: Sequence[str]) -> None:
532         """POSITIVE privacy assertion (§4c, red-team ?): raise if ``text`` leaks anything
attributable.
533
534         Scans the written JSON *text* for (a) any ``FORBIDDEN_KEYS`` biometric/identity
substring,
535         (b) a 32-hex MD5 (the ``video_id`` shape), or (c) any identifying token of a source
path
536         (full path / basename / stem) ? excluding the fixed schema filename literals the
writers
537         always emit. The trajectory is shuttle-only by nature; this guards against a
*regression*
538         re-introducing a person/audio/pose stream or an id/path back-channel ? never the
silent default.
539         """
540         low = text.lower()
541         for key in FORBIDDEN_KEYS:
542             if key in low:
543                 raise RefusalError(
544                     f"{where}: forbidden key substring {key!r} found in written output ? "
545                     f"Tier-0 fixtures are shuttle-only; person/pose/gait/face/audio/player
MUST NOT "
546                     f"appear (docs/GOLDEN_REGRESSION_FIXTURES.md §4c).")
547         m = _MD5_RE.search(text)
548         if m:
549             raise RefusalError(
550                 f"{where}: a 32-hex MD5 ({m.group(0)!r}) found in written output ? that is
the "
551                 f"video_id shape and an attribution back-channel; never write it (§4c/§5).")
552         for sp in source_paths:
553             if not sp:
554                 continue
555             p = Path(sp)
556             for token in (sp, p.name, p.stem):
557                 t = (token or "").strip().lower()
558                 # Skip ONLY a token that EXACTLY equals a schema-literal the writers always
emit
```

```
559                 # (e.g. a source named ``trajectory.csv``) ? that collision is not a leak.
We must NOT
560                 # skip on substring containment: a token like ``label`` or ``traj`` is a
SUBSTRING of
561                 # ``labels.csv``/``trajectory.csv``, so substring-skip would silently miss a
real source
562                 # named ``label.csv`` leaking its stem (red-team #4). Exact membership only.
563                 if not t or t in _SCHEMA_FILENAME_LITERALS:
564                     continue
565                 if t in low:
566                     raise RefusalError(
567                         f"{where}: source-path token {token!r} found in written output ? the
source "
568                         f"filename/path must never be persisted (§4c metadata strip).")
569
570
571     def _reset_trajectory_rows(points: Sequence[Any], t0_frame: int
572                                ) -> List[Tuple[int, int, float, float]]:
573         """Shift every ``TrajectoryPoint`` frame by ``-t0_frame`` ?
``(Frame,Visibility,X,Y)`` rows.
574
575         Visibility/X/Y are passed through untouched (the shuttle geometry is metric-
invariant under a
576         pure time shift); only the absolute frame index moves toward 0, severing the match
timeline.
577         """
578         rows: List[Tuple[int, int, float, float]] = []
579         for p in points:
580             rows.append((int(p.frame) - t0_frame, 1 if p.visible else 0, float(p.x),
float(p.y)))
581         rows.sort(key=lambda r: r[0])
582         return rows
583
584
585     def _reset_labels(gts: Sequence[Interval], t0_sec: float) -> List[Interval]:
586         """Shift every label by ``-t0_sec`` (same offset as the trajectory ? metric-
invariant).
587
588         ``t0_sec`` MUST already be quantized to ``_FLOAT_ROUND_DP`` decimals by the caller.
Each label
589         endpoint is ALSO rounded to 6dp BEFORE subtracting, so the shift is the difference
of two exact
590         6dp quantities ? no double-rounding (``round6(orig - round6(t0))`` vs the no-reset
591         ``round6(orig)`` accumulated error; red-team #2). This keeps the RAW reset-vs-no-
reset score
592         delta at the float-error floor (~3.3e-7, well within ``_FLOAT_TOL``); see
``freeze_real_fixture``
593         for why the SERIALIZED 6dp boundary error can still differ by one 6dp ULP on a sub-
second
594         offset (the grid, not a real metric divergence)."""
595         return [(_round(s) - t0_sec, _round(e) - t0_sec) for s, e in gts]
596
597
598     def freeze_real_fixture(
599         label: str,
600         trajectory_csv_path: Any,
601         labels_csv_path: Any,
602         *,
603         license: str,
604         minors_free: bool,
605         owned: bool,
606         fps: float,
607         frame_width: float,
608         fixtures_dir: Path = FIXTURES_DIR,
609         slug: Optional[str] = None,
```

```
        time_origin_reset: bool = True,
        source_note: str = "",
    ) -> Dict[str, Any]:
        """Turn an OWNED, minors-free real trajectory + golden labels into a committable,
de-attributed
        Tier-0 fixture ? reusing M1's ``replay_fixture`` for the snapshot. (§4c / §5 / §7
row P1.)

        The refusal gate (M2) makes the unsafe path impossible to reach silently: a fixture
is emitted
        ONLY when ``minors_free is True`` AND ``owned is True`` AND ``license`` is a non-
blank string,
        AND ``label`` is a strict slug (lowercase alnum + ``_``/``-`` ? a free-text
identifier like a
        player/event name is refused, red-team #1). The committed bytes carry NO filename /
video_id /
        match / player / capture-date / operator free-text ? only an opaque random surrogate
slug, the
        shuttle-only trajectory (time-origin reset by default), the reset labels, and a
        ``kind="cleared_real"`` provenance block. A positive privacy assertion re-reads the
written
        ``expected.json`` + ``provenance.json`` and refuses if anything attributable slipped
in.

        ``source_note`` is **accepted but NOT persisted** ? operator free-text is an
attribution
        back-channel ``_scan_forbidden`` cannot fully police (an identifying note that
happens to
        contain no FORBIDDEN_KEY / MD5 / source-path token would slip through), so it never
reaches the
        committed ``provenance.json`` (red-team #1). It is still scanned defensively and the
flag is
        retained for CLI compatibility / future logging.

        NOTE (§6 honest limits): the random surrogate + reset timeline make the fixture non-
attributable
        only *relative to the unpublished source* ? it is NOT absolutely anonymous (EDPS v.
SRB). De-
        attribution does not cure provenance/licensing; this sits BEHIND counsel sign-off
(P2) and the
        owner-asserted Fork-A / minors / biometric-exclusion flags.

        The fixture is written into a temp dir and promoted (``os.replace``) to the final
slug dir ONLY
        after the privacy scan passes ? a refused / failed freeze leaves NO partial fixture
dir
        (red-team #3).

        Returns the recomputed ``expected.json`` snapshot dict (identical to
``replay_fixture``).
        """
        # --- (a) REFUSAL GATE (M2) ? the code-enforced provenance hard-fail (§4c / §7 ?)
-------
        reasons: List[str] = []
        if minors_free is not True:
            reasons.append("minors_free must be True (DPDP child = under 18; behavioural
monitoring "
                           "of a minor is prohibited ? §3, guardrail 'exclude minors')")
        if owned is not True:
            reasons.append("owned must be True (only owner-recorded Fork-A source is safe to
commit; "
                           "broadcast/scraped = Fork B, do NOT commit ? §3 D1)")
        if not (isinstance(license, str) and license.strip()):
            reasons.append("license must be a non-blank string (per-record provenance tag ?
§4c "
```

```
                           "'provenance hard-fail gate')")
        # label is committed verbatim into provenance ? it MUST be a strict slug, never
operator free
        # text (a name/event), which _scan_forbidden cannot reliably catch (red-team #1
free-text leak).
        if not (isinstance(label, str) and _LABEL_SLUG_RE.match(label)):
            reasons.append(
                f"label must match the strict slug charset {_LABEL_SLUG_RE.pattern!r}
(lowercase "
                f"alnum + '_'/'-', no spaces/punctuation) ? a free-text identifier
(player/event "
                f"name) is an attribution leak; got {label!r} (§4c metadata strip, red-team
#1)")
        if reasons:
            raise RefusalError(
                "REFUSED to freeze a real Tier-0 fixture ? unsafe provenance (de-attribution
does NOT "
                "cure provenance/licensing; this gate sits behind counsel sign-off,
§6/P2):\n  - "
                + "\n  - ".join(reasons))

        traj_path = Path(str(trajectory_csv_path))
        labels_path = Path(str(labels_csv_path))

        # --- (b) opaque RANDOM surrogate slug ? never the video_id MD5, never the source
name ---
        out_slug = slug if slug is not None else _new_surrogate_slug()

        # --- (c) read via the SHIPPED readers; labels via the STRICT loader (never lenient)
----
        points = TrackNetRunner.parse_trajectory_csv(str(traj_path))
        if not points:
            raise ValueError(f"{traj_path}: no parseable trajectory rows
(Frame,Visibility,X,Y)")
        gts = load_gt_intervals(str(labels_path), strict=True)

        # --- (d) consistent time-origin reset (default ON; metric-invariant within
_FLOAT_TOL, §4c) --
        # Subtract the SAME offset from BOTH the trajectory frames and the label seconds:
metrics.score
        # is built from pure differences (interval_iou, |start-start|, |end-end|), so a pure
time shift
        # leaves the score essentially unchanged ? only the absolute timeline is severed. To
keep the
        # two shifts a consistent quantity (and avoid double-rounding past _FLOAT_TOL ? red-
team #2),
        # t0_sec is ROUNDED to 6dp BEFORE subtracting (and _reset_labels rounds each
endpoint first), so
        # the RAW reset-vs-no-reset score delta sits at the float-error floor (~3.3e-7 ?
_FLOAT_TOL).
        #
        # HONEST LIMIT (not a bug): the trajectory shifts by an INTEGER frame count while
labels shift
        # in SECONDS, and 6dp-rounding is NOT additive (round6(8/fps) - round6(7/fps) ?
round6(1/fps)),
        # so the *serialized* 6dp boundary error (mean_start/end_error) can differ by ONE
6dp ULP
        # (1e-6) on a sub-second offset. That is the serialization grid, not a metric
divergence ? the
        # score is bit-identical for whole-second offsets and within _FLOAT_TOL otherwise
(test #2).
        if time_origin_reset:
            t0_frame = min(int(p.frame) for p in points)
            t0_sec = _round(t0_frame / float(fps))
        else:
```

```
            t0_frame, t0_sec = 0, 0.0
        rows = _reset_trajectory_rows(points, t0_frame)
        reset_gts = _reset_labels(gts, t0_sec)

        # --- (d2) negative-label guard BEFORE any write (red-team #3a)
------------------------
        # A reset label that starts < 0 means the label timeline predates the earliest
trajectory
        # frame ? an inconsistent input that would crash the strict gt_loader on replay
(negative
        # start). Refuse loudly up-front so the partial-write path is never even reached.
        neg = [(s, e) for (s, e) in reset_gts if s < 0]
        if neg:
            raise RefusalError(
                f"REFUSED: label timeline predates the trajectory ? {len(neg)} reset
label(s) have a "
                f"negative start after the t0={t0_sec:.6f}s time-origin reset (e.g.
{neg[0]!r}). The "
                f"earliest label must not precede the earliest trajectory frame (§4c time-
origin reset).")

        iou = 0.5
        # NOTE (red-team #1b): operator free-text `source_note` is DELIBERATELY NOT a key in
`prov` ?
        # it never reaches the committed bytes (an identifying note is an attribution back-
channel
        # _scan_forbidden cannot fully police). The fixed boilerplate `note` constant below
stays.
        prov = {
            "slug": out_slug,
            "schema_version": CURRENT_SCHEMA,
            "kind": "cleared_real",
            "label": str(label),
            "license": license,
            "minors_free": True,
            "owned": True,
            "fps": _round(fps),
            "frame_width": _round(frame_width),
            "iou": _round(iou),
            "time_origin_reset": bool(time_origin_reset),
            "windowing": {
                "vt": PINNED_VT,
                "mg": PINNED_MERGE_GAP,
                "mwd": PINNED_MIN_WINDOW,
                "max_win": PINNED_MAX_WINDOW,
            },
            "paths": {
                "trajectory": "trajectory.csv",
                "labels": "labels.csv",
                "expected": "expected.json",
            },
            "gt_hash": gt_hash(reset_gts),
            "note": (
                "De-attributed REAL Fork-A fixture (owner-recorded, owned, minors-excluded,
biometric-"
                "free, shuttle-only). Opaque RANDOM surrogate slug; absolute timeline reset.
NON-"
                "attributable only relative to the UNPUBLISHED source (EDPS v. SRB) ? never
'anonymous'. "
                "Tier-0 / CONTROL-only. §6: behaviour-locking, not correctness; sits behind
counsel (P2)."
            ),
        }

        # --- (e) write into a TEMP dir, replay, scan, and only THEN promote (red-team #3b)
```

```
-----
747         # Everything is written under a sibling temp dir; the final slug dir is created by
an atomic
748         # os.replace ONLY after the privacy scan passes. A refused/failed freeze therefore
leaves NO
749         # partial fixture dir behind. source_note (if any) is scanned defensively but never
written.
750         fdir = fixtures_dir / out_slug
751         fixtures_dir.mkdir(parents=True, exist_ok=True)
752         if fdir.exists():
753             raise ValueError(f"{fdir}: fixture dir already exists ? refuse to clobber
(choose a new slug)")
754         tmp_parent = Path(tempfile.mkdtemp(prefix=f".{out_slug}.tmp.",
dir=str(fixtures_dir)))
755         tmp_dir = tmp_parent / out_slug
756         tmp_dir.mkdir(parents=True, exist_ok=True)
757         try:
758             write_trajectory_csv(rows, tmp_dir / "trajectory.csv")
759             write_labels_csv(reset_gts, tmp_dir / "labels.csv")
760             _write_json(tmp_dir / "provenance.json", prov)
761
762             expected = replay_fixture(tmp_dir)   # REUSE M1 ? correct-by-construction
snapshot
763             _write_json(tmp_dir / "expected.json", expected)
764
765             # --- (f) POSITIVE privacy assertion BEFORE promotion (scan written text +
source_note) ---
766             source_paths = [str(traj_path), str(labels_path)]
767             _scan_forbidden((tmp_dir / "expected.json").read_text(encoding="utf-8"),
768                             where=f"{out_slug}/expected.json", source_paths=source_paths)
769             _scan_forbidden((tmp_dir / "provenance.json").read_text(encoding="utf-8"),
770                             where=f"{out_slug}/provenance.json", source_paths=source_paths)
771             # source_note is never persisted, but a leak in it is still a contract violation
? scan it.
772             if source_note:
773                 _scan_forbidden(str(source_note),
774                                 where=f"{out_slug}/source_note(not-persisted)",
source_paths=source_paths)
775
776             # --- (g) PROMOTE atomically: temp slug dir ? final slug dir (scan passed)
-----------
777             os.replace(str(tmp_dir), str(fdir))
778         finally:
779             # Clean up the temp parent whether we promoted or refused (a refusal leaves NO
partial dir).
780             shutil.rmtree(str(tmp_parent), ignore_errors=True)
781
782         # --- (h) append to the manifest with kind="cleared_real" (don't clobber synthetic)
-----
783         _append_manifest_entry(prov, fixtures_dir)
784         return expected
785
786
787     def _manifest_entry_for(prov: Dict[str, Any]) -> Dict[str, Any]:
788         """A manifest row for a single fixture, derived from its provenance block."""
789         slug = prov["slug"]
790         return {
791             "slug": slug,
792             "fps": _round(float(prov["fps"])),
793             "frame_width": _round(float(prov["frame_width"])),
794             "windowing": dict(prov.get("windowing", {})),
795             "kind": prov.get("kind", "synthetic"),
796             "schema_version": int(prov.get("schema_version", CURRENT_SCHEMA)),
797             "paths": {
798                 "dir": slug,
```

```
                "trajectory": f"{slug}/trajectory.csv",
                "labels": f"{slug}/labels.csv",
                "expected": f"{slug}/expected.json",
                "provenance": f"{slug}/provenance.json",
            },
        }


    def _append_manifest_entry(prov: Dict[str, Any], fixtures_dir: Path) -> None:
        """Add/replace one fixture's manifest row WITHOUT clobbering the others (e.g.
synthetic)."""
        entries = load_manifest(fixtures_dir)
        entries = [e for e in entries if e.get("slug") != prov["slug"]]
        entries.append(_manifest_entry_for(prov))
        entries.sort(key=lambda e: str(e.get("slug")))
        _write_json(fixtures_dir / "manifest.json", entries)


    # ============================================================================
    # Manifest + loaders
    # ============================================================================


    def _write_manifest(specs: Sequence[FixtureSpec], fixtures_dir: Path) -> None:
        entries = [{
            "slug": spec.slug,
            "fps": _round(spec.fps),
            "frame_width": _round(spec.frame_width),
            "windowing": {
                "vt": PINNED_VT, "mg": PINNED_MERGE_GAP,
                "mwd": PINNED_MIN_WINDOW, "max_win": PINNED_MAX_WINDOW,
            },
            "kind": "synthetic",
            "schema_version": CURRENT_SCHEMA,
            "paths": {
                "dir": spec.slug,
                "trajectory": f"{spec.slug}/trajectory.csv",
                "labels": f"{spec.slug}/labels.csv",
                "expected": f"{spec.slug}/expected.json",
                "provenance": f"{spec.slug}/provenance.json",
            },
        } for spec in specs]
        _write_json(fixtures_dir / "manifest.json", entries)


    def load_manifest(fixtures_dir: Path = FIXTURES_DIR) -> List[Dict[str, Any]]:
        """Load the fixtures manifest (returns [] if absent)."""
        path = fixtures_dir / "manifest.json"
        if not path.is_file():
            return []
        data = _read_json(path)
        if not isinstance(data, list):  # pragma: no cover - corrupt manifest
            raise ValueError(f"{path}: manifest must be a JSON array")
        return data


    def load_fixture(slug: str, fixtures_dir: Path = FIXTURES_DIR) -> Dict[str, Any]:
        """Load one fixture's committed artifacts: ``{provenance, expected, dir}``. Refuses
a fixture
        whose ``schema_version`` exceeds ``CURRENT_SCHEMA`` (forward-incompatible) on EITHER
the
        ``expected`` OR the ``provenance`` block ? they are guarded INDEPENDENTLY so a
forward-version
        bump in one is never masked by an in-range value in the other (red-team #7)."""
        fdir = fixtures_dir / slug
```

```
        prov = _read_json(fdir / "provenance.json")
        expected = _read_json(fdir / "expected.json")
        for which, blob in (("expected", expected), ("provenance", prov)):
            sv = int(blob.get("schema_version", 0))
            if sv > CURRENT_SCHEMA:
                raise ValueError(
                    f"{slug}: {which} schema_version {sv} > CURRENT_SCHEMA {CURRENT_SCHEMA}
? "
                    f"upgrade the harness")
        return {"slug": slug, "dir": fdir, "provenance": prov, "expected": expected}


    # ============================================================================
    # Parse-compare (NEVER byte-compare): exact ints/structure, abs-tol floats
    # ============================================================================


    def compare_expected(recomputed: Dict[str, Any], committed: Dict[str, Any]) ->
List[str]:
        """Return a list of human-readable mismatch messages (empty == match).

        Parse-compare per HARD CONSTRAINT 2: structural/int fields (candidate count,
net_crossings,
        TP/FP/FN, segment count, num_pred/num_gt, slug, schema_version, gt_hash) must be
EXACTLY
        equal; float fields compared with abs tolerance 1e-6. Never a byte compare ?
CRLF/float-format
        safe (the repo runs with core.autocrlf=true).

        Key-set equality is enforced on BOTH the top-level dict AND the ``score`` block
(mirroring the
        per-candidate ``shuttle`` check) so an EXTRA key on one side is a mismatch, not a
silent pass ?
        a fixed-field loop alone would ignore an added field (red-team #5)."""
        out: List[str] = []

        def exact(path: str, a: Any, b: Any) -> None:
            if a != b:
                out.append(f"{path}: {a!r} != {b!r} (exact)")

        def approx(path: str, a: Any, b: Any) -> None:
            try:
                if abs(float(a) - float(b)) > _FLOAT_TOL:
                    out.append(f"{path}: {a!r} != {b!r} (|?| > {_FLOAT_TOL})")
            except (TypeError, ValueError):
                out.append(f"{path}: non-numeric float field {a!r} vs {b!r}")

        # Top-level key-set equality ? an extra/missing top-level key is a mismatch (red-
team #5).
        if set(recomputed) != set(committed):
            out.append(f"(top-level): keys {sorted(recomputed)} != {sorted(committed)}
(exact)")

        exact("slug", recomputed.get("slug"), committed.get("slug"))
        exact("schema_version", recomputed.get("schema_version"),
committed.get("schema_version"))
        exact("gt_hash", recomputed.get("gt_hash"), committed.get("gt_hash"))
        approx("iou", recomputed.get("iou"), committed.get("iou"))

        rc, cc = recomputed.get("candidates", []), committed.get("candidates", [])
        if len(rc) != len(cc):
            out.append(f"candidates: count {len(rc)} != {len(cc)} (exact)")
        else:
            for i, (r, c) in enumerate(zip(rc, cc)):
                approx(f"candidates[{i}].start", r.get("start"), c.get("start"))
```

```
                approx(f"candidates[{i}].end", r.get("end"), c.get("end"))
                rs, cs = r.get("shuttle", {}), c.get("shuttle", {})
                if set(rs) != set(cs):
                    out.append(f"candidates[{i}].shuttle: keys {sorted(rs)} != {sorted(cs)}
(exact)")
                else:
                    for k in rs:
                        if k == "net_crossings":
                            exact(f"candidates[{i}].shuttle.net_crossings", rs[k], cs[k])
                        else:
                            approx(f"candidates[{i}].shuttle.{k}", rs[k], cs[k])

        rseg, cseg = recomputed.get("control_segments", []),
committed.get("control_segments", [])
        if len(rseg) != len(cseg):
            out.append(f"control_segments: count {len(rseg)} != {len(cseg)} (exact)")
        else:
            for i, (r, c) in enumerate(zip(rseg, cseg)):
                approx(f"control_segments[{i}][0]", r[0], c[0])
                approx(f"control_segments[{i}][1]", r[1], c[1])

        rsc, csc = recomputed.get("score", {}), committed.get("score", {})
        # Score-block key-set equality ? an extra/missing score key is a mismatch (red-team
#5).
        if set(rsc) != set(csc):
            out.append(f"score: keys {sorted(rsc)} != {sorted(csc)} (exact)")
        int_fields = ("num_pred", "num_gt", "true_positives", "false_positives",
"false_negatives")
        float_fields = ("iou_threshold", "precision", "recall", "f1", "mean_iou",
                        "mean_start_error", "mean_end_error")
        for k in int_fields:
            exact(f"score.{k}", rsc.get(k), csc.get(k))
        for k in float_fields:
            approx(f"score.{k}", rsc.get(k), csc.get(k))

        return out


    # ============================================================================
    # Small JSON / path helpers (deterministic writers: sort_keys, LF, trailing newline)
    # ============================================================================


    def _write_json(path: Path, obj: Any) -> None:
        text = json.dumps(obj, indent=2, sort_keys=True, ensure_ascii=False)
        path.write_text(text + "\n", encoding="utf-8", newline="\n")


    def _read_json(path: Path) -> Any:
        with open(path, encoding="utf-8") as f:
            return json.load(f)


    def _resolve_dir(slug_or_dir: Any) -> Path:
        if isinstance(slug_or_dir, Path):
            return slug_or_dir
        p = Path(str(slug_or_dir))
        if p.is_dir():
            return p
        return FIXTURES_DIR / str(slug_or_dir)


    def _fps_fw(prov: Optional[Dict[str, Any]], slug: str) -> Tuple[float, float]:
        if prov and "fps" in prov and "frame_width" in prov:
            return float(prov["fps"]), float(prov["frame_width"])
```

```
        spec = _spec_by_slug().get(slug)
        if spec is not None:
            return spec.fps, spec.frame_width
        raise ValueError(f"{slug}: no fps/frame_width in provenance and not a built-in
spec")


    def _fps_fw_from_prov(fdir: Path, slug: str) -> Tuple[float, float]:
        prov_path = fdir / "provenance.json"
        prov = _read_json(prov_path) if prov_path.is_file() else None
        return _fps_fw(prov, slug)


    # ============================================================================
    # CLI
    # ============================================================================


    def _cmd_check(fixtures_dir: Path) -> int:
        manifest = load_manifest(fixtures_dir)
        if not manifest:
            print("[golden-fixtures] no fixtures found ? run --freeze-synthetic first",
file=sys.stderr)
            return 1
        failures = 0
        for entry in manifest:
            slug = entry["slug"]
            committed = _read_json(fixtures_dir / slug / "expected.json")
            recomputed = replay_fixture(fixtures_dir / slug)
            mismatches = compare_expected(recomputed, committed)
            if mismatches:
                failures += 1
                print(f"[FAIL] {slug}: {len(mismatches)} mismatch(es) "
                      f"(characterization = behaviour-locking, NOT correctness ? §6):")
                for m in mismatches:
                    print(f"    - {m}")
            else:
                sc = recomputed["score"]
                print(f"[ok]   {slug}: {len(recomputed['candidates'])} candidates, "
                      f"{len(recomputed['control_segments'])} segments, F1={sc['f1']:.6f}")
        print(f"[golden-fixtures] {len(manifest) - failures}/{len(manifest)} fixtures match
"
              f"(synthetic-tier = plumbing correctness, not field quality ? §6)")
        return 0 if failures == 0 else 1


    def main(argv: Optional[Sequence[str]] = None) -> int:
        ap = argparse.ArgumentParser(
            description="Golden Regression Fixtures (M1) ? synthetic, CONTROL-only replay
harness.")
        g = ap.add_mutually_exclusive_group(required=True)
        g.add_argument("--freeze-synthetic", action="store_true",
                       help="generate synthetic trajectory+labels for all built-in
scenarios, then "
                            "freeze expected.json+provenance.json + write manifest.json")
        g.add_argument("--refresh-snapshot", action="store_true",
                       help="re-freeze expected.json from COMMITTED trajectory+labels (no
regeneration)")
        g.add_argument("--check", action="store_true",
                       help="replay all fixtures + parse-compare vs committed expected.json
(exit 0/1)")
        g.add_argument("--freeze-real", action="store_true",
                       help="(P1) de-attribute an OWNED real trajectory+labels into a
committable Tier-0 "
                            "fixture; REFUSED without --owned --minors-free and a non-blank
```

```
--license")
1033        ap.add_argument("--slug", help="single slug for --refresh-snapshot, or the surrogate
slug for "
1034                                        "--freeze-real (default: a random fx_<hex> id)")
1035        ap.add_argument("--all", action="store_true", help="all slugs for --refresh-
snapshot")
1036        ap.add_argument("--reason", help="REQUIRED recorded reason for --refresh-snapshot
(review the diff!)")
1037        # --freeze-real (P1 / M2 refusal gate) options:
1038        ap.add_argument("--trajectory", help="(--freeze-real) source trajectory CSV
(Frame,Visibility,X,Y)")
1039        ap.add_argument("--labels", help="(--freeze-real) source golden labels CSV
(start,end)")
1040        ap.add_argument("--license", dest="license_", default=None,
1041                        help="(--freeze-real) the source license tag (required, non-blank)")
1042        ap.add_argument("--owned", action="store_true",
1043                        help="(--freeze-real) assert owner-recorded Fork-A source
(required)")
1044        ap.add_argument("--minors-free", dest="minors_free", action="store_true",
1045                        help="(--freeze-real) assert the source contains NO minors
(required)")
1046        ap.add_argument("--fps", type=float, help="(--freeze-real) source fps")
1047        ap.add_argument("--frame-width", dest="frame_width", type=float, help="(--freeze-
real) source frame width (px)")
1048        ap.add_argument("--label", default="cleared_real",
1049                        help="(--freeze-real) non-identifying scenario label ? STRICT slug
only "
1050                             "(lowercase alnum + '_'/'-', no spaces/punctuation/names)")
1051        ap.add_argument("--source-note", dest="source_note", default="",
1052                        help="(--freeze-real) ACCEPTED BUT NOT PERSISTED ? operator free-
text is an "
1053                             "attribution back-channel, so it never reaches the committed
provenance "
1054                             "(scanned defensively, then dropped). Retained for CLI
compatibility.")
1055        ap.add_argument("--no-time-origin-reset", dest="time_origin_reset",
action="store_false",
1056                        help="(--freeze-real) DISABLE the anti-attribution time-origin reset
(default ON)")
1057        ap.set_defaults(time_origin_reset=True)
1058        ap.add_argument("--fixtures-dir", type=Path, default=FIXTURES_DIR,
1059                        help="override the fixtures directory (default: repo
eval_baselines/fixtures)")
1060        args = ap.parse_args(list(argv) if argv is not None else None)
1061
1062        fixtures_dir: Path = args.fixtures_dir
1063
1064        if args.freeze_synthetic:
1065            slugs = freeze_synthetic(fixtures_dir)
1066            print(f"[golden-fixtures] froze {len(slugs)} synthetic fixture(s): {',
'.join(slugs)}")
1067            print(f"  -> {fixtures_dir}")
1068            return 0
1069
1070        if args.refresh_snapshot:
1071            if not args.slug and not args.all:
1072                ap.error("--refresh-snapshot requires --slug S or --all")
1073            # --reason is MANDATORY (red-team #6): --refresh-snapshot can rubber-stamp a
real
1074            # regression, so the deliberate, recorded reason is a required arg, not a soft
warning.
1075            if not (args.reason and args.reason.strip()):
1076                ap.error("--refresh-snapshot requires --reason \"<why>\" ? it can rubber-
stamp a real "
1077                         "regression; record the deliberate reason and review the visible
```

```
diff (§6).")
1078            targets = refresh_snapshot(slug=args.slug, all_slugs=args.all,
fixtures_dir=fixtures_dir)
1079            print(f"[golden-fixtures] refreshed snapshot for: {', '.join(targets)}"
1080                  f"  (reason: {args.reason})")
1081            return 0
1082
1083        if args.freeze_real:
1084            missing = [n for n, v in (("--trajectory", args.trajectory),
1085                                      ("--labels", args.labels),
1086                                      ("--fps", args.fps),
1087                                      ("--frame-width", args.frame_width)) if v is None]
1088            if missing:
1089                ap.error("--freeze-real requires " + ", ".join(missing))
1090            try:
1091                expected = freeze_real_fixture(
1092                    args.label, args.trajectory, args.labels,
1093                    license=args.license_ if args.license_ is not None else "",
1094                    minors_free=args.minors_free, owned=args.owned,
1095                    fps=args.fps, frame_width=args.frame_width,
1096                    fixtures_dir=fixtures_dir, slug=args.slug,
1097                    time_origin_reset=args.time_origin_reset, source_note=args.source_note)
1098            except RefusalError as e:
1099                # The code-enforced provenance gate (§4c / §7 ?): refuse loudly, exit non-
zero.
1100                print(f"[golden-fixtures] REFUSED (--freeze-real provenance gate):\n{e}",
file=sys.stderr)
1101                return 2
1102            print(f"[golden-fixtures] froze de-attributed REAL fixture '{expected['slug']}'
"
1103                  f"(kind=cleared_real) -> {fixtures_dir / expected['slug']}")
1104            print("  §6: behaviour-locking, NOT correctness; non-attributable only relative
to the "
1105                  "UNPUBLISHED source ? never 'anonymous'. Sits behind counsel sign-off
(P2).")
1106            return 0
1107
1108        # --check
1109        return _cmd_check(fixtures_dir)
1110
1111
1112    if __name__ == "__main__":
1113        raise SystemExit(main())
1114
```

22. assistant (2026-06-18T01:33:58.582Z)

This module is exceptionally well-guarded. Let me look at the readers that consume DB rows / candidate dicts in the eval pipeline ? `experiment.py`, `distill_local.py`, `fusion_golden.py`, `per_user_*`, `serving.py`, and `promotion.py` ? these read `c["person"]`, classifier dicts, and promotion-evidence JSON. The system reminder in golden_fixtures even hints at `c['person']` access. Let me check those.

23. user (2026-06-18T01:33:59.751Z)

```
1       """Per-user serving bridge ? resolve a user's fusion config + classifier from the v5
store.
2
3       PERSONALIZATION_PLAN.md §2 (Phase 1, the "#1 wall"): a personalized model is a per-user
4       ``Variant`` (cue_flags + ``min_crossings`` / ``min_players`` + an optional
``LogisticRegression``
5       + threshold). It does NOT require a segmenter rewrite ? it is a per-user
``FusionConfig`` overlay
6       resolved UPSTREAM of detection, plus a classifier the decision seam re-scores persisted
features
```

```
7       with. This module is that resolution step, and **only** that step.
8
9       What this module does (additive, pure-ish ? one DB read, no writes):
10        - ``resolve_user_fusion_config`` ? overlay the active ``user_models`` row's
``fusion_config``
11          onto the served fusion config dict, returning the config the served path WOULD use.
Falls
12          back to the served baseline (a copy, unchanged) when there is no active model, or
the model
13          is below the promotion-confidence floor (``n_videos_at_fit < min_n``), or it carries
no
14          fusion overlay. The min-N floor here is the SAME ``per_user_gate`` promotion floor:
below it
15          a per-user A/B is not trusted enough to override a baseline default (PLAN §1.3).
16        - ``load_user_classifier`` ? rehydrate the stored ``classifier_json`` into a
17          ``LogisticRegression`` via ``from_dict`` (None when the user has no active model or
it is a
18          config-only model). ``LogisticRegression.load`` is never called in
``backend/pipeline/``
19          today ? this is the load half of the serving bridge.
20
21      ? NOT WIRED INTO THE SERVED PATH. ``fusion_hybrid._select_for_handoff`` still reads only
the
22      static ``idx_cfg["fusion"]`` ? it does NOT call anything here. Wiring this resolver into
that
23      seam (so a pinned, promoted per-user model actually changes the served handoff) is the
NEXT,
24      **owner-gated** PR, because it is the first change to served behavior. Until then this
is pure
25      groundwork: import it, unit-test it, but nothing in production reads it. With NULL
``user_id``
26      (the local install) and no active model, ``resolve_user_fusion_config`` returns the
served
27      config unchanged ? byte-identical to today.
28      """
29      from __future__ import annotations
30
31      import copy
32      from typing import Any, Dict, Optional
33
34      from backend.eval.classifier import LogisticRegression
35
36      __all__ = [
37          "DEFAULT_MIN_N",
38          "resolve_user_fusion_config",
39          "load_user_classifier",
40      ]
41
42      #: Promotion-confidence floor (PERSONALIZATION_PLAN.md §1.3, owner-confirmed ``min_n =
5``).
43      #: A per-user model fit on fewer than this many of the user's videos is NOT trusted
enough to
44      #: override a baseline fusion default ? we fall back to the served baseline. This
mirrors
45      #: ``backend.eval.per_user_gate.should_promote_for_user``'s ``min_n`` default.
46      DEFAULT_MIN_N = 5
47
48
49      def resolve_user_fusion_config(
50          db: Any,
51          user_id: Optional[str],
52          served_cfg: Dict[str, Any],
53          *,
54          min_n: int = DEFAULT_MIN_N,
55      ) -> Dict[str, Any]:
```

```
56          """Resolve the fusion config the served path should use for ``user_id``.
57
58          Overlays the active ``user_models`` row's ``fusion_config`` (a partial per-user
59          ``FusionConfig`` dict ? e.g. raised ``min_crossings`` / ``min_players`` / flipped
60          ``cue_flags``) onto a COPY of ``served_cfg`` (the baseline served fusion config
dict, i.e.
61          ``idx_cfg["fusion"]``). The original ``served_cfg`` is never mutated.
62
63          Falls back to the served baseline (an unmodified deep copy) when ANY of:
64            - the user has no active model;
65            - the active model was fit below the promotion floor (``n_videos_at_fit < min_n``)
? too
66              little of the user's own footage to trust a per-user override (PLAN §1.3);
67            - the active model carries no ``fusion_config`` overlay (a classifier-only model).
68
69          So a brand-new / NULL (local) user with no active model gets the served baseline
unchanged
70          ? never denied service, never altered. The overlay is a shallow key-level merge:
per-user
71          keys win, every key the user did not set keeps the baseline value.
72          """
73          base = copy.deepcopy(served_cfg)
74          model = db.get_active_user_model(user_id)
75          if not model:
76              return base
77          if int(model.get("n_videos_at_fit") or 0) < min_n:
78              # Below the trust floor ? keep the baseline (the conservative default).
79              return base
80          overlay = model.get("fusion_config")
81          if not overlay:
82              return base  # config-only fallback: a classifier-only model has nothing to
overlay
83          base.update(overlay)
84          return base
85
86
87      def load_user_classifier(db: Any, user_id: Optional[str]) ->
Optional[LogisticRegression]:
88          """Rehydrate the active per-user classifier for ``user_id``, or None.
89
90          Returns a ``LogisticRegression`` rebuilt from the stored ``classifier_json`` (via
91          ``from_dict``) when the user has an active model that carries one; None when there
is no
92          active model or it is a config-only (no-classifier) model. The caller (the FUTURE
served
93          wiring) would feed this model's ``predict_proba`` over the persisted
``window_stats`` ?
94          the same math ``experiment.predict`` runs offline. Nothing calls this on the served
path
95          yet; it is the load half of the serving bridge.
96          """
97          model = db.get_active_user_model(user_id)
98          if not model:
99              return None
100         blob = model.get("classifier_json")
101         if not blob:
102             return None
103         return LogisticRegression.from_dict(blob)
104
```

24. assistant (2026-06-18T01:34:03.115Z)

Well-guarded with `.get()`. The `from_dict` on classifier_json is interesting ? let me check `LogisticRegression.from_dict` since it parses persisted JSON (the `classifier_json` column). That's a prime unguarded-access spot if it does `d["coef"]` without guards.

25. user (2026-06-18T01:34:04.217Z)

```
    """A tiny, dependency-free logistic-regression classifier (numpy only).

    The offline learned segmenter (ADR-004) must run locally with no external API and
    no heavy ML stack ? the project already depends on numpy, and sklearn is not
    installed. This implements just enough: standardized features, balanced-class
    gradient descent, sigmoid probabilities, and JSON (de)serialization so a trained
    model is a small, human-readable, version-controllable file.

    It deliberately stays simple: the rally-window features are few and roughly
    linearly separable (high player motion + sustained activity => real rally), so a
    well-regularized linear model is a sensible, inspectable baseline. Swap in a
    richer model later behind the same fit/predict_proba/save/load interface.
    """

    import json
    from typing import List, Optional

    import numpy as np


    class LogisticRegression:
        def __init__(self, lr: float = 0.1, epochs: int = 1000, l2: float = 1e-3):
            self.lr = lr
            self.epochs = epochs
            self.l2 = l2
            self.w: Optional[np.ndarray] = None
            self.b: float = 0.0
            self.mean: Optional[np.ndarray] = None
            self.std: Optional[np.ndarray] = None
            self.feature_names: Optional[List[str]] = None

        @staticmethod
        def _sigmoid(z: np.ndarray) -> np.ndarray:
            # Clip for numerical stability (avoid overflow in exp).
            return 1.0 / (1.0 + np.exp(-np.clip(z, -30, 30)))

        def _standardize(self, X: np.ndarray) -> np.ndarray:
            return (X - self.mean) / self.std

        def fit(self, X, y, feature_names: Optional[List[str]] = None,
                class_weight: str = "balanced") -> "LogisticRegression":
            """Fit with optional balanced class weighting (rally windows are imbalanced
            ? many spurious candidates per real rally)."""
            X = np.asarray(X, dtype=float)
            y = np.asarray(y, dtype=float)
            if X.ndim != 2 or X.shape[0] == 0:
                raise ValueError("X must be a non-empty 2D array")
            self.feature_names = feature_names

            self.mean = X.mean(axis=0)
            self.std = X.std(axis=0)
            self.std[self.std == 0] = 1.0  # guard constant features
            Xs = self._standardize(X)
            n, d = Xs.shape

            if class_weight == "balanced":
                n_pos = max(1, int(y.sum()))
                n_neg = max(1, int((1 - y).sum()))
                w_pos, w_neg = n / (2.0 * n_pos), n / (2.0 * n_neg)
            else:
                w_pos = w_neg = 1.0
            sw = np.where(y == 1, w_pos, w_neg)

```

```
64              self.w = np.zeros(d)
65              self.b = 0.0
66              for _ in range(self.epochs):
67                  p = self._sigmoid(Xs @ self.w + self.b)
68                  err = (p - y) * sw
69                  grad_w = Xs.T @ err / n + self.l2 * self.w
70                  grad_b = float(err.mean())
71                  self.w -= self.lr * grad_w
72                  self.b -= self.lr * grad_b
73              return self
74
75          def predict_proba(self, X) -> np.ndarray:
76              if self.w is None:
77                  raise RuntimeError("model is not fitted")
78              Xs = self._standardize(np.asarray(X, dtype=float))
79              return self._sigmoid(Xs @ self.w + self.b)
80
81          def predict(self, X, threshold: float = 0.5) -> np.ndarray:
82              return (self.predict_proba(X) >= threshold).astype(int)
83
84          # --- persistence (small JSON, no pickle/joblib) --------------------------
85
86          def to_dict(self) -> dict:
87              return {
88                  "model": "logreg",
89                  "lr": self.lr, "epochs": self.epochs, "l2": self.l2,
90                  "w": self.w.tolist() if self.w is not None else None,
91                  "b": self.b,
92                  "mean": self.mean.tolist() if self.mean is not None else None,
93                  "std": self.std.tolist() if self.std is not None else None,
94                  "feature_names": self.feature_names,
95              }
96
97          @classmethod
98          def from_dict(cls, d: dict) -> "LogisticRegression":
99              m = cls(lr=d.get("lr", 0.1), epochs=d.get("epochs", 1000), l2=d.get("l2", 1e-3))
100             m.w = np.asarray(d["w"], dtype=float) if d.get("w") is not None else None
101             m.b = float(d.get("b", 0.0))
102             m.mean = np.asarray(d["mean"], dtype=float) if d.get("mean") is not None else
None
103             m.std = np.asarray(d["std"], dtype=float) if d.get("std") is not None else None
104             m.feature_names = d.get("feature_names")
105             return m
106
107         def save(self, path: str) -> None:
108             with open(path, "w") as f:
109                 json.dump(self.to_dict(), f, indent=2)
110
111         @classmethod
112         def load(cls, path: str) -> "LogisticRegression":
113             with open(path) as f:
114                 return cls.from_dict(json.load(f))
115
```

26. assistant (2026-06-18T01:34:08.216Z)

`from_dict` uses `.get()` throughout ? well-guarded. Now let me look at the eval modules that read candidate dicts with `c["person"]` style access (the golden_fixtures comment explicitly says `fusion_compare.load_videos / ablation.load_videos` read `c['person']` unconditionally). Those two are excluded. But let me check `experiment.py`, `distill_local.py`, `fusion_golden.py`, and the per_user modules.

27. user (2026-06-18T01:34:09.789Z)

```
1       """Experiment harness ? A/B + shadow + side-by-side for rally detection.
```

```
2
3       This is the thin **variant/experiment layer** that `docs/EXPERIMENT_HARNESS.md`
4       identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5       no-regression gate, the pinnable classifier artifact, the persisted-candidate
6       substrate) already exists ? this module composes it. **No new scoring math lives
7       here**; everything defers to:
8
9         - `backend.eval.metrics`        ? `score`, `match_intervals`, `interval_iou`
10        - `backend.eval.training`       ? `label_candidates` (y by the same IoU matcher)
11        - `backend.eval.classifier`     ? `LogisticRegression` (the pinnable JSON model)
12        - `backend.eval.gemini_refine`  ? `merge_overlaps` (the blessed window merge)
13        - `backend.eval.regression`     ? `gt_hash` (label-set fingerprint)
14        - `backend.eval.calibration`    ? `pct_improvement`
15
16      The thesis (`EXPERIMENT_HARNESS.md` §2): separate the **expensive, risky DETECTION**
17      (per-frame signals ? run once on the GPU/WSL box, persisted into
18      `candidate_segments.stats`) from the **cheap, swappable DECISION** (given those
19      signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
20      given persisted candidate features it produces `List[Interval]` deterministically,
21      with no GPU and zero production risk. So most experiments are pure offline
22      re-scoring of stored data and never touch the served pipeline.
23
24      Three modes (`EXPERIMENT_HARNESS.md` §5):
25        - `compare()`           ? labeled A/B vs golden labels: per-video + LOVO-honest
26          aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
27          held-out loop, but variant-parameterised.
28        - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
29          variants agree / diverge (A-only, B-only, agreed). The divergences are what a
30          human spot-checks (and what two highlight reels would show side by side).
31        - the promotion gate stays `run_regression.py` + `regression_baseline.json`
32          (exit 0/1/3); **rollback = flip the variant/config, never `git revert`.**
33
34      Pure + offline + unit-tested. The only DB-touching helper is
35      `load_video_candidates`, which reads persisted candidates via
36      `Database.query_candidate_segments`.
37      """
38      from __future__ import annotations
39
40      import json
41      import logging
42      import os
43      from dataclasses import dataclass, field
44      from datetime import datetime, timezone
45      from hashlib import sha1
46      from typing import Any, Callable, Dict, List, Optional, Sequence
47
48      from backend.eval.calibration import pct_improvement
49      from backend.eval.classifier import LogisticRegression
50      from backend.eval.gemini_refine import merge_overlaps
51      from backend.eval.metrics import Interval, match_intervals, score
52      from backend.eval.regression import gt_hash
53      from backend.eval.training import label_candidates
54      from backend.eval import eval_stats as _stats           # C1: CIs + paired sign test
55      from backend.eval import segmentation_metrics as _segm   # C4: F1@k + segment-count
ratio
56      from backend.eval.score_calibration import fit_isotonic, fit_platt   # C2: calibrate cue
scores
57
58      logger = logging.getLogger(__name__)
59
60      # Where a comparison run appends one reproducible record. Tracked location (NOT under
61      # the gitignored output/) so the leaderboard survives a clean checkout, mirroring
62      # regression.DEFAULT_BASELINE_PATH.
63      DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
64      _METRICS = ("precision", "recall", "f1", "mean_iou")
```

```
65
66
67      class ExperimentError(ValueError):
68          """A variant cannot be evaluated as configured (e.g. a learned variant asked to
69          score an unlabeled video with no pinned model, or a gate referencing an absent
70          feature)."""
71
72
73      # ---------------------------------------------------------------------------- Variant
74
75
76      @dataclass
77      class Variant:
78          """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).
79
80          A variant turns one video's persisted candidate windows + features into rally
81          intervals. Every knob defaults to the **control** behaviour (no gate, no learned
82          filter), so a variant that merely *carries* a new, disabled cue is byte-identical
83          to control ? the core no-regression guarantee.
84
85          Fields:
86            - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
87              for the leaderboard and for selecting the served path (the existing
88              `segmenter_registry` + `IndexingConfig` do the actual selection in production).
89              New cues are recorded here default-OFF.
90            - ``min_crossings`` ? decision-gate **Gate A**: keep only windows whose persisted
91              ``net_crossings`` feature is >= this. 0 = off. This is the eval-only gate from
92              `rally_gate.py` expressed as a re-scoring knob (kills service-feed / held-
shuttle
93              windows ? see `MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
94            - ``min_players`` ? decision-gate **Gate B** (the future person cue): keep windows
95              whose persisted ``players_in_court`` is >= this. 0 = off (no-op until the person
96              cue persists that feature).
97            - ``classifier_model`` ? path to a frozen `LogisticRegression` JSON. When set,
98              `compare()` scores videos with the SHIPPED model as-is (no per-fold training);
99              required for `compare_unlabeled` on a learned variant.
100           - ``feature_names`` ? the persisted features the learned filter consumes. When set
101             WITHOUT ``classifier_model``, `compare()` trains a fresh model per LOVO fold
102             (honest) ? the recipe, not a pinned artifact.
103           - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
104             overlap-merge gap (seconds), the same `gemini_refine.merge_overlaps` default.
105         """
106
107         name: str
108         segmenter: str = "wasb_hybrid"
109         config_overrides: Dict[str, Any] = field(default_factory=dict)
110         cue_flags: Dict[str, bool] = field(default_factory=dict)
111         min_crossings: int = 0
112         min_players: int = 0
113         classifier_model: Optional[str] = None
114         feature_names: Optional[List[str]] = None
115         threshold: float = 0.5
116         merge_gap: float = 1.0
117         # C2 / threshold-in-LOVO (default OFF ? unchanged behaviour). When set, the
operating
118         # point is chosen on the TRAINING fold, never the held-out one ? fixing the
first-A/B
119         # failure where a fixed 0.5 zeroed out a small-candidate video.
120         tune_threshold: bool = False          # pick the F1-best threshold on the train fold
121         calibrate: Optional[str] = None       # None | "platt" | "isotonic" ? calibrate
proba first
122
123         def is_learned(self) -> bool:
124             """True if this variant applies a learned classifier (pinned model or
recipe)."""
```

```
            return self.classifier_model is not None or bool(self.feature_names)

        def to_dict(self) -> dict:
            return {
                "name": self.name,
                "segmenter": self.segmenter,
                "config_overrides": self.config_overrides,
                "cue_flags": self.cue_flags,
                "min_crossings": self.min_crossings,
                "min_players": self.min_players,
                "classifier_model": self.classifier_model,
                "feature_names": self.feature_names,
                "threshold": self.threshold,
                "merge_gap": self.merge_gap,
                "tune_threshold": self.tune_threshold,
                "calibrate": self.calibrate,
            }

        @classmethod
        def from_dict(cls, d: dict) -> "Variant":
            known = {f for f in cls.__dataclass_fields__}  # type: ignore[attr-defined]
            return cls(**{k: v for k, v in d.items() if k in known})

        def spec_hash(self) -> str:
            """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
            and makes a result reproducible. The pinned **control** variant always hashes
the
            same, so a regression is always traceable to a recipe change, never to drift."""
            blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
            return sha1(blob.encode()).hexdigest()[:12]


    #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
    #: filter. Always the comparison reference; "rollback" = make this the served variant.
    CONTROL = Variant(name="control")


    # ---------------------------------------------------------------- persisted candidates


    @dataclass
    class VideoCandidates:
        """One video's persisted detection, ready for offline re-scoring.

        ``intervals`` are the candidate windows; ``features`` is column-major
        (``feature_name -> per-candidate value``, each list aligned to ``intervals``);
        ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
        with `load_video_candidates`, or directly in tests.
        """

        name: str
        intervals: List[Interval]
        features: Dict[str, List[float]] = field(default_factory=dict)
        gts: Optional[List[Interval]] = None


    def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
        """Persisted feature column, defaulting missing/short columns to 0.0 (matches
        `training.extract_yolo_features`, which lets a sparse/old row still vectorise)."""
        col = vc.features.get(name)
        n = len(vc.intervals)
        if col is None:
            logger.warning("variant references feature %r absent from %s ? treating as 0.0",
                           name, vc.name)
            return [0.0] * n
```

```
        if len(col) != n:
            raise ExperimentError(
                f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
        return [float(x) for x in col]


    def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
List[List[float]]:
        cols = [_feature_column(vc, name) for name in feature_names]
        return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]


    def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
        """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
players)."""
        n = len(vc.intervals)
        mask = [True] * n
        if variant.min_crossings > 0:
            col = _feature_column(vc, "net_crossings")
            mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
        if variant.min_players > 0:
            col = _feature_column(vc, "players_in_court")
            mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
        return mask


    def predict(variant: Variant, vc: VideoCandidates,
                model: Optional[LogisticRegression] = None,
                threshold: Optional[float] = None,
                calibrator: Optional[Callable[[float], float]] = None) -> List[Interval]:
        """Variant prediction: gate by persisted features, optionally filter by a learned
        model, then merge. Pure and deterministic.

        ``model`` (an already-fitted `LogisticRegression`) is supplied by `compare` per LOVO
        fold; if omitted and the variant pins ``classifier_model``, that frozen model is
        loaded. A learned variant with neither a supplied nor a pinned model raises ? there
        is nothing to score with. ``threshold``/``calibrator`` (both fitted on the TRAINING
        fold) override the variant defaults when the C2 / threshold-in-LOVO path supplies
them.
        """
        mask = _gate_mask(variant, vc)
        if variant.feature_names:
            if model is None:
                if variant.classifier_model is None:
                    raise ExperimentError(
                        f"variant {variant.name!r} is learned but has no model to predict "
                        f"with (pass a fitted model or set classifier_model)")
                model = LogisticRegression.load(variant.classifier_model)
            proba = [float(p) for p in model.predict_proba(_feature_matrix(vc,
variant.feature_names))]
            if calibrator is not None:
                proba = [calibrator(p) for p in proba]
            thr = variant.threshold if threshold is None else threshold
            mask = [m and (proba[i] >= thr) for i, m in enumerate(mask)]
        kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
        return merge_overlaps(kept, gap_tol=variant.merge_gap)


    def _calibrate_and_tune(variant: Variant, model: LogisticRegression,
                            train_videos: Sequence[VideoCandidates], iou: float
                            ) -> "tuple[Optional[Callable[[float], float]],
Optional[float]]":
        """Fit a calibrator and/or pick the operating threshold on the TRAINING fold only
        (C2 + threshold-in-LOVO). Returns ``(calibrator, threshold)`` ? either may be None
        (use the variant default). Honest: never looks at the held-out video.
```

```
        """
        calibrator: Optional[Callable[[float], float]] = None
        if variant.calibrate:
            raw: List[float] = []
            y: List[int] = []
            for vc in train_videos:
                raw.extend(float(p) for p in
                           model.predict_proba(_feature_matrix(vc, variant.feature_names or
[])))
                y.extend(label_candidates(vc.intervals, vc.gts or [], iou))
            if variant.calibrate == "platt":
                calibrator = fit_platt(raw, y)
            elif variant.calibrate == "isotonic":
                calibrator = fit_isotonic(raw, y)
            else:
                raise ExperimentError(f"unknown calibrate={variant.calibrate!r} (use
'platt'/'isotonic')")
        threshold: Optional[float] = None
        if variant.tune_threshold:
            best_t, best_f1 = variant.threshold, -1.0
            for k in range(1, 20):                      # grid 0.05..0.95 on the train
fold's rally F1
                t = round(0.05 * k, 2)
                f1s = [score(predict(variant, vc, model=model, threshold=t,
calibrator=calibrator),
                             vc.gts or [], iou).f1 for vc in train_videos]
                mean_f1 = sum(f1s) / len(f1s) if f1s else 0.0
                if mean_f1 > best_f1:
                    best_f1, best_t = mean_f1, t
            threshold = best_t
        return calibrator, threshold


    # ------------------------------------------------------------------- variant scoring


    def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
LogisticRegression:
        """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via
the
        evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`."""
        X: List[List[float]] = []
        y: List[int] = []
        for vc in videos:
            if vc.gts is None:
                raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
            X.extend(_feature_matrix(vc, variant.feature_names or []))
            y.extend(label_candidates(vc.intervals, vc.gts, iou))
        if not X:
            raise ExperimentError("cannot train a learned variant on zero candidates")
        return LogisticRegression().fit(X, y, variant.feature_names)


    def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
                         iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
        """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a
        mean aggregate.

        Prediction policy:
          - **static variant** (no learned filter): predict each video directly ? no
training,
            so no leakage; per-video scoring is already honest.
          - **learned, pinned model** (``classifier_model`` set): score every video with the
            SHIPPED model. ? optimistic if those videos were in its training set ? use this
to
```

```
306             report "how the shipped artifact does here", not for the promotion decision.
307           - **learned recipe** (``feature_names``, no pinned model) + ``honest=True``: LOVO
?
308             train on the OTHER videos, predict the held-out one. The number to trust.
309           - learned recipe + ``honest=False``: train on all, predict each (overfit; sanity
only).
310         """
311         for vc in videos:
312             if vc.gts is None:
313                 raise ExperimentError(f"{vc.name} has no golden labels; use
compare_unlabeled")
314
315         per_video: List[Dict[str, Any]] = []
316         predictions: Dict[str, List[Interval]] = {}
317
318         recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None
319         pinned_model = (LogisticRegression.load(variant.classifier_model)
320                         if variant.classifier_model is not None else None)
321         all_model = None
322         if recipe_lovo and not honest:
323             all_model = _train(variant, videos, iou)
324
325         for i, vc in enumerate(videos):
326             cal: Optional[Callable[[float], float]] = None
327             thr: Optional[float] = None
328             if recipe_lovo and honest:
329                 others = [v for j, v in enumerate(videos) if j != i]
330                 if not others:
331                     raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a
pinned model")
332                 model: Optional[LogisticRegression] = _train(variant, others, iou)
333                 if variant.tune_threshold or variant.calibrate:    # operating point from
the train fold
334                     assert model is not None                       # _train never returns
None
335                     cal, thr = _calibrate_and_tune(variant, model, others, iou)
336             elif recipe_lovo:                          # honest=False ? one model over all
videos
337                 model = all_model
338             else:                                      # static variant or pinned model
339                 model = pinned_model
340             preds = predict(variant, vc, model=model, threshold=thr, calibrator=cal)
341             assert vc.gts is not None                  # guarded at function top
342             sc = score(preds, vc.gts, iou)
343             per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m)
for m in _METRICS}})
344             predictions[vc.name] = preds
345
346         aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else
0.0)
347                      for m in _METRICS}
348         return {
349             "variant": variant.name,
350             "spec_hash": variant.spec_hash(),
351             "is_learned": variant.is_learned(),
352             "honest_lovo": recipe_lovo and honest,
353             "per_video": per_video,
354             "aggregate": aggregate,
355             "predictions": predictions,
356         }
357
358
359     def _ci_block(eval_v: Dict[str, Any]) -> Dict[str, Any]:
360         """Per-metric mean + bootstrap CI over the per-video scores (C1 ? never a bare
mean)."""
```

```
        return {m: _stats.mean_with_ci([p[m] for p in eval_v["per_video"]]) for m in
_METRICS}


    def _segmentation_block(videos: Sequence[VideoCandidates], eval_v: Dict[str, Any]) ->
Dict[str, Any]:
        """Mean F1@k + segment-count ratio across videos (C4 ? the over-segmentation tIoU
hides)."""
        gts_by_name = {v.name: (v.gts or []) for v in videos}
        overlaps = _segm.DEFAULT_OVERLAPS
        f1k: Dict[float, List[float]] = {ov: [] for ov in overlaps}
        ratios: List[float] = []
        for name, preds in eval_v.get("predictions", {}).items():
            gts = gts_by_name.get(name, [])
            got = _segm.f1_at_overlaps(preds, gts, overlaps)
            for ov in overlaps:
                f1k[ov].append(got[ov])
            ratios.append(_segm.segment_count_ratio(preds, gts))

        def _mean(xs: List[float]) -> float:
            return sum(xs) / len(xs) if xs else 0.0
        out: Dict[str, Any] = {f"f1@{int(ov * 100)}": _mean(vals) for ov, vals in
f1k.items()}
        out["segment_count_ratio"] = _mean(ratios)
        return out


    def honest_summary(videos: Sequence[VideoCandidates], eval_a: Dict[str, Any],
                       eval_b: Dict[str, Any]) -> Dict[str, Any]:
        """C1 + C4 honest reporting layered over a compare() pair (additive,
EXPERIMENT_HARNESS §6).

        ``per_video`` lists are video-aligned (``evaluate_variant`` iterates ``videos`` in
order),
        so ``paired_delta_f1`` pairs B?A by video ? the promotion-grade view: a lift is real
when
        the ?F1 CI clears 0 *and* the sign test agrees, not when a bare mean ticked up.
        """
        a_f1 = [p["f1"] for p in eval_a["per_video"]]
        b_f1 = [p["f1"] for p in eval_b["per_video"]]
        return {
            "variant_a_ci": _ci_block(eval_a),
            "variant_b_ci": _ci_block(eval_b),
            "paired_delta_f1": _stats.paired_delta_ci(a_f1, b_f1),
            "segmentation": {"variant_a": _segmentation_block(videos, eval_a),
                             "variant_b": _segmentation_block(videos, eval_b)},
        }


    def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,
                iou: float = 0.5, honest: bool = True, honest_stats: bool = True) ->
Dict[str, Any]:
        """Offline labeled A/B (`EXPERIMENT_HARNESS.md` §5.1). Scores both variants on the
same
        golden corpus and reports the **B ? A** deltas, per video and in aggregate.

        The deltas use the existing `metrics.score` (per video) +
`calibration.pct_improvement`;
        learned variants are scored LOVO-honestly. The result is a self-contained record fit
for
        `record_experiment` ? variant specs + hashes + the label-set fingerprint travel with
it.

        ``honest_stats`` (default True) **adds** a ``"honest"`` block ? per-metric bootstrap
CIs, a
```

```
        paired ?F1 CI + exact sign test (C1), and F1@k + segment-count ratio (C4) ?
*alongside* the
        existing bare-mean fields (additive: nothing existing changes). Set False for the
legacy
        shape. This is the measurement-honesty wiring (`ANALYZERS_AND_RECONCILERS.md`
§13/C1+C4): a
        bare LOVO mean at n?6 is not trustworthy on its own.
        """
        if len(videos) < 1:
            raise ExperimentError("compare needs at least one labeled video")
        eval_a = evaluate_variant(videos, variant_a, iou, honest)
        eval_b = evaluate_variant(videos, variant_b, iou, honest)

        delta = {m: eval_b["aggregate"][m] - eval_a["aggregate"][m] for m in _METRICS}
        delta_pct = {m: pct_improvement(eval_a["aggregate"][m], eval_b["aggregate"][m]) for
m in _METRICS}

        by_video_a = {p["video"]: p for p in eval_a["per_video"]}
        per_video_delta = [
            {"video": p["video"], **{m: p[m] - by_video_a[p["video"]][m] for m in _METRICS}}
            for p in eval_b["per_video"]
        ]

        # One fingerprint over the whole label set ? detects "the deltas were measured
against
        # different labels" the same way regression.gt_hash guards the no-regression
baseline.
        all_gts: List[Interval] = [iv for vc in videos for iv in (vc.gts or [])]

        result: Dict[str, Any] = {
            "iou": iou,
            "label_set_hash": gt_hash(all_gts),
            "num_videos": len(videos),
            "variant_a": eval_a,
            "variant_b": eval_b,
            "delta": delta,
            "delta_pct": delta_pct,
            "per_video_delta": per_video_delta,
        }
        if honest_stats:
            result["honest"] = honest_summary(videos, eval_a, eval_b)
        return result


    # ------------------------------------------------------------- SxS on unlabeled video


    def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
                          agree_iou: float = 0.5) -> Dict[str, Any]:
        """Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).

        With no labels there is no lift to measure ? instead this surfaces where the two
        variants **agree vs diverge**, which is exactly what a human reviews (and what two
        highlight reels would show side by side):

          - ``agreed``  ? windows both variants found (matched at ``agree_iou``);
          - ``a_only``  ? A fired, B suppressed (B's added precision, if A was over-firing);
          - ``b_only``  ? B fired, A did not (B's new detections ? real recall or new FPs:
            the human / a later golden label adjudicates).

        Both variants must be predictable without labels: a learned variant therefore needs
a
        pinned ``classifier_model`` (you cannot train on an unlabeled video). Reuses
        `metrics.match_intervals` ? no new matching logic.
        """
```

```
        preds_a = predict(variant_a, video)
        preds_b = predict(variant_b, video)
        mr = match_intervals(preds_a, preds_b, agree_iou)

        agreed = [{"a": preds_a[m.pred_index], "b": preds_b[m.gt_index], "iou": m.iou}
                  for m in mr.matches]
        agree_ious = [m.iou for m in mr.matches]
        a_only = [preds_a[i] for i in mr.unmatched_pred_indices]
        b_only = [preds_b[i] for i in mr.unmatched_gt_indices]

        def _dur(ivs: List[Interval]) -> float:
            return sum(e - s for s, e in ivs)

        return {
            "video": video.name,
            "agree_iou": agree_iou,
            "variant_a": variant_a.name,
            "variant_b": variant_b.name,
            "num_a": len(preds_a),
            "num_b": len(preds_b),
            "num_agreed": len(agreed),
            "mean_agree_iou": (sum(agree_ious) / len(agree_ious)) if agree_ious else 0.0,
            "duration_a": _dur(preds_a),
            "duration_b": _dur(preds_b),
            "agreed": agreed,
            "a_only": a_only,
            "b_only": b_only,
        }


    # ---------------------------------------------------------------------- leaderboard / DB


    def record_experiment(result: Dict[str, Any], path: str = DEFAULT_LEADERBOARD_PATH,
                          timestamp: Optional[str] = None) -> Dict[str, Any]:
        """Append a compact, reproducible record of a `compare()` result to the JSON
        leaderboard (`EXPERIMENT_HARNESS.md` §6: experiments are records). Generalises
        `regression_baseline.json`; deliberately the size of a baseline file, not a service
        (§9). Returns the row written. ``timestamp`` is injectable for deterministic tests.
        """
        row: Dict[str, Any] = {
            "timestamp": timestamp or datetime.now(timezone.utc).isoformat(),
            "iou": result.get("iou"),
            "label_set_hash": result.get("label_set_hash"),
            "num_videos": result.get("num_videos"),
            "variant_a": {"name": result["variant_a"]["variant"],
                          "spec_hash": result["variant_a"]["spec_hash"],
                          "aggregate": result["variant_a"]["aggregate"]},
            "variant_b": {"name": result["variant_b"]["variant"],
                          "spec_hash": result["variant_b"]["spec_hash"],
                          "aggregate": result["variant_b"]["aggregate"]},
            "delta": result.get("delta"),
        }
        # Record the honest ?F1 (CI + sign test) when present, so the leaderboard carries
    the
        # promotion-grade evidence, not just the bare-mean delta (additive ? C1).
        honest = result.get("honest") or {}
        if honest.get("paired_delta_f1"):
            row["honest_delta_f1"] = honest["paired_delta_f1"]
        entries: List[Dict[str, Any]] = []
        if os.path.isfile(path):
            with open(path, encoding="utf-8") as f:
                entries = json.load(f)
        entries.append(row)
        os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
```

```
535         with open(path, "w", encoding="utf-8") as f:
536             json.dump(entries, f, indent=2)
537         logger.info("recorded experiment %s vs %s (?f1=%.3f) -> %s",
538                     row["variant_a"]["name"], row["variant_b"]["name"],
539                     (result.get("delta") or {}).get("f1", 0.0), path)
540         return row
541
542
543     def load_video_candidates(db: Any, video_id: str, *, detector: Optional[str] = None,
544                               gts: Optional[List[Interval]] = None,
545                               feature_names: Sequence[str] = ()) -> VideoCandidates:
546         """Bridge persisted detection ? `VideoCandidates` via
`Database.query_candidate_segments`.
547
548         ``feature_names`` selects which persisted features to lift out of each row's
``stats``
549         JSON (``"duration"`` is derived from the window bounds). This is the only DB-
touching
550         function; the comparison itself is pure and offline. Exercised owner-side on a real
DB,
551         but works against any `Database` (including an in-memory one in tests).
552         """
553         rows = db.query_candidate_segments(video_id, detector=detector)
554         intervals: List[Interval] = [(float(r["start_time"]), float(r["end_time"])) for r in
rows]
555         features: Dict[str, List[float]] = {}
556         for name in feature_names:
557             if name == "duration":
558                 features[name] = [float(r["end_time"]) - float(r["start_time"]) for r in
rows]
559             else:
560                 features[name] = [float((r.get("stats") or {}).get(name, 0.0)) for r in
rows]
561         return VideoCandidates(name=video_id, intervals=intervals, features=features,
gts=gts)
562
```

28. assistant (2026-06-18T01:34:15.685Z)

`record_experiment` at line 516-521 accesses `result["variant_a"]["variant"]`, `["spec_hash"]`, `["aggregate"]` directly. But `result` comes from `compare()` which always builds those keys. Not external.

Line 530-532: reads leaderboard JSON file, `entries = json.load(f)` then `entries.append(row)`. If the existing leaderboard file is corrupt (a JSON object, not a list), `.append` would fail ? but that's a maintainer-written file, low concern.

Let me look at the remaining persisted-data readers: `fusion_golden.py`, `per_user_regression.py`, `per_user_gate.py`, `promotion.py`, `distill_local.py`, and the `harvest.py`/`run_signals.py` reporting which may read DB. Let me also check `tools/export.py` and `rally_slicer.py`. Let me batch-read several.

29. user (2026-06-18T01:34:16.523Z)

```
1       """Offline golden-set person-feature extractor ? the fusion-P3 measurement's GPU step.
2
3       Mirrors `distill_local`'s CSV-driven flow but ADDS the ~5 person-cue features per
shuttle
4       candidate window, by running `PersonDetector` over the ORIGINAL video. It reuses the
5       EXISTING golden trajectory CSVs (so **no WASB re-run**), needs **no Gemini and no DB**,
and
6       writes one small JSON per video. The LOVO comparison (`fusion_compare.py`) then runs
100% on
7       CPU offline. So the GPU touches exactly one thing ? person detection ? and the lift is
8       measured by the shipped experiment harness; person features enter only as feature
```

```
columns the
9       learned scorer weights (no special-casing, default-OFF until the golden ?F1 justifies
them).
10
11      Run (on the GPU box):
12          python -m backend.eval.fusion_golden --manifest output/human_lovo_manifest.json
--out-dir output
13
14      Each output `output/<name>_golden_features.json` is the replayable feature substrate ?
re-run
15      `fusion_compare` with different variants/thresholds forever without re-detecting on the
GPU.
16      """
17      from __future__ import annotations
18
19      import argparse
20      import json
21      import os
22      import time
23      from typing import Any, Dict, List, Optional
24
25      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
26      from backend.pipeline.detectors import rally_gate
27      from backend.pipeline.detectors import fusion_features as ff
28      from backend.eval import distill_local
29      from backend.eval.calibrate_wasb import load_golden_gts
30      from backend.eval.training import label_candidates
31      from backend.utils.run_telemetry import RunTelemetry
32
33      # The 5 fps/resolution-invariant shuttle features distill_local already computes.
34      SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
35      # The 5 person-cue features (fusion_features.PERSON_FEATURE_NAMES).
36      PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)
37
38
39      def _derive_video_path(spec: Dict[str, Any]) -> str:
40          """Original MP4 for a manifest entry ? explicit `video_path`, else derived from the
41          golden labels path (``X.rallies.csv`` -> ``X.MP4``, the golden-set naming
convention)."""
42          if spec.get("video_path"):
43              return str(spec["video_path"])
44          labels = str(spec["labels"])
45          if labels.endswith(".rallies.csv"):
46              return labels[: -len(".rallies.csv")] + ".MP4"
47          raise ValueError(f"{spec.get('name')}: no video_path and labels {labels!r} isn't
*.rallies.csv")
48
49
50      def extract_video(spec: Dict[str, Any], detector: Optional[Any] = None,
51                        frame_skip: int = 3, iou: float = 0.5,
52                        log_every_sec: float = 15.0,
53                        telemetry: Optional[Any] = None) -> Dict[str, Any]:
54          """One golden video -> a feature record (shuttle + person features per candidate
window).
55
56          Returns ``{name, fps, frame_width,
candidates:[{start,end,shuttle{},person{},label}], gts}``.
57          `detector` (anything with ``detections_over_windows``) is injectable for GPU-free
tests;
58          when None a real `PersonDetector` is built (loads torchvision on first use).
59          `telemetry` (a `RunTelemetry` or None) records per-frame velocity + machine-health
snapshots
60          from the detection progress callback so a hard kill leaves a forensic runlog.
61          """
62          fps, fw = float(spec["fps"]), float(spec["frame_width"])
```

```
        traj = str(spec["trajectory"])
        points = TrackNetRunner.parse_trajectory_csv(traj)
        intervals, x_shuttle = distill_local.build_candidates(traj, fps, fw)
        gts = load_golden_gts(str(spec["labels"]))
        labels = label_candidates(intervals, gts, iou)
        # Net axis for the person two-sided split ? reuse rally_gate's camera-agnostic
estimate
        # (parity with the shuttle net-crossing gate) rather than hard-coding an
orientation.
        axis, net_px, _span = rally_gate.estimate_play_axis(points)
        video_path = _derive_video_path(spec)

        if detector is None:
            from backend.pipeline.detectors.person_detector import PersonDetector
            detector = PersonDetector({"indexing": {"use_gpu": True}})

        # Single-pass person detection over ALL candidate windows (open the video ONCE, read
        # forward ? no per-window seek). Then compute features per window (pure, no GPU).
        n = len(intervals)
        win_frames = [(round(s * fps), round(e * fps)) for (s, e) in intervals]
        print(f"[fusion_golden]   {spec['name']}: {n} windows ? person-detecting in one
pass...", flush=True)
        t0 = time.monotonic()
        state = {"last": t0}

        def _progress(done: int, total: int) -> None:
            if telemetry is not None:
                telemetry.snapshot(done, total, phase=spec["name"])
            now = time.monotonic()
            if now - state["last"] >= log_every_sec or done >= total:
                el = now - t0
                rate = done / el if el > 0 else 0.0
                eta = (total - done) / rate if rate > 0 else 0.0
                pct = 100 * done // total if total else 0
                print(f"[fusion_golden]   {spec['name']}: detect frame {done}/{total}
({pct}%) "
                      f"| {el:.0f}s elapsed | ETA {eta:.0f}s", flush=True)
                state["last"] = now

        per_window = detector.detections_over_windows(video_path, win_frames, frame_skip,
                                                      progress_cb=_progress)

        candidates: List[Dict[str, Any]] = []
        for i, ((s, e), xs) in enumerate(zip(intervals, x_shuttle)):
            det = per_window[i]
            det_fw = det.get("frame_width") or fw
            det_fh = det.get("frame_height") or 0.0
            # Normalise the (pixel) net coord to 0-1 on the play axis, matching the foot-
point
            # normalisation fusion_features uses (x/width, y/height).
            if axis == "x":
                net_norm = net_px / det_fw if det_fw > 0 else 0.5
            else:
                net_norm = net_px / det_fh if det_fh > 0 else 0.5
            sf, ef = win_frames[i]
            shuttle_in = [p for p in points if sf <= p.frame <= ef]
            person = ff.compute_person_features(det["frames"], shuttle_in, det_fw, det_fh,
                                                court_polygon=None, net=(axis, net_norm))
            candidates.append({
                "start": float(s), "end": float(e),
                "shuttle": {k: float(v) for k, v in zip(SHUTTLE_FEATURE_NAMES, xs)},
                "person": {k: float(v) for k, v in person.items()},
                "label": int(labels[i]),
            })
        return {
```

```
            "name": spec["name"], "fps": fps, "frame_width": fw,
            "candidates": candidates,
            "gts": [[float(a), float(b)] for a, b in gts],
        }


    def run(manifest_path: str, out_dir: str, frame_skip: int = 3, iou: float = 0.5,
            fresh: bool = False, smallest_first: bool = False) -> List[str]:
        with open(manifest_path, encoding="utf-8") as f:
            specs = json.load(f)
        if smallest_first:
            # Process small videos first for quick feedback (the slow full match lands
last);
            # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
row/frame).
            specs = sorted(specs, key=lambda s: (os.path.getsize(s["trajectory"])
                                                 if os.path.isfile(s.get("trajectory", ""))
else 0))
        os.makedirs(out_dir, exist_ok=True)
        # Run-telemetry black box: a long GPU run can be OS-killed with no traceback, so
keep a tiny
        # rotating runlog of velocity + machine health for the post-mortem. Constructing it
must never
        # break extraction, so guard the build and fall back to no telemetry on any error.
        telem: Optional[RunTelemetry]
        try:
            telem = RunTelemetry(os.path.join(out_dir, "run_telemetry.jsonl"),
label="fusion_golden")
        except Exception:  # noqa: BLE001 - telemetry wiring must never break the run it
observes
            telem = None
        written: List[str] = []
        for vi, spec in enumerate(specs, 1):
            out = os.path.join(out_dir, f"{spec['name']}_golden_features.json")
            # Resumable: skip a video whose JSON already exists (a re-run picks up where a
prior
            # run left off instead of re-detecting). --fresh forces a redo.
            if not fresh and os.path.isfile(out):
                print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: already
extracted "
                      f"-> {out} (use --fresh to redo)", flush=True)
                written.append(out)
                continue
            print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: extracting...",
flush=True)
            t0 = time.monotonic()
            if telem is not None:
                telem.event("video_start", spec["name"])
            try:
                data = extract_video(spec, frame_skip=frame_skip, iou=iou, telemetry=telem)
            except Exception as e:  # noqa: BLE001 - record machine state at death, then re-
raise
                if telem is not None:
                    telem.event("error", spec["name"], err=str(e))
                raise
            n_pos = sum(c["label"] for c in data["candidates"])
            with open(out, "w", encoding="utf-8") as f:
                json.dump(data, f, indent=2)
            if telem is not None:
                telem.event("video_done", spec["name"],
                            windows=len(data["candidates"]), positives=n_pos)
            print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: DONE in
{time.monotonic() - t0:.0f}s "
                  f"| {len(data['candidates'])} windows ({n_pos} positive) |
{len(data['gts'])} golden rallies "
```

```
175                  f"-> {out}", flush=True)
176              written.append(out)
177          return written
178
179
180      def main() -> None:
181          ap = argparse.ArgumentParser(description="Extract golden-set shuttle+person features
(fusion P3 GPU step).")
182          ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
183          ap.add_argument("--out-dir", default="output")
184          ap.add_argument("--frame-skip", type=int, default=3)
185          ap.add_argument("--iou", type=float, default=0.5)
186          ap.add_argument("--fresh", action="store_true", help="re-extract even if a JSON
already exists")
187          ap.add_argument("--smallest-first", action="store_true",
188                          help="process smaller videos first for quick feedback (LOVO is
order-independent)")
189          args = ap.parse_args()
190          run(args.manifest, args.out_dir, args.frame_skip, args.iou, fresh=args.fresh,
191              smallest_first=args.smallest_first)
192
193
194      if __name__ == "__main__":
195          main()
196
```

30. assistant (2026-06-18T01:34:23.959Z)

Here's a candidate. In `extract_video` (line 62-64), `spec["fps"]`, `spec["frame_width"]`, `spec["trajectory"]`, `spec["labels"]`, `spec["name"]` are accessed unguarded from the manifest JSON. The manifest is an external file loaded at line 131 (`specs = json.load(f)`). `_derive_video_path` (line 39-47) uses `spec.get("video_path")` and `spec["labels"]` ? guarded for video_path but `spec["labels"]` is unguarded.

This is a manifest the operator writes by hand. A missing key would KeyError. But this is an operator-facing eval CLI; a KeyError on a malformed manifest is arguably acceptable behavior (loud failure). Let me note it as low-confidence and keep looking for a higher-value defect where renamed/typed-differently data is more likely.

Let me check `per_user_regression.py`, `per_user_gate.py`, `promotion.py`, `distill_local.py`, and the reporting modules which read DB rows / persisted JSON.

31. user (2026-06-18T01:34:24.788Z)

```
1        """Per-user NO-REGRESSION guard ? the missing per-user analogue of `regression.py`.
2
3        The shared golden set has a disciplined no-regression baseline (`regression.py` +
4        ``regression_baseline.json``): a candidate is gated so a global change can never
silently
5        make the *shared* corpus worse. Personalization has no such rail ? a deliberately-tuned
6        per-user model "has no shared baseline to regress against", so a per-user retrain or a
7        baseline upgrade could quietly make ONE user worse than the model they're already
running.
8        This module builds that missing per-user regression baseline, mirroring the shared
discipline.
9
10       The decision it makes is deliberately narrow and CONSERVATIVE ? it is a *guarantee
against
11       regression*, not a promotion decision (that is `per_user_gate.should_promote_for_user`):
12
13         * **CONTROL** = the user's *frozen incumbent* ? the variant/config currently active
FOR THAT
14           USER. For a brand-new user with nothing personalized yet, the incumbent simply IS
the
15           shipped baseline. This is the thing we must never regress below.
```

```
16        * **CANDIDATE** = the proposed replacement (a per-user retrain, or a global baseline
upgrade
17          rolled out to this user).
18
19      Both are evaluated on the user's own held-out videos (`experiment.evaluate_variant`,
LOVO-honest
20      for a learned recipe ? no new scoring math), paired by video, and summarised with the
shipped
21      honest statistics (`eval_stats.paired_delta_ci` ? bootstrap ?F1 CI + exact sign test).
Then:
22
23        * **regression** ? the candidate is *significantly worse*: the paired ?F1 (candidate ?
control)
24          95% CI excludes 0 on the WRONG side (``ci_high < 0``). ? REJECT, keep the incumbent.
25        * **clear improvement** ? the candidate is *significantly better*: the CI excludes 0
on the
26          right side AND the sign test agrees (candidate wins on strictly more videos than it
loses).
27          ? ACCEPT the swap.
28        * **tie / insufficient evidence** ? the CI spans 0 (the small-N near-null regime):
there is no
29          evidence the swap helps, so we never take the risk. ? KEEP THE INCUMBENT.
30
31      So we only ACCEPT a swap we can *defend* and only ever toward a measured improvement;
everything
32      ambiguous keeps the incumbent. That is strictly stronger than "don't regress" (it also
refuses
33      risk-neutral churn), which is exactly the safety the personalization feature was
missing.
34
35      A small per-user baseline record is keyed by a ``user_id::stage`` string (``user_id``
nullable ?
36      ``None`` = the local single-user deployment; design-for-north-star, implement-for-
current) and
37      saved/loaded as JSON, a sibling to `regression.py`'s ``regression_baseline.json``
approach.
38
39      Pure-CPU, deterministic given ``seed`` (the only randomness is the CI bootstrap); no GPU
/ torch /
40      cv2 / file I/O beyond the explicit baseline save/load. See `docs/EXPERIMENT_HARNESS.md`
+
41      `docs/ANALYZERS_AND_RECONCILERS.md` §13/C1.
42      """
43      from __future__ import annotations
44
45      import json
46      import logging
47      import os
48      from dataclasses import asdict, dataclass, field
49      from typing import Any, Dict, List, Optional, Sequence
50
51      from backend.eval import eval_stats as _stats
52      from backend.eval import experiment
53      from backend.eval.experiment import Variant, VideoCandidates
54
55      logger = logging.getLogger(__name__)
56
57      __all__ = [
58          "DEFAULT_PER_USER_BASELINE_PATH",
59          "PerUserRegressionResult",
60          "PerUserBaseline",
61          "baseline_key",
62          "load_baselines",
63          "save_baseline",
64          "evaluate_no_regression",
```

```
65      ]
66
67      # Tracked location (NOT under the gitignored output/), sibling to
68      # regression.DEFAULT_BASELINE_PATH so a fresh checkout / CI can load a committed per-
user
69      # baseline. NULL user_id (local single-user) keys as the literal "local".
70      DEFAULT_PER_USER_BASELINE_PATH = os.path.join("eval_baselines",
"per_user_baseline.json")
71
72
73      # ------------------------------------------------------------------------ result
74
75      @dataclass
76      class PerUserRegressionResult:
77          """The per-user no-regression decision plus the evidence behind it.
78
79          - ``accept`` ? swap the candidate in for this user (only ever ``True`` on a
*defended*
80            improvement: the ?F1 CI clears 0 on the right side AND the sign test agrees).
81          - ``keep_incumbent`` ? the user keeps their frozen incumbent (control). Always the
exact
82            complement of ``accept`` ? a swap is taken or it is not.
83          - ``regressed`` ? the candidate is *significantly worse* than the incumbent (CI
excludes 0
84            on the wrong side). A True here always implies ``accept=False``.
85          - ``reason`` ? short human-readable explanation, for logging / audit.
86          - the remaining fields surface the paired evidence (per-variant mean held-out F1,
mean ?F1,
87            CI bounds, and the sign test) so a caller can audit exactly why the swap was/
wasn't taken.
88          """
89
90          accept: bool
91          keep_incumbent: bool
92          reason: str
93          n: int
94          control_f1: float
95          candidate_f1: float
96          mean_delta: float
97          ci_low: float
98          ci_high: float
99          ci_excludes_zero: bool
100         regressed: bool
101         sign_test: Dict[str, float] = field(default_factory=dict)
102
103         def as_dict(self) -> Dict[str, Any]:
104             """Plain-dict view (for JSON logging / telemetry)."""
105             return asdict(self)
106
107
108     # ------------------------------------------------------------------ per-user baseline
store
109
110     @dataclass
111     class PerUserBaseline:
112         """A snapshotted per-user incumbent record ? the per-user analogue of a regression
baseline.
113
114         Keyed by ``user_id::stage``. ``user_id`` is nullable (``None`` ? the local single-
user
115         deployment, keyed as the literal ``"local"``). ``control_spec_hash`` pins WHICH
incumbent
116         variant the recorded numbers were measured for, so a later check against a different
117         incumbent is detectable rather than silently trusted (mirrors regression.py's
118         iou/detector/gt fingerprint guards).
```

```
        """
        user_id: Optional[str]
        stage: str
        control_f1: float
        control_spec_hash: str
        n: int
        iou: float

        def as_dict(self) -> Dict[str, Any]:
            return asdict(self)


    def baseline_key(user_id: Optional[str], stage: str) -> str:
        """Stable ``user_id::stage`` key. A NULL ``user_id`` (local single-user) keys as
``local``."""
        return f"{user_id if user_id is not None else 'local'}::{stage}"


    def load_baselines(path: str = DEFAULT_PER_USER_BASELINE_PATH) -> Dict[str, dict]:
        """Load the per-user baseline file (returns ``{}`` if absent)."""
        if not os.path.isfile(path):
            return {}
        with open(path, encoding="utf-8") as f:
            return json.load(f)


    def save_baseline(baseline: PerUserBaseline,
                      path: str = DEFAULT_PER_USER_BASELINE_PATH) -> str:
        """Snapshot a user's incumbent numbers as the new per-user baseline, keyed
``user_id::stage``.

        Round-trips through `load_baselines`; returns the key written. Mirrors
        `regression.save_baseline` (atomic-ish read-modify-write of the one JSON file).
        """
        baselines = load_baselines(path)
        key = baseline_key(baseline.user_id, baseline.stage)
        baselines[key] = baseline.as_dict()
        os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
        with open(path, "w", encoding="utf-8") as f:
            json.dump(baselines, f, indent=2, sort_keys=True)
        logger.info("Saved per-user baseline %s (control_f1=%.3f n=%d)",
                    key, baseline.control_f1, baseline.n)
        return key


    # ---------------------------------------------------------------------- the guard

    def _mean(xs: Sequence[float]) -> float:
        return sum(xs) / len(xs) if xs else 0.0


    def evaluate_no_regression(
        videos: Sequence[VideoCandidates],
        control: Variant,
        candidate: Variant,
        *,
        user_id: Optional[str] = None,
        stage: str = "final",
        iou: float = 0.5,
        honest: bool = True,
        p_threshold: float = 0.05,
        confidence: float = 0.95,
        n_boot: int = 2000,
        seed: int = 0,
```

```
182         eps: float = 0.0,
183         baseline_path: Optional[str] = None,
184     ) -> PerUserRegressionResult:
185         """Guard a per-user swap: ACCEPT only if ``candidate`` does NOT regress vs the
``control``.
186
187         ``control`` is the user's FROZEN incumbent (their currently-active variant ? for a
188         brand-new user this is just the shipped baseline). ``candidate`` is the proposed
189         replacement (a per-user retrain or a baseline upgrade). Both are scored on the
user's own
190         ``videos`` via `experiment.evaluate_variant` (LOVO-honest for a learned recipe ? no
new
191         scoring math), the per-video held-out F1s are paired by video, and the paired ?F1
192         (candidate ? control) is summarised with `eval_stats.paired_delta_ci` (bootstrap CI
+
193         exact sign test). The decision (faithful to the module docstring):
194
195           * **regression** (``ci_high < 0`` ? candidate significantly worse): REJECT, keep
incumbent.
196           * **clear improvement** (CI excludes 0 on the right side AND the sign test agrees
?
197             candidate wins on strictly more videos than it loses): ACCEPT the swap.
198           * **tie / insufficient evidence** (CI spans 0): KEEP THE INCUMBENT (conservative;
never
199             accept a risky swap).
200
201         ``user_id``/``stage`` key the baseline record (``user_id=None`` ? local single-
user). When
202         ``baseline_path`` is given the incumbent's measured numbers are snapshotted there
(the swap
203         is then evaluated as usual). Deterministic given ``seed``. Raises `ExperimentError`
(from
204         `evaluate_variant`) on unlabeled videos or a learned recipe with too few videos for
LOVO.
205         """
206         eval_control = experiment.evaluate_variant(videos, control, iou, honest)
207         eval_candidate = experiment.evaluate_variant(videos, candidate, iou, honest)
208
209         # Per-video held-out F1, video-aligned (evaluate_variant iterates ``videos`` in
order, so
210         # both lists are pairable by position ? exactly how honest_summary pairs B ? A).
211         control_f1s: List[float] = [p["f1"] for p in eval_control["per_video"]]
212         candidate_f1s: List[float] = [p["f1"] for p in eval_candidate["per_video"]]
213         n = min(len(control_f1s), len(candidate_f1s))
214
215         control_mean = _mean(control_f1s)
216         candidate_mean = _mean(candidate_f1s)
217
218         if n == 0:
219             # No paired videos at all ? no evidence either way ? keep the incumbent.
220             sign_test: Dict[str, float] = _stats.paired_sign_test([], eps)
221             result = PerUserRegressionResult(
222                 accept=False, keep_incumbent=True,
223                 reason="no held-out videos to evaluate; keeping incumbent",
224                 n=0, control_f1=control_mean, candidate_f1=candidate_mean,
225                 mean_delta=0.0, ci_low=0.0, ci_high=0.0, ci_excludes_zero=False,
226                 regressed=False, sign_test=sign_test,
227             )
228         else:
229             delta = _stats.paired_delta_ci(
230                 control_f1s, candidate_f1s,
231                 confidence=confidence, n_boot=n_boot, seed=seed, eps=eps,
232             )
233             mean_delta = float(delta["mean_delta"])
234             ci_low = float(delta["ci_low"])
```

```
235             ci_high = float(delta["ci_high"])
236             ci_excludes_zero = bool(delta["ci_excludes_zero"])
237             sign_test = delta["sign_test"]
238             sign_p = float(sign_test.get("p_value", 1.0))
239             sign_wins = float(sign_test.get("wins", 0.0))
240             sign_losses = float(sign_test.get("losses", 0.0))
241
242             # A regression = the candidate is significantly WORSE: the ?F1 CI excludes 0
entirely
243             # on the wrong side (its whole high end is below 0). Never accept such a swap.
244             regressed = ci_excludes_zero and ci_high < 0.0
245             # A defended improvement = the CI clears 0 on the RIGHT side AND the sign test
agrees
246             # (more candidate-wins than losses, significant) ? the same promotion-grade bar
the
247             # global eval uses, applied per user. Everything else is a tie / insufficient
evidence.
248             sign_agrees = (sign_p <= p_threshold) and (sign_wins > sign_losses)
249             improved = ci_excludes_zero and ci_low > 0.0 and sign_agrees
250
251             if regressed:
252                 accept = False
253                 reason = (
254                     f"REGRESSION: candidate ?F1 95% CI [{ci_low:.4f}, {ci_high:.4f}] "
255                     f"is entirely below 0 (n={n}); keeping incumbent"
256                 )
257             elif improved:
258                 accept = True
259                 reason = (
260                     f"accepted: candidate ?F1 95% CI [{ci_low:.4f}, {ci_high:.4f}] excludes
0 "
261                     f"(improvement), sign test {int(sign_wins)}-{int(sign_losses)} "
262                     f"(p={sign_p:.4f} <= {p_threshold}); swapping in candidate"
263                 )
264             else:
265                 accept = False
266                 reason = (
267                     f"tie / insufficient evidence: candidate ?F1 95% CI "
268                     f"[{ci_low:.4f}, {ci_high:.4f}] spans 0 (n={n}); keeping incumbent "
269                     "(no defended improvement, never a risky swap)"
270                 )
271
272             result = PerUserRegressionResult(
273                 accept=accept, keep_incumbent=not accept, reason=reason,
274                 n=n, control_f1=control_mean, candidate_f1=candidate_mean,
275                 mean_delta=mean_delta, ci_low=ci_low, ci_high=ci_high,
276                 ci_excludes_zero=ci_excludes_zero, regressed=regressed, sign_test=sign_test,
277             )
278
279         # Snapshot the incumbent's measured numbers as this user's baseline when asked ? the
280         # per-user analogue of regression.save_baseline. Does not change the decision above.
281         if baseline_path is not None:
282             save_baseline(
283                 PerUserBaseline(
284                     user_id=user_id, stage=stage,
285                     control_f1=control_mean, control_spec_hash=control.spec_hash(),
286                     n=n, iou=iou,
287                 ),
288                 path=baseline_path,
289             )
290
291         return result
292
```

32. assistant (2026-06-18T01:34:29.251Z)

At line 237, `sign_test = delta["sign_test"]` and then `.get()` on it ? guarded. `delta["mean_delta"]`, `["ci_low"]` etc are from `paired_delta_ci` which is an internal builder. Fine.

Let me check `promotion.py`, `per_user_gate.py`, `per_user_eval.py`, and the reporting modules `harvest.py`/`run_signals.py`, plus `distill_local.py`. Let me batch.

33. user (2026-06-18T01:34:30.277Z)

```
1       """Promotion gate with the eval?serve guard: a cue is promotable ONLY if it does not
regress
2       at the SERVED windowing.
3
4       **Why this exists.** ``per_user_gate.should_promote_for_user`` already grades a B?A lift
5       honestly (min-N floor + ?F1 CI excludes 0 + paired sign test). But it grades whatever
scores
6       it is handed ? and the foundational bug was that those scores came from the *recall-
oriented
7       PROPOSAL* windowing (``backend.eval.windowing.PROPOSAL``), which production never
serves. So a
8       cue could clear the bar on a windowing nobody runs (Gate-A: proposal ?F1 **+0.074**)
while
9       regressing on the served one (**?0.008**). Grading harder isn't the fix; **measuring on
the
10      served windowing** is.
11
12      This module composes ? it adds no new statistics. It layers ONE rule on top of
13      ``should_promote_for_user``:
14
15          A cue is promotable only if, at the **SERVED** windowing, it WINS or at least DOES
NOT
16          REGRESS (its served paired ?F1 mean ? ``-served_regress_eps``, and ? when its served
CI is
17          estimable ? that CI's lower bound does not sit clearly in regression territory).
18
19      A proposal-windowing win is still *reported* (it's useful signal for where to look
next), but it
20      can never carry a promotion on its own. The decision returns BOTH the served and the
proposal
21      verdicts so the eval?serve contrast is explicit and auditable.
22
23      Pure, deterministic, stdlib-only over ``eval_stats``/``per_user_gate``. Additive:
existing eval
24      paths are untouched until a caller routes its A/B through
:func:`promote_if_served_safe`.
25      """
26      from __future__ import annotations
27
28      from dataclasses import dataclass, field
29      from typing import Any, Dict, Optional, Sequence
30
31      from backend.eval.eval_stats import paired_delta_ci
32      from backend.eval.per_user_gate import PromotionDecision, should_promote_for_user
33
34      __all__ = ["ServedGateResult", "promote_if_served_safe", "served_regresses"]
35
36
37      def served_regresses(a_scores: Sequence[float], b_scores: Sequence[float], *,
38                           eps: float = 0.0, n_boot: int = 2000, seed: int = 0) -> Dict[str,
Any]:
39          """Does variant B REGRESS variant A at the served windowing? (the eval?serve guard
core).
40
41          ``a_scores``/``b_scores`` are per-video held-out scores **measured at the SERVED
windowing**
```

```
42          (aligned by video). Returns the paired B?A summary (mean ?F1 + bootstrap CI + sign
test) plus
43          a single ``regresses`` boolean:
44
45            - ``regresses=True``  when the served mean ?F1 is below ``-eps`` (a real served
drop), OR
46              the served ?F1 CI lies **entirely** below ``-eps`` (a confidently-negative
band).
47            - ``regresses=False`` when the cue holds the line at the served operating point
(mean ?
48              ``-eps`` and the CI is not a confident net-negative).
49
50          ``eps`` is the regression tolerance: 0.0 = "must not drop at all on the served
mean". A tiny
51          positive ``eps`` lets noise-level wobble through (e.g. 0.005 ? half an F1 point at
n?6).
52          """
53          delta = paired_delta_ci(a_scores, b_scores, n_boot=n_boot, seed=seed)
54          mean_delta = float(delta["mean_delta"])
55          ci_low = float(delta["ci_low"])
56          ci_high = float(delta["ci_high"])
57          # Regression if the served mean dropped beyond tolerance, OR the whole CI is net-
negative.
58          mean_regresses = mean_delta < -eps
59          ci_confidently_negative = ci_high < -eps
60          regresses = bool(mean_regresses or ci_confidently_negative)
61          return {
62              "regresses": regresses,
63              "mean_delta": mean_delta,
64              "ci_low": ci_low,
65              "ci_high": ci_high,
66              "ci_excludes_zero": bool(delta["ci_excludes_zero"]),
67              "sign_test": delta["sign_test"],
68              "n": int(delta["n"]),
69              "eps": float(eps),
70          }
71
72
73      @dataclass
74      class ServedGateResult:
75          """A promotion decision that BOTH measures the cue on the served windowing AND
reports the
76          proposal-windowing view, so the eval?serve contrast is never hidden.
77
78          - ``promote`` ? final verdict. ``True`` only when the underlying honest gate would
promote
79            *and* the served check did not veto (the cue does not regress at the served
windowing).
80          - ``served_safe`` ? the eval?serve guard's verdict in isolation (cue does not
regress served).
81          - ``vetoed_by_served`` ? ``True`` when the proposal/base gate WOULD have promoted
but the
82            served regression check overruled it (the exact Gate-A failure mode this module
prevents).
83          - ``base_decision`` ? the ``PromotionDecision`` from ``should_promote_for_user`` on
the
84            *promotion-grade* scores (served by default ? see :func:`promote_if_served_safe`).
85          - ``served`` / ``proposal`` ? the paired B?A summary at each windowing (means + CIs
+ sign
86            tests), so a reviewer sees "+0.074 proposal / ?0.008 served" side by side.
87          - ``reason`` ? short human-readable explanation of the final verdict.
88          """
89
90          promote: bool
91          served_safe: bool
```

```
92          vetoed_by_served: bool
93          reason: str
94          base_decision: PromotionDecision
95          served: Dict[str, Any]
96          proposal: Optional[Dict[str, Any]] = field(default=None)
97
98          def as_dict(self) -> Dict[str, Any]:
99              return {
100                 "promote": self.promote,
101                 "served_safe": self.served_safe,
102                 "vetoed_by_served": self.vetoed_by_served,
103                 "reason": self.reason,
104                 "base_decision": self.base_decision.as_dict(),
105                 "served": self.served,
106                 "proposal": self.proposal,
107             }
108
109
110     def promote_if_served_safe(
111         served_a: Sequence[float],
112         served_b: Sequence[float],
113         *,
114         proposal_a: Optional[Sequence[float]] = None,
115         proposal_b: Optional[Sequence[float]] = None,
116         structural: bool = False,
117         min_n: int = 5,
118         p_threshold: float = 0.05,
119         served_regress_eps: float = 0.0,
120         n_boot: int = 2000,
121         seed: int = 0,
122     ) -> ServedGateResult:
123         """Promote a cue ONLY if it does not regress at the SERVED windowing (the
foundational rule).
124
125         The honest bar (``should_promote_for_user``: min-N floor + ?F1 CI excludes 0 + sign
test) is
126         applied to the **served** scores ? the operating point production runs ? so a
proposal-only
127         win can never carry a promotion. The optional ``proposal_*`` scores are scored too
and
128         reported alongside, purely to surface the eval?serve contrast (they NEVER promote on
their
129         own).
130
131         Args:
132           served_a/served_b: per-video held-out scores for variant A (control) and B (the
cue),
133             measured at the **SERVED** windowing. These drive the decision.
134           proposal_a/proposal_b: the same scores at the PROPOSAL windowing (optional).
Reported for
135             contrast; the served regression guard still has the final say.
136           structural: forwarded to ``should_promote_for_user`` ? a structural guard (e.g.
Gate-A vs
137             held-shuttle FPs) is never *disabled* on "no lift", but it is ALSO not
auto-*promoted* to
138             the served default if it regresses served. So for ``structural=True`` we report
139             ``keep_on`` honestly but ``promote`` reflects the served-safety check.
140           served_regress_eps: served regression tolerance (0.0 = "no served-mean drop
allowed").
141
142         Returns a :class:`ServedGateResult` with the final verdict + both windowings'
evidence.
143         """
144         # 1) The honest gate, applied to the SERVED scores (never the proposal ones).
145         base = should_promote_for_user(
```

```
            served_a, served_b,
            structural=structural, min_n=min_n, p_threshold=p_threshold,
            n_boot=n_boot, seed=seed,
        )

        # 2) The eval?serve guard: does B regress A at the served windowing?
        served = served_regresses(served_a, served_b, eps=served_regress_eps,
                                  n_boot=n_boot, seed=seed)
        served_safe = not served["regresses"]

        # 3) Optional proposal-windowing view (reported, never promoting on its own).
        proposal: Optional[Dict[str, Any]] = None
        if proposal_a is not None and proposal_b is not None:
            proposal = served_regresses(proposal_a, proposal_b, eps=served_regress_eps,
                                        n_boot=n_boot, seed=seed)

        # 4) Compose the verdict. The served guard can only ever REMOVE a promotion, never
add one.
        if structural:
            # A structural guard ships ON regardless of measured lift (base.keep_on stays
True), but
            # it is only auto-promoted to the SERVED default when it is served-safe. If it
regresses
            # served, promote=False (don't bake an eval-only win into production) while
keep_on holds.
            promote = served_safe
            vetoed = False
            if served_safe:
                reason = ("structural guard kept ON; served-safe (does not regress at the
served "
                          f"windowing, ?F1 {served['mean_delta']:+.4f}) ? eligible for
served default")
            else:
                reason = ("structural guard kept ON, but NOT promoted to the served default:
it "
                          f"REGRESSES at the served windowing (?F1
{served['mean_delta']:+.4f}); "
                          "an eval-only win must not be served")
        else:
            would_promote = base.promote
            if would_promote and not served_safe:
                promote = False
                vetoed = True
                reason = ("VETOED by the served-windowing guard: the cue clears the honest
bar but "
                          f"REGRESSES at the served windowing (served ?F1
{served['mean_delta']:+.4f}"
                          f", CI [{served['ci_low']:+.4f}, {served['ci_high']:+.4f}]). A win
on a "
                          "windowing production doesn't serve cannot promote.")
            elif would_promote:
                promote = True
                vetoed = False
                reason = ("promoted: clears the honest bar AND is served-safe (served ?F1 "
                          f"{served['mean_delta']:+.4f}). " + base.reason)
            else:
                promote = False
                vetoed = False
                reason = base.reason

        return ServedGateResult(
            promote=promote,
            served_safe=served_safe,
            vetoed_by_served=vetoed,
            reason=reason,
```

```
200             base_decision=base,
201             served=served,
202             proposal=proposal,
203         )
204
```

34. assistant (2026-06-18T01:34:36.133Z)

Clean. Let me look at the reporting modules (`harvest.py`, `run_signals.py`) and `distill_local.py` which read DB rows / persisted JSON, and the `tools/export.py` and `rally_slicer.py`.

35. user (2026-06-18T01:34:37.220Z)

```
1       """Build an observability report for one indexer run.
2
3       Pure-ish harvest: given the validation results/context, the persisted segments,
4       and run metadata, populate an `obsreport.RunRecorder` with the raw signals. The
5       *footgun rules* (which turn signals into flags) live declaratively in spec.yaml,
6       so this module only records facts faithfully ? it does not decide warnings.
7
8       Imported only after `backend.reporting` confirms obsreport is installed, so it may
9       import obsreport freely.
10      """
11      from __future__ import annotations
12
13      import os
14      from pathlib import Path
15      from typing import Any, Dict, List, Optional
16
17      try:
18          import obsreport  # soft dep; only needed for RunRecorder / load_spec at runtime
19      except ImportError:
20          obsreport = None  # type: ignore[assignment]
21
22      _AI_SEGMENTERS = {"yolo_hybrid", "tracknet_hybrid", "wasb_hybrid", "gemini"}
23      _SPEC_PATH = Path(__file__).resolve().parent / "spec.yaml"
24
25
26      def load_spec() -> "obsreport.ReportSpec":
27          return obsreport.load_spec(_SPEC_PATH)
28
29
30      # --- media metadata ------------------------------------------------------
31      def video_meta_from_context(val_context: Optional[Dict[str, Any]], video_path:
Optional[str]) -> Dict[str, Any]:
32          """Derive VideoMeta fields from the validators' shared ffprobe_meta. fps_source
33          distinguishes a real probe from the absence of one (the fallback footgun)."""
34          meta: Dict[str, Any] = {"fps_source": "unknown"}
35          if video_path and os.path.exists(video_path):
36              try:
37                  meta["size_mb"] = round(os.path.getsize(video_path) / (1024 * 1024), 1)
38              except OSError:
39                  pass
40
41          ffmeta = (val_context or {}).get("ffprobe_meta") or {}
42          streams = ffmeta.get("streams") or []
43          fmt = ffmeta.get("format") or {}
44          if streams:
45              s = streams[0]
46              w, h = s.get("width"), s.get("height")
47              meta["width"], meta["height"] = w, h
48              if w and h:
49                  meta["resolution"] = f"{w}x{h}"
50              fps_str = s.get("r_frame_rate", "0/1")
```

```
            try:
                num, den = (int(x) for x in fps_str.split("/"))
                if den:
                    meta["fps"] = round(num / den, 2)
                    meta["fps_source"] = "probed"
            except (ValueError, ZeroDivisionError):
                pass
            if s.get("codec_name"):
                meta["container"] = fmt.get("format_name") or s.get("codec_name")
        if fmt.get("duration"):
            try:
                meta["duration_s"] = round(float(fmt["duration"]), 1)
            except (ValueError, TypeError):
                pass
        if fmt.get("format_name"):
            meta["container"] = fmt.get("format_name")

        # Audio readiness signal (for Input Quality UX extension per review item 2).
        # Guidelines make "record with audio on, mic unobstructed" a first-class requirement
        # (ADR-007) because shuttle thwacks are a strong rally cue for recall.
        # We only record presence here (cheap, from ffprobe already in context); the actual
        # thwack onset probe (pipeline/audio/hit_detector) remains a diagnostic /
calibration
        # tool for now.
        audio_streams = [s for s in streams if (s or {}).get("codec_type") == "audio"]
        meta["audio_present"] = len(audio_streams) > 0
        return meta


    # --- stage builders -----------------------------------------------------
    def _validation_stage(rec: "obsreport.RunRecorder", val_results: List[Any]) -> None:
        with rec.stage("validation") as st:
            total = len(val_results)
            passed = sum(1 for r in val_results if getattr(r, "passed", False))
            st.add_metric("checks_passed", passed)
            st.add_metric("checks_total", total)
            failures = [r for r in val_results if not getattr(r, "passed", True)]
            for r in failures:
                st.messages.append(f"FAILED {getattr(r, 'validator_name', '?')}: {getattr(r,
'message', '')}")
            st.status = "error" if failures else ("ok" if total else "not_run")


    def _segmentation_stage(
        rec: "obsreport.RunRecorder",
        play_segments: List[Dict[str, Any]],
        candidates: List[Dict[str, Any]],
        segmenter_requested: Optional[str],
        segmenter_actual: Optional[str],
        config_default: Optional[str],
        was_reingest: bool,
        ran: bool,
    ) -> None:
        with rec.stage("segmentation") as st:
            if not ran:
                st.status = "not_run"
            seg_count = len(play_segments)
            total_play = round(sum(float(s.get("duration", 0.0)) for s in play_segments), 1)
            detector_tag = None
            if play_segments:
                detector_tag = (play_segments[0].get("metadata") or {}).get("detector")
            st.add_metric("candidate_windows", len(candidates))
            st.add_metric("segment_count", seg_count, audience="both")
            st.add_metric("total_play_s", total_play, unit="s")
            # "diverges" only when a segmenter actually RAN, the user did NOT explicitly
```

```
            # pick it, and it differs from config's default (the genuine surprise case).
            # An explicit --segmenter override or a halted run is not a divergence.
            matches_default = True
            if ran and segmenter_actual and config_default:
                matches_default = (segmenter_actual == config_default)
            st.details.update({
                "segmenter_requested": segmenter_requested,
                "segmenter_actual": segmenter_actual,
                "config_default": config_default,
                "segmenter_explicitly_requested": segmenter_requested is not None,
                "matches_config_default": matches_default,
                "was_reingest": was_reingest,
                "detector": detector_tag,
            })
            # Surface metadata trustworthiness at the data level (not just in a warning):
            # the motion baseline fabricates per-segment metadata; everything else measures
it.
            if ran and segmenter_actual:
                st.details["metadata_source"] = (
                    "synthetic/heuristic ? NOT measured" if segmenter_actual == "motion"
                    else "measured (detector/AI)"
                )


    def _ai_stage(rec: "obsreport.RunRecorder", config: Dict[str, Any], segmenter_actual:
Optional[str],
                  play_segments: List[Dict[str, Any]], ran: bool,
                  run_signals: Optional[Dict[str, Any]] = None) -> None:
        """Only meaningful for AI-based segmenters. Records whether the provider key was
        present ? the signal the silent_provider_noop rule keys on ? plus any in-process
        run signals (backend.reporting.run_signals), e.g. oversize chunk refusals."""
        ai_based = segmenter_actual in _AI_SEGMENTERS
        provider = (config.get("ai_provider") or "gemini")
        key_env = f"{provider.upper()}_API_KEY"
        with rec.stage("ai_handoff") as st:
            if not ran or not ai_based:
                st.status = "not_run" if not ran else "skipped"
            st.details.update({
                "ai_based": ai_based,
                "provider": provider if ai_based else None,
                "model": config.get("ai_model") if ai_based else None,
                "api_key_present": bool(os.environ.get(key_env)) if ai_based else None,
                # Chunks the provider refused as over its upload cap (no analysis ran for
                # those time ranges). Detail name couples to the oversize_clip_skipped rule.
                "oversize_clips_skipped": int((run_signals or
{}).get("oversize_clips_skipped", 0)) if ai_based else None,
            })
            if ai_based:
                st.add_metric("confirmed_segments", len(play_segments))


    def _highlights(rec: "obsreport.RunRecorder", play_segments: List[Dict[str, Any]],
                    video_meta: Dict[str, Any]) -> None:
        total = round(sum(float(s.get("duration", 0.0)) for s in play_segments), 1)
        coverage = None
        dur = video_meta.get("duration_s")
        if dur:
            coverage = round(100.0 * total / dur, 1)
        notes = []
        if not play_segments:
            notes.append("No highlights were produced for this video.")
        rec.set_highlights(segment_count=len(play_segments), total_highlight_s=total,
                           coverage_pct=coverage, notes=notes)


```

```
176     # --- top-level ----------------------------------------------------------
177     def record_run(
178         *,
179         video_id: str,
180         video_path: Optional[str],
181         config: Dict[str, Any],
182         val_results: List[Any],
183         val_context: Optional[Dict[str, Any]],
184         play_segments: List[Dict[str, Any]],
185         candidates: List[Dict[str, Any]],
186         segmenter_requested: Optional[str] = None,
187         segmenter_actual: Optional[str] = None,
188         was_reingest: bool = False,
189         segmentation_ran: bool = True,
190         tool_version: Optional[str] = None,
191         spec: Optional["obsreport.ReportSpec"] = None,
192         run_signals: Optional[Dict[str, Any]] = None,
193         **recorder_kwargs: Any,
194     ) -> "obsreport.RunRecorder":
195         """Assemble a finalized recorder for an indexer run (does NOT write)."""
196         rec = obsreport.RunRecorder(
197             "badminton-highlight-indexer",
198             tool_version=tool_version,
199             video_id=video_id,
200             video_id_kind="file_md5",
201             video_path=video_path,
202             spec=spec if spec is not None else load_spec(),
203             **recorder_kwargs,
204         )
205         vm = video_meta_from_context(val_context, video_path)
206         # If we never probed an fps but segmentation still ran, the segmenter must have
207         # used its internal fallback (30fps / 1280px) ? surface that as the footgun it is,
208         # AND show the assumed numeric value so the report reads "30 (fallback)" not "?
(fallback)".
209         if vm.get("fps_source") == "unknown" and segmentation_ran:
210             vm["fps_source"] = "fallback"
211             if vm.get("fps") is None:
212                 vm["fps"] = 30.0  # matches the runners' fallback (_probe_fps_width -> 30.0
/ 1280)
213         rec.set_video_meta(**vm)
214
215         _validation_stage(rec, val_results)
216         config_default = (config.get("indexing") or {}).get("default_segmenter")
217         _segmentation_stage(rec, play_segments, candidates, segmenter_requested,
218                             segmenter_actual, config_default, was_reingest,
segmentation_ran)
219         _ai_stage(rec, config, segmenter_actual, play_segments, segmentation_ran,
run_signals)
220         _highlights(rec, play_segments, vm)
221
222         rec.finalize()
223         return rec
224
```

36. assistant (2026-06-18T01:34:43.092Z)

Notice line 47: `meta["width"], meta["height"] = w, h` where `s = streams[0]`. `streams` comes from ffprobe JSON. If `streams[0]` is `None` (which would only happen with corrupt ffprobe output), `s.get(...)` would fail ? but the `if streams:` guard plus the audio comprehension uses `(s or {})`. The `s.get()` here is not protected by `(s or {})`. But ffprobe always returns dict streams, so this is theoretical. Not a strong finding.

This `harvest.py` handles ffprobe data carefully. Let me look at the remaining unexamined readers: `distill_local.py`, `run_signals.py`, `tools/export.py`, `rally_slicer.py`, `batch.py`, and `per_user_gate.py`/`per_user_eval.py`. Let me also check the segmenters' hybrid pipelines

which read config. Let me batch read the higher-probability ones.

37. user (2026-06-18T01:34:44.520Z)

```
"""In-process signal channel: pipeline internals -> the per-video report.

Some report-worthy facts (e.g. the provider refusing an oversize chunk) surface
deep inside a segmenter's worker threads, with no return path to the harvest ?
segmenters return only their segment list. Layers record those facts here keyed
by video_id; `emit_indexer_report` pops them when the run's report is built.

Process-local and best-effort BY DESIGN: recording must never break a run, the
store is consumed (popped) once per report, and signals for a video are simply
dropped if no report is emitted. No obsreport dependency ? importable everywhere.
"""
from __future__ import annotations

import threading
from typing import Any, Dict

_lock = threading.Lock()
_signals: Dict[str, Dict[str, Any]] = {}


def incr(video_id: str, key: str, by: int = 1) -> None:
    """Add `by` to the counter `key` for this video's in-flight run."""
    if not video_id or by == 0:
        return
    with _lock:
        sig = _signals.setdefault(video_id, {})
        sig[key] = int(sig.get(key, 0)) + by


def pop(video_id: str) -> Dict[str, Any]:
    """Consume (remove and return) all signals recorded for this video's run."""
    with _lock:
        return _signals.pop(video_id, {})

```

38. user (2026-06-18T01:34:45.057Z)

```
"""Minimal Value Layer exports (first slice of review item 3).

Provides timeline exports (EDL, JSON, CSV) and basic stats over the
play_segments returned by the indexer (or query_play_segments). These are
the "time-saved editing is table stakes" deliverables and also give a
machine-readable foundation for later human correction / golden feeding
and clip galleries.

Kept deliberately small and pure (no DB, no side effects) so they are
easy to call from CLI, notebooks, or a future API endpoint.
"""

from __future__ import annotations

import json
from typing import Any, Dict, Iterable, List, Optional


def _to_list(segments: Iterable[Dict[str, Any]]) -> List[Dict[str, Any]]:
    return list(segments) if not isinstance(segments, list) else segments


def _segment_metadata(s: Dict[str, Any]) -> Dict[str, Any]:
    """A play segment's metadata as a dict, whether stored inline (segmenter output)
```

```
        or as a JSON string (DB round-trip via query_play_segments)."""
        md = s.get("metadata") or {}
        if isinstance(md, str):
            try:
                md = json.loads(md)
            except Exception:
                md = {}
        return md if isinstance(md, dict) else {}


    def _fmt_hms(seconds: float) -> str:
        """HH:MM:SS, matching the segmenters' _fmt_ts so the summary block lines up with
        the per-rally [rally] log lines and the existing timestamp style."""
        total = max(0, int(float(seconds)))
        h, rem = divmod(total, 3600)
        m, s = divmod(rem, 60)
        return f"{h:02d}:{m:02d}:{s:02d}"


    def format_rally_summary(
        segments: Iterable[Dict[str, Any]], title: str = "Rally summary"
    ) -> str:
        """Render a compact, scannable end-of-run block for manual QA against the rally CSV.

        One header line (total rallies + total highlight duration in HH:MM:SS) followed by a
        per-rally table ``# start end dur(s) shot_count`` in time order. Pure (no DB, no
I/O):
        the same play_segment dicts that feed ``segments_to_csv`` / the rally CSV go in and
        text comes out, so the log and CSV line up row-for-row for eyeball verification.

        A segment with no ``shot_count`` in its metadata renders the count as ``-``.
        """
        segs = _to_list(segments)
        total_dur = sum(
            float(s.get("duration", float(s.get("end_time", 0)) - float(s.get("start_time",
0))))
            for s in segs
        )
        header = (
            f"=== {title}: {len(segs)} rallies, "
            f"{_fmt_hms(total_dur)} ({total_dur:.1f}s) total play ==="
        )
        if not segs:
            return header + "\n  (no rallies detected)"

        lines = [header, f"{'#':>3}  {'start':<8}  {'end':<8}  {'dur(s)':>7}  {'shots':>6}"]
        for i, s in enumerate(segs, start=1):
            start = float(s.get("start_time", 0.0))
            end = float(s.get("end_time", start))
            dur = float(s.get("duration", end - start))
            shot = _segment_metadata(s).get("shot_count")
            shot_str = "-" if shot is None else str(shot)
            lines.append(
                f"{i:>3}  {_fmt_hms(start):<8}  {_fmt_hms(end):<8}  {dur:>7.1f}
{shot_str:>6}"
            )
        return "\n".join(lines)


    def basic_stats(segments: Iterable[Dict[str, Any]]) -> Dict[str, Any]:
        """Return simple aggregate stats for a list of play segments.

        Keys: count, total_duration_s, avg_duration_s, min_duration_s, max_duration_s.
        All durations are in seconds (float).
        """
```

```
87          segs = _to_list(segments)
88          if not segs:
89              return {
90                  "count": 0,
91                  "total_duration_s": 0.0,
92                  "avg_duration_s": 0.0,
93                  "min_duration_s": 0.0,
94                  "max_duration_s": 0.0,
95              }
96          durs = [float(s.get("duration", (s.get("end_time", 0) - s.get("start_time", 0))))
for s in segs]
97          total = sum(durs)
98          return {
99              "count": len(segs),
100             "total_duration_s": round(total, 2),
101             "avg_duration_s": round(total / len(durs), 2),
102             "min_duration_s": round(min(durs), 2),
103             "max_duration_s": round(max(durs), 2),
104         }
105
106
107     def segments_to_json(
108         segments: Iterable[Dict[str, Any]],
109         video_id: str,
110         video_path: Optional[str] = None,
111         meta: Optional[Dict[str, Any]] = None,
112     ) -> Dict[str, Any]:
113         """Return a JSON-serializable timeline dict.
114
115         Shape (stable):
116         {
117           "video_id": "...",
118           "video_path": "...",
119           "meta": {...},
120           "segments": [ {"id":, "start_time":, "end_time":, "duration":, "metadata": {...}},
... ],
121           "stats": basic_stats(...)
122         }
123         """
124         segs = _to_list(segments)
125         return {
126             "video_id": video_id,
127             "video_path": video_path,
128             "meta": meta or {},
129             "segments": segs,
130             "stats": basic_stats(segs),
131         }
132
133
134     def segments_to_csv(segments: Iterable[Dict[str, Any]], include_header: bool = True) ->
str:
135         """Return a CSV string (start,end,duration,shot_count,detector,...) .
136
137         Compatible with many editors and the existing golden/slicer schemas
138         (extra columns are harmless).
139         """
140         segs = _to_list(segments)
141         lines: List[str] = []
142         if include_header:
143
lines.append("start_time,end_time,duration,shot_count,shot_count_confidence,detector,id")
144         for s in segs:
145             md = s.get("metadata") or {}
146             shot = md.get("shot_count", "")
147             conf = md.get("shot_count_confidence", "")
```

```
148             det = md.get("detector", "")
149             lines.append(
150                 f"{s.get('start_time',0)},{s.get('end_time',0)},{s.get('duration',0)},"
151                 f"{shot},{conf},{det},{s.get('id','')}"
152             )
153         return "\n".join(lines) + "\n"
154 
155 
156     def segments_to_edl(segments: Iterable[Dict[str, Any]], title: str = "Highlights") ->
str:
157         """Return a simple EDL (CMX 3600-ish) for the segments.
158 
159         Many NLEs will ingest this for a rough cut. Uses record timecode 00:00:00:00
160         as origin; clips are placed sequentially with the original source in/out.
161         """
162         segs = _to_list(segments)
163         lines: List[str] = [f"TITLE: {title}", "FCM: NON-DROP FRAME"]
164         for i, s in enumerate(segs, 1):
165             start = float(s.get("start_time", 0.0))
166             end = float(s.get("end_time", start))
167             # EDL is HH:MM:SS:FF (assume 30fps for the timecode display; real edit rate
168             # is irrelevant for a rough cut ? the numbers are just labels here).
169             def _tc(t: float) -> str:
170                 h = int(t // 3600)
171                 m = int((t % 3600) // 60)
172                 s_ = int(t % 60)
173                 f = int((t - int(t)) * 30)  # 30 fps display
174                 return f"{h:02d}:{m:02d}:{s_:02d}:{f:02d}"
175             rec_in = _tc((i - 1) * 0.0)  # place back-to-back in the EDL timeline
176             rec_out = _tc((i - 1) * 0.0 + (end - start))
177             src_in = _tc(start)
178             src_out = _tc(end)
179             lines.append(f"{i:03d}  AX  V     C        {rec_in} {rec_out} {src_in}
{src_out}")
180             lines.append(f"* FROM CLIP NAME: {title} {i}")
181             lines.append("")
182         return "\n".join(lines)
183 
```

39. assistant (2026-06-18T01:34:48.668Z)

Both robust with `.get()` defaults. Let me check the segmenter hybrid pipelines, `distill_local.py`, the `batch.py`, and `per_user_gate.py`. Let me also check `annotations/file_provider.py` and `automated_provider.py` which read annotation files. Let me batch-read.

40. user (2026-06-18T01:34:49.651Z)

```
1       """FileProvider ? ground-truth rally labels from pre-made CSV files.
2 
3       This formalizes today's manual / ShuttleSet annotation flow behind the
4       `AnnotationProvider` interface: the "annotation job" is simply a CSV that already
5       exists on disk (the same `start,end` format `run_eval.py --scaffold` writes). The
6       job_id IS the resolved CSV path, so submit/poll/fetch all operate on that path.
7 
8       It's the synchronous, zero-cost provider used as the default golden-set source and
9       as the stand-in in tests/CI before a human-vendor or automated provider is wired in.
10      """
11      import os
12      from typing import List, Optional
13 
14      from backend.annotations.base import (
15          AnnotationProvider,
16          annotation_registry,
17          Interval,
```

```
        DONE,
        FAILED,
    )
    from backend.eval.annotations import load_annotations


    class FileProvider(AnnotationProvider):
        def __init__(self, csv_dir: Optional[str] = None, ext: str = ".csv"):
            """
            Args:
                csv_dir: if set, a bare `video_ref` (not itself a path) is resolved to
                         `<csv_dir>/<video_ref><ext>` ? handy for keying by video_id/MD5.
                ext: file extension used for that directory-based resolution.
            """
            self.csv_dir = csv_dir
            self.ext = ext

        @property
        def name(self) -> str:
            return "file"

        def _resolve(self, video_ref: str) -> str:
            """Map a video_ref to a CSV path.

            An existing path is used as-is; otherwise it's treated as a video_id and
            looked up under csv_dir (if configured).
            """
            if os.path.isfile(video_ref):
                return video_ref
            if self.csv_dir:
                return os.path.join(self.csv_dir, f"{video_ref}{self.ext}")
            return video_ref

        def submit(self, video_ref: str, guideline: Optional[str] = None,
                   sport: str = "badminton") -> str:
            # Nothing to do ? the label file is expected to already exist. The job_id
            # is the resolved CSV path. (guideline/sport are irrelevant for a file.)
            return self._resolve(video_ref)

        def poll(self, job_id: str) -> str:
            return DONE if os.path.isfile(job_id) else FAILED

        def fetch(self, job_id: str) -> List[Interval]:
            if not os.path.isfile(job_id):
                raise FileNotFoundError(f"annotation file not found: {job_id}")
            return load_annotations(job_id)


    annotation_registry.register(FileProvider)

```

41. user (2026-06-18T01:34:50.259Z)

```
    """AutomatedProvider ? the fast-tier "oracle" annotation source (GS-4, interface stub).

    The fast/CI tier of the golden-set flow needs an *automated* labeler that turns a
    video into rally `(start, end)` intervals quickly and cheaply, as a smoke alarm
    between authoritative human-vendor runs (the slow tier).

    This slice ships the **interface only**, deliberately model-agnostic: an
    `AutomatedProvider` that delegates labeling to an injected `labeler` callable, plus
    a deterministic fake for tests/CI. No concrete model is wired yet ? that's a
    tracked backlog item (docs/BACKLOG.md), to be chosen alongside the production LLM
    step so the lineages differ (see the oracle rule below).

```

```
    ?? **Oracle rule.** The labeler MUST be a *different model lineage* than whatever
    production uses. If production becomes the offline ActionFormer segmenter and the
    oracle is also ActionFormer, they share blind spots and the regression test is
    blind ? green while both are wrong. `lineage` records the oracle's model family so
    callers can guard against it matching production (`warn_if_same_lineage`).
    """
    import logging
    from typing import Callable, Dict, List, Optional

    from backend.annotations.base import (
        AnnotationProvider,
        annotation_registry,
        Interval,
        DONE,
        FAILED,
    )

    logger = logging.getLogger(__name__)

    # A labeler turns (video_ref, guideline, sport) into rally intervals.
    Labeler = Callable[[str, Optional[str], str], List[Interval]]


    def _not_configured(video_ref: str, guideline: Optional[str], sport: str) ->
List[Interval]:
        """Default labeler: refuses to run until a real model is injected.

        Keeps `AutomatedProvider()` constructible (so the registry can key it as 'auto')
        while making accidental use of an unconfigured oracle fail loudly rather than
        silently returning no rallies.
        """
        raise NotImplementedError(
            "AutomatedProvider has no labeler configured. Inject one via "
            "annotation_registry.get('auto', labeler=<fn>, lineage='<model-family>'). "
            "Choosing the concrete oracle model is a tracked item ? see docs/BACKLOG.md
(GS-4)."
        )


    def warn_if_same_lineage(oracle_lineage: Optional[str], production_lineage:
Optional[str]) -> bool:
        """Warn (and return True) if the oracle shares production's model lineage.

        Encodes the oracle rule in code, not just docs: a same-lineage oracle can't be
        trusted to catch production's failures. Callers wiring CI should pass both.
        """
        if oracle_lineage and production_lineage and oracle_lineage == production_lineage:
            logger.warning(
                "Oracle lineage '%s' matches production's ? the regression test may be "
                "blind to shared failure modes. Use a different model lineage.",
                oracle_lineage,
            )
            return True
        return False


    class AutomatedProvider(AnnotationProvider):
        def __init__(self, labeler: Optional[Labeler] = None, lineage: str = "unspecified"):
            """
            Args:
                labeler: callable producing rally intervals for a video. Defaults to a
                         stub that raises until configured.
                lineage: the oracle's model family (e.g. 'gemini', 'qwen-vl') ? recorded
                         so callers can enforce the oracle rule via warn_if_same_lineage.
            """
```

```
75              self._labeler = labeler or _not_configured
76              self.lineage = lineage
77              # In-process synchronous job store: job_id -> labeled intervals.
78              self._jobs: Dict[str, List[Interval]] = {}
79
80          @property
81          def name(self) -> str:
82              return "auto"
83
84          def submit(self, video_ref: str, guideline: Optional[str] = None,
85                     sport: str = "badminton") -> str:
86              # Synchronous stub: run the labeler now and stash the result. A real async
87              # oracle (e.g. a queued API job) would kick off here and resolve in poll().
88              intervals = self._labeler(video_ref, guideline, sport)
89              intervals = sorted((float(s), float(e)) for s, e in intervals)
90              job_id = video_ref
91              self._jobs[job_id] = intervals
92              return job_id
93
94          def poll(self, job_id: str) -> str:
95              return DONE if job_id in self._jobs else FAILED
96
97          def fetch(self, job_id: str) -> List[Interval]:
98              if job_id not in self._jobs:
99                  raise KeyError(f"AutomatedProvider: unknown job '{job_id}' (submit it
first)")
100             return self._jobs[job_id]
101
102
103     annotation_registry.register(AutomatedProvider)
104
```

42. assistant (2026-06-18T01:34:55.149Z)

Clean. Let me check `distill_local.py`, `batch.py`, `per_user_gate.py`, `per_user_eval.py`, and the segmenter hybrids. The hybrids and runners read config and CSV trajectories. Let me look at the WASB/tracknet runners and fusion_hybrid which read persisted candidate stats and config heavily.

43. user (2026-06-18T01:34:56.089Z)

```
1       """Batch evaluation across a collector export manifest.
2
3       Phase 3 of the collector->indexer bridge: walk the `manifest.json` emitted by
4       sports-data-collector's `curate export`, (optionally) process each video through
5       a segmenter so detections land in the indexer DB, then score every video's
6       detected rallies against its ground-truth CSV and aggregate the results.
7
8       This module holds the *pure* pieces (path resolution, stage selection, metric
9       aggregation) so they can be unit-tested without touching video files, the GPU,
10      or any API. The orchestration/side-effects live in `scripts/run_phase3_eval.py`.
11      """
12
13      import os
14      from typing import Dict, List, Optional
15
16
17      # Segmenters that only ever populate the `final` play_segments table (no raw
18      # candidate stage), so scoring `candidates` for them would be meaninglessly empty.
19      _FINAL_ONLY_SEGMENTERS = {"motion", "gemini"}
20
21
22      def resolve_video_path(local_path: Optional[str], videos_root: str) -> Optional[str]:
23          """Resolve a manifest `local_path` to an absolute path.
24
```

```
        The collector stores paths like ``./data/videos\\shuttleset_1.mp4`` relative
        to its own repo root and with Windows separators. Normalise separators, strip
        a leading ``./``, and resolve relative paths against `videos_root` (the
        collector root). Returns None for a missing/empty path.
        """
        if not local_path:
            return None
        p = local_path.replace("\\", "/")
        if p.startswith("./"):
            p = p[2:]
        # Cross-platform: treat Windows drive-letter absolute paths (C:/...) as absolute
        # even when running on Linux (os.path.isabs("C:/...") == False on Unix).
        # This keeps collector manifest paths working in tests and mixed environments.
        if (len(p) > 1 and p[1] == ":" and p[0].isalpha()) or os.path.isabs(p):
            return os.path.normpath(p)
        return os.path.normpath(os.path.join(videos_root, p))


    def default_stages_for(segmenter: str, requested: str = "auto") -> List[str]:
        """Pick which prediction stages to score.

        ``auto`` scores both stages for two-stage detectors (yolo_hybrid/tracknet_*)
        but only ``final`` for segmenters that don't emit candidates. An explicit
        request ('candidates' | 'final' | 'both') overrides the heuristic.
        """
        if requested == "both":
            return ["candidates", "final"]
        if requested in ("candidates", "final"):
            return [requested]
        # auto
        if segmenter in _FINAL_ONLY_SEGMENTERS:
            return ["final"]
        return ["candidates", "final"]


    def _safe_div(num: float, den: float) -> float:
        return num / den if den else 0.0


    def aggregate(per_video: List[dict], stages: List[str]) -> Dict[str, dict]:
        """Aggregate per-video stage scores into corpus-level metrics.

        `per_video` is a list of dicts shaped like ``report_to_dict`` output (each has
        ``stages[stage]["score"]`` with tp/fp/fn/mean_iou/...). For each stage we
        compute **micro** precision/recall/F1 (pooling TP/FP/FN across all videos) and
        a TP-weighted mean IoU, plus the macro mean of per-video F1.
        """
        out: Dict[str, dict] = {}
        for stage in stages:
            tp = fp = fn = 0
            iou_weighted = 0.0
            f1s: List[float] = []
            n_videos = 0
            for rep in per_video:
                sr = rep.get("stages", {}).get(stage)
                if not sr:
                    continue
                s = sr["score"]
                tp += s["true_positives"]
                fp += s["false_positives"]
                fn += s["false_negatives"]
                iou_weighted += s["mean_iou"] * s["true_positives"]
                f1s.append(s["f1"])
                n_videos += 1
            precision = _safe_div(tp, tp + fp)
```

```
            recall = _safe_div(tp, tp + fn)
            out[stage] = {
                "videos": n_videos,
                "tp": tp, "fp": fp, "fn": fn,
                "micro_precision": precision,
                "micro_recall": recall,
                "micro_f1": _safe_div(2 * precision * recall, precision + recall),
                "macro_f1": _safe_div(sum(f1s), len(f1s)),
                "mean_iou": _safe_div(iou_weighted, tp),
            }
        return out


    def format_aggregate(agg: Dict[str, dict]) -> str:
        """Human-readable corpus-level summary table."""
        lines = [
            "=" * 78,
            " CORPUS AGGREGATE",
            "=" * 78,
            f" {'Stage':<12} {'Vids':>4} {'TP':>5} {'FP':>5} {'FN':>5} "
            f"{'mPrec':>6} {'mRec':>6} {'microF1':>8} {'macroF1':>8} {'mIoU':>6}",
            "-" * 78,
        ]
        for stage, a in agg.items():
            lines.append(
                f" {stage:<12} {a['videos']:>4} {a['tp']:>5} {a['fp']:>5} {a['fn']:>5} "
                f"{a['micro_precision']:>6.2f} {a['micro_recall']:>6.2f} "
                f"{a['micro_f1']:>8.2f} {a['macro_f1']:>8.2f} {a['mean_iou']:>6.2f}"
            )
        lines.append("=" * 78)
        return "\n".join(lines)

```

44. assistant (2026-06-18T01:35:02.755Z)

At line 82-86, `aggregate` reads `sr["score"]` then `s["true_positives"]`, `s["false_positives"]`, `s["false_negatives"]`, `s["mean_iou"]`, `s["f1"]`. The `sr = rep.get("stages", {}).get(stage)` is guarded, but once `sr` is truthy, `sr["score"]` and the `s[...]` accesses are unguarded. `per_video` is described as "report_to_dict output" ? internally produced, so the shape is controlled. The docstring says it's shaped like `report_to_dict` output. This is an internal contract, not external data. Lower concern.

Let me look at the segmenter hybrids and runners which parse trajectory CSVs and persisted candidate stats ? `wasb_hybrid.py`, `fusion_hybrid.py`, `tracknet_runner.py`, `wasb_runner.py`. These read persisted detection data. Let me also check `distill_local.py`.

45. user (2026-06-18T01:35:04.989Z)

```
"""TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).

This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:

    1. Translate the Windows video path to its WSL-visible form (and optionally
       stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
    2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
    3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
    4. Convert that trajectory into high-motion "action windows" ? the same
       candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
       pipeline (AI handoff, DB population) is unchanged.

Nothing here imports TensorFlow; all model code stays in WSL.
"""

import os
```

```
    import csv
    import shlex
    import logging
    import subprocess
    from dataclasses import dataclass
    from typing import List, Dict, Any, Optional, Tuple

    from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote

    logger = logging.getLogger(__name__)


    @dataclass
    class TrackNetConfig:
        """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
        distro: str = "Ubuntu"
        repo_dir: str = "~/models/TrackNetV4"          # WSL path to the cloned repo
        conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
        conda_env: str = "TrackNetV4"
        weights_path: str = ""                          # WSL path to the .h5/.keras weights
(REQUIRED)
        queue_length: int = 5
        stage_in_wsl: bool = True                       # copy video into WSL fs first
(perf)
        wsl_stage_dir: str = "~/clips"                  # where staged videos/outputs live
in WSL
        timeout_sec: int = 1800                         # 30 min; kills a hung call (0 = no
timeout)
        keep_frames: bool = False                       # retain staged video after success
(debug only)
        min_free_gb: float = 0.0                        # warn if WSL free space below this
(0 = off)

        @classmethod
        def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
            tn = (idx_cfg or {}).get("tracknet", {}) or {}
            return cls(
                distro=tn.get("wsl_distro", "Ubuntu"),
                repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
                conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
                conda_env=tn.get("conda_env", "TrackNetV4"),
                weights_path=tn.get("weights_path", ""),
                queue_length=int(tn.get("queue_length", 5)),
                stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
                wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
                timeout_sec=int(tn.get("timeout_sec", 1800)),
                keep_frames=bool(tn.get("keep_frames", False)),
                min_free_gb=float(tn.get("min_free_gb", 0.0)),
            )


    def to_wsl_mnt_path(win_path: str) -> str:
        """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).

        Already-POSIX paths (starting with / or ~) are returned unchanged so the
        same helper is safe to call on either kind of path.
        """
        p = str(win_path)
        if p.startswith("/") or p.startswith("~"):
            return p
        p = p.replace("\\", "/")
        if len(p) >= 2 and p[1] == ":":
            drive = p[0].lower()
```

```
            return f"/mnt/{drive}{p[2:]}"
        return p


    @dataclass
    class TrajectoryPoint:
        frame: int
        visible: bool
        x: float
        y: float


    class TrackNetRunner(WslCommandMixin, DetectorRunner):
        """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.

        Implements the common DetectorRunner contract so a future Linux-native or
        alternative trajectory runner can be substituted without touching the hybrids.
        """

        def __init__(self, cfg: TrackNetConfig):
            self.cfg = cfg

        def healthcheck(self) -> Tuple[bool, str]:
            """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.

            Returns (ok, message). Cheap to call before a long run.
            """
            cmd = (
                f"conda activate {self.cfg.conda_env} && "
                f"python -c \"import tensorflow as tf; "
                f"print('TF', tf.__version__, 'GPUs',
len(tf.config.list_physical_devices('GPU')))\""
            )
            try:
                res = self._wsl_bash(cmd)
            except FileNotFoundError:
                return False, "`wsl` not found ? is this running on Windows with WSL2
installed?"
            except subprocess.TimeoutExpired:
                return False, "WSL healthcheck timed out."
            out = (res.stdout or "").strip()
            if res.returncode != 0:
                return False, f"WSL env check failed: {(res.stderr or out).strip()}"
            return True, out

        # ------------------------------------------------------------------ inference

        def run_predict(self, video_win_path: str, output_win_dir: str,
                        video_id: Optional[str] = None) -> Optional[str]:
            """Run predict.py on a video. Returns the Windows path to the produced CSV, or
None.

            ``output_win_dir`` is a Windows directory (under the repo's output/) that
            WSL writes into via its /mnt view, so the Windows process can read the CSV
            back with no copy. ``video_id`` (file MD5) namespaces the staged video ? and
            therefore predict.py's ``<stem>_predict.csv`` output ? so two videos that share
            a filename cannot collide on the output CSV.
            """
            if not self.cfg.weights_path:
                logger.error(
                    "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
                    "in config.json to the WSL path of your trained TrackNetV4 weights."
                )
                return None

```

```
            # Resolve relative dirs BEFORE the /mnt translation ? to_wsl_mnt_path passes
            # relative paths through unchanged, so WSL would resolve them against the
            # predict.py cwd instead of the repo (same bug class as wasb_runner).
            output_win_dir = os.path.abspath(output_win_dir)
            os.makedirs(output_win_dir, exist_ok=True)
            # Normalize separators so basename/splitext work when tests (or collector)
            # pass Windows-style paths on a Linux host.
            video_win_path = str(video_win_path).replace("\\", "/")
            base = os.path.basename(video_win_path)
            stem, ext = os.path.splitext(base)

            # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
            if ext.lower() not in (".mp4", ".avi"):
                logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
                return None

            wsl_out = to_wsl_mnt_path(output_win_dir)
            # Namespace by video_id so two same-filename videos don't collide on the CSV.
            # predict.py derives its output name from the (staged) video stem, so staging
            # under the namespaced name propagates into "<key>_predict.csv".
            key = f"{stem}__{video_id[:12]}" if video_id else stem
            expected_csv_win = os.path.join(output_win_dir, f"{key}_predict.csv")

            # Resolve the video path WSL will read.
            if self.cfg.stage_in_wsl:
                staged = self._stage_video(video_win_path, f"{key}{ext}")
                if staged is None:
                    return None
                wsl_video = staged
            else:
                # No staging: predict.py reads /mnt directly and writes
"<stem>_predict.csv".
                # We cannot rename its output, so fall back to the un-namespaced name.
                wsl_video = to_wsl_mnt_path(video_win_path)
                expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")

            if self.cfg.min_free_gb > 0:
                free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
                if free is not None and free < self.cfg.min_free_gb:
                    logger.warning("Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f); "
                                   "consider running tools/cleanup_caches.py.",
                                   free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)

            cmd = (
                f"conda activate {self.cfg.conda_env} && "
                f"cd {self.cfg.repo_dir}/src && "
                f"python predict.py "
                f"--video_path {wsl_tilde_quote(wsl_video)} "
                f"--model_weights {wsl_tilde_quote(self.cfg.weights_path)} "
                f"--output_dir {wsl_tilde_quote(wsl_out)} "
                f"--queue_length {self.cfg.queue_length}"
            )
            logger.info("Running TrackNet inference on %s ...", base)
            try:
                res = self._wsl_bash(cmd)
            except subprocess.TimeoutExpired:
                logger.error("TrackNet inference timed out after %ss.",
self.cfg.timeout_sec)
                return None

            if res.returncode != 0:
                logger.error("TrackNet predict.py failed:\n%s", (res.stderr or
res.stdout)[-2000:])
                return None

```

```
            if not os.path.exists(expected_csv_win):
                logger.error("TrackNet finished but expected CSV not found at %s",
expected_csv_win)
                return None
            logger.info("TrackNet trajectory CSV: %s", expected_csv_win)

            # Success ? the CSV is the durable output; drop the staged video copy (a pure,
            # regenerable intermediate). On earlier failure we return above without
cleaning.
            if self.cfg.stage_in_wsl and not self.cfg.keep_frames:
                logger.info("Cache hygiene: removing staged video for %s "
                            "(set indexing.tracknet.keep_frames=true to retain).", key)
                self._wsl_rm_rf([wsl_video])
            return expected_csv_win

        def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
            """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).

            Returns the WSL path to the staged copy, or None on failure.
            """
            src_mnt = to_wsl_mnt_path(video_win_path)
            reason = self.wsl_source_unreadable_reason(src_mnt)
            if reason:
                logger.error("Cannot stage video into WSL: %s", reason)
                return None
            dest = f"{self.cfg.wsl_stage_dir}/{base}"
            cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
            try:
                res = self._wsl_bash(cmd)
            except subprocess.TimeoutExpired:
                logger.error("Staging video into WSL timed out.")
                return None
            if res.returncode != 0 or "OK" not in (res.stdout or ""):
                logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
                return None
            return dest

        # ------------------------------------------------------------- CSV -> windows

        @staticmethod
        def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
            """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
            points: List[TrajectoryPoint] = []
            with open(csv_win_path, newline="") as f:
                reader = csv.DictReader(f)
                for row in reader:
                    try:
                        points.append(TrajectoryPoint(
                            frame=int(row["Frame"]),
                            visible=int(row["Visibility"]) == 1,
                            x=float(row["X"]),
                            y=float(row["Y"]),
                        ))
                    except (KeyError, ValueError):
                        continue
            points.sort(key=lambda p: p.frame)
            return points

        @staticmethod
        def trajectory_to_action_windows(
            points: List[TrajectoryPoint],
            fps: float,
            frame_width: float,
```

```
            chunk_start: float = 0.0,
            velocity_thresh: float = 0.02,
            merge_gap: float = 2.0,
            min_window_duration: float = 2.0,
            chunk_index: Optional[int] = None,
            max_window_duration: float = 0.0,
            min_split_gap: float = 0.5,
            window_hard_cap: bool = False,
            window_inactivity_end: float = 0.0,
            inactivity_min_density: float = 0.34,
            inactivity_rally_thresh: float = 0.08,
            window_blob_fallback: bool = False,
            window_blob_factor: float = 4.0,
        ) -> List[Dict[str, Any]]:
            """Convert a shuttle trajectory into candidate rally windows.

            A frame is "active" when the shuttle is visible AND its inter-frame
            displacement (normalised by frame width, per second) exceeds
            ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
            ``merge_gap`` seconds of each other are grouped into one window; short
            windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
            shape feeding the AI-handoff phase is identical.

            ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard
against
            the *over-merge* failure where "shuttle moving" is conflated with "rally in
progress"
            and a single window swallows several rallies + the dead time between them. When
set, any
            window longer than this is recursively SPLIT at its largest internal inactivity
gap (the
            longest stretch with no in-flight shuttle ? the most likely rally boundary),
provided
            that gap is at least ``min_split_gap`` seconds. This is the bounded-windowing
fix from
            ``docs/RALLY_QUALITY_RESEARCH.md`` §6 (W1); it never merges, only cuts, so
recall cannot
            drop, and it is config-gated default-OFF until measured on the golden set.

            ``window_hard_cap`` (default OFF) makes ``max_window_duration`` a TRUE cap: when
an
            over-long window has NO internal inactivity gap to split on (a continuously-
active
            trajectory ? the shuttle never stops moving, e.g. the real-footage
``mahadevpura-1``
            blob where the gap-splitter alone left a single 174 s window), it is recursively
            hard-chopped at its temporal midpoint until every piece is ? the cap. Still only
cuts
            (never merges), so recall cannot drop; gated default-OFF until measured.

            ``window_blob_fallback`` (default OFF ? the R5 last-resort blob splitter):
unlike
            ``window_hard_cap`` (which midpoint-chops BEFORE the R4 trim, so it fires on
every gap-less
            over-cap window and over-segments normal clips), this runs AFTER the R4
inactivity trim and
            force-chops ONLY a genuine *blob* ? a group still longer than
``window_blob_factor`` ×
            ``max_window_duration`` once the gap-split and the principled rally-end trim
have both had
            their chance (e.g. mahadevpura-1's ~65 s residual ? 11× a 6 s cap; a 7 s rally
is left
            alone). The blob is midpoint-chopped down to ? the cap, those forced cuts are
logged, and
            each resulting window is flagged ``forced_cut=True`` ? surfacing the clip as one
```

```
that needs
307             rally-STATE cues (net-crossings / two-sided motion), not a bigger cap. Only cuts
(recall
308             can't drop). Requires ``max_window_duration`` > 0; gated default-OFF until
measured.
309             """
310             if fps <= 0:
311                 fps = 30.0
312             if frame_width <= 0:
313                 frame_width = 1280.0
314
315             # Compute per-frame normalised velocity from the last *visible* point.
316             active = []  # (frame_idx, velocity)
317             prev: Optional[TrajectoryPoint] = None
318             for p in points:
319                 if not p.visible:
320                     prev = None  # break the trajectory; absent shuttle = not in flight
321                     continue
322                 if prev is not None:
323                     dframes = p.frame - prev.frame
324                     if dframes > 0:
325                         dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
326                         velocity = (dist / frame_width) * (fps / dframes)
327                         if velocity > velocity_thresh:
328                             active.append((p.frame, velocity))
329                 prev = p
330
331             if not active:
332                 return []
333
334             max_gap_frames = int(merge_gap * fps)
335
336             def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
337                 vels = [v for _, v in group]
338                 return {
339                     "start": group[0][0] / fps + chunk_start,
340                     "end": group[-1][0] / fps + chunk_start,
341                     "active_frame_count": len(group),
342                     "peak_velocity": max(vels),
343                     "mean_velocity": sum(vels) / len(vels),
344                     "track_id_count": 1,  # single shuttle; kept for window-shape parity
345                     "chunk_index": chunk_index,
346                 }
347
348             groups: List[List[Tuple[int, float]]] = []
349             group = [active[0]]
350             for af in active[1:]:
351                 if af[0] - group[-1][0] <= max_gap_frames:
352                     group.append(af)
353                 else:
354                     groups.append(group)
355                     group = [af]
356             groups.append(group)
357
358             if max_window_duration > 0:
359                 groups = TrackNetRunner._split_overlong_groups(
360                     groups, int(max_window_duration * fps), int(min_split_gap * fps),
361                     hard_cap=window_hard_cap)
362
363             # R4 ? rally-END inactivity trim (default OFF). Applied AFTER the cap split so
each rally
364             # piece has its own post-rally drift tail removed: the velocity windowing keeps
a window
365             # "active" while the shuttle drifts/gets-picked-up above threshold, over-
extending the END
```

```
            # (the measured gap vs golden). Trim back to the last frame whose trailing
window is still
            # dense with motion (rally on); a sustained low-density run = rally over. Only
cuts.
            if window_inactivity_end > 0:
                tw = int(window_inactivity_end * fps)
                groups = [TrackNetRunner._trim_inactive_tail(g, tw, inactivity_min_density,
                                                             inactivity_rally_thresh) for g
in groups]

            # R5 ? blob-fallback (default OFF). LAST resort: after the gap-split AND the R4
trim, any
            # group still longer than the cap is a gap-less, rally-end-signal-less blob;
force-chop it to
            # ? the cap at the midpoint and flag every piece, so the degenerate single-
window outcome is
            # avoided and the clip is surfaced (logged) as needing rally-STATE cues, not a
bigger cap.
            forced_flags = [False] * len(groups)
            if window_blob_fallback and max_window_duration > 0:
                groups, forced_flags = TrackNetRunner._blob_fallback_chop(
                    groups, int(max_window_duration * fps), window_blob_factor)

            windows = []
            for g, forced in zip(groups, forced_flags):
                w = _finalize(g)
                if forced:
                    w["forced_cut"] = True
                windows.append(w)
            return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]

        @staticmethod
        def _trim_inactive_tail(group: List[Tuple[int, float]], trail_frames: int,
                                min_density: float, rally_thresh: float) -> List[Tuple[int,
float]]:
            """R4: trim the post-rally drift tail. After a rally the shuttle keeps moving
*slowly*
            (picked up / walked / knock-up) above ``velocity_thresh``, so the velocity
windowing
            over-extends the END. A frame is "fast" (real rally motion) iff its velocity >
``rally_thresh``
            (higher than the active threshold). The rally is "on" at frame ``i`` while its
trailing
            ``trail_frames`` window holds >= ``min_density`` fraction of fast frames; trim
back to the
            LAST such frame ? everything after is sustained slow drift (rally over). Only
cuts (recall
            can't rise); a window whose tail never goes slow is untouched, and if no fast-
dense point
            exists at all the group is kept whole (fail-safe, no over-trim). O(n) two-
pointer; the START
            is left alone (already ~Gemini-accurate per the n=2 boundary-error finding)."""
            if trail_frames <= 0 or len(group) < 2:
                return group
            frames = [f for f, _ in group]
            fast = [1 if v > rally_thresh else 0 for _, v in group]
            lo = 0
            fast_sum = 0
            last_dense = -1
            for i in range(len(frames)):
                fast_sum += fast[i]
                while frames[lo] <= frames[i] - trail_frames:
                    fast_sum -= fast[lo]
                    lo += 1
                win_count = i - lo + 1
```

```
                if win_count > 0 and fast_sum / win_count >= min_density:
                    last_dense = i
            return group[: last_dense + 1] if last_dense >= 0 else group

        @staticmethod
        def _split_overlong_groups(groups: List[List[Tuple[int, float]]], max_win_frames:
int,
                                   min_split_frames: int,
                                   hard_cap: bool = False) -> List[List[Tuple[int, float]]]:
            """Recursively split any active-frame group longer than ``max_win_frames`` at
its largest
            internal gap (? ``min_split_frames``) ? an inactivity-aware cut at the most
likely rally
            boundary. Splitting only subdivides existing groups (never merges/extends), so
it cannot
            invent or drop coverage; a group with no gap ? the floor is left intact (a
genuinely long
            rally) UNLESS ``hard_cap`` is set, in which case such a group is hard-chopped at
its
            temporal midpoint until every piece is ? ``max_win_frames`` (makes the cap a
true upper
            bound on continuously-active trajectories). Iterative worklist to bound stack
depth."""
            if max_win_frames <= 0:
                return groups
            out: List[List[Tuple[int, float]]] = []
            work = list(groups)
            while work:
                g = work.pop()
                if len(g) < 2 or (g[-1][0] - g[0][0]) <= max_win_frames:
                    out.append(g)
                    continue
                best_i, best_gap = -1, -1
                for i in range(1, len(g)):
                    gap = g[i][0] - g[i - 1][0]
                    if gap > best_gap:
                        best_gap, best_i = gap, i
                if best_i < 0 or best_gap < min_split_frames:
                    # Too long but no real dead-time gap. Default: keep as one (a genuine
long rally).
                    # hard_cap: chop at the temporal midpoint so the cap is actually
enforced.
                    if hard_cap:
                        mid = (g[0][0] + g[-1][0]) / 2.0
                        split_i = min(range(1, len(g)), key=lambda i: abs(g[i][0] - mid))
                        work.append(g[:split_i])
                        work.append(g[split_i:])
                    else:
                        out.append(g)
                    continue
                work.append(g[:best_i])
                work.append(g[best_i:])
            out.sort(key=lambda grp: grp[0][0])
            return out

        @staticmethod
        def _blob_fallback_chop(groups: List[List[Tuple[int, float]]], max_win_frames: int,
                                blob_factor: float = 4.0
                                ) -> Tuple[List[List[Tuple[int, float]]], List[bool]]:
            """R5 blob splitter ? the gap-less, R4-survived residual. After the W1 gap-split
AND the R4
            inactivity trim have each had their chance, a group still longer than
``blob_factor`` ×
            ``max_win_frames`` is a genuine continuously-active blob (no internal gap, no
rally-end
```

```
467             signal the principled rules could find ? the real-footage ``mahadevpura-1`` ~65
s residual ?
468             11× a 6 s cap). The factor is what keeps this SURGICAL: a normal long rally
(~1?2× the cap)
469             is left ALONE; only the degenerate blob qualifies. As a LAST RESORT the blob is
470             midpoint-chopped to ? the cap and every resulting piece is FLAGGED as a forced
cut, so the
471             degenerate single-window outcome is avoided and the clip is surfaced (logged) as
needing
472             rally-STATE cues ? net-crossings / two-sided motion ? not a bigger cap. Only
cuts (recall
473             can't drop). Returns ``(groups, forced_flags)`` aligned 1:1; ``forced_flags[i]``
is True iff
474             that piece came from a forced chop. Iterative worklist to bound stack depth."""
475             if max_win_frames <= 0:
476                 return groups, [False] * len(groups)
477             blob_frames = max_win_frames * max(1.0, blob_factor)
478             out: List[List[Tuple[int, float]]] = []
479             flags: List[bool] = []
480             forced = 0
481             for g in groups:
482                 if len(g) < 2 or (g[-1][0] - g[0][0]) <= blob_frames:
483                     out.append(g)
484                     flags.append(False)
485                     continue
486                 pieces: List[List[Tuple[int, float]]] = []
487                 work = [g]
488                 while work:
489                     h = work.pop()
490                     if len(h) < 2 or (h[-1][0] - h[0][0]) <= max_win_frames:
491                         pieces.append(h)
492                         continue
493                     mid = (h[0][0] + h[-1][0]) / 2.0
494                     split_i = min(range(1, len(h)), key=lambda i: abs(h[i][0] - mid))  #
noqa: B023
495                     work.append(h[:split_i])
496                     work.append(h[split_i:])
497                 pieces.sort(key=lambda grp: grp[0][0])
498                 out.extend(pieces)
499                 flags.extend([True] * len(pieces))
500                 forced += len(pieces) - 1
501             if forced:
502                 logger.warning(
503                     "R5 blob-fallback: forced %d midpoint cut(s) on continuously-active
group(s) with "
504                     "no rally-end signal ? this clip needs rally-STATE cues (net-crossings /
two-sided "
505                     "motion), not a bigger cap", forced)
506             return out, flags
507
```

46. assistant (2026-06-18T01:35:08.917Z)

`parse_trajectory_csv` properly catches KeyError/ValueError on missing CSV columns (line 250). Well-guarded.

Let me now check `distill_local.py`, `fusion_hybrid.py`, `wasb_runner.py`, and the `per_user_gate.py`/`per_user_eval.py`/`served_gate_a.py`. Let me also look at `eval/annotations.py` and `metrics.py` more briefly. Let me batch.

47. user (2026-06-18T01:35:09.552Z)

```
1       """Distill Gemini (teacher) into a LEARNED local rally classifier (student).
2
3       Threshold-tuning the local knobs hit a low ceiling (see calibrate_local). This goes
```

```
    further: learn "is this WASB candidate a real rally?" from Gemini silver labels,
    using a few resolution/fps-INVARIANT local features, so the model transfers across
    videos and runs 100% local at inference (no Gemini, privacy lever intact).

    Flow (all local at runtime):
      WASB trajectory -> recall-oriented candidate windows (propose many; classifier
      filters) -> per-candidate features [duration, peak_velocity, mean_velocity,
      active_ratio, net_crossings] -> LogisticRegression(prob >= threshold) -> merge
      -> rally windows.

    Training labels come from Gemini full-context labels via the SAME IoU matching the
    evaluator uses (training.label_candidates). Honest leave-one-video-out: train on the
    other videos' candidates, predict the held-out one, score vs its Gemini labels, and
    compare against the threshold baseline. Reuses classifier.py, training.label_candidates,
    rally_gate, metrics, calibrate_wasb.load_golden_gts, gemini_refine.merge_overlaps.

    Run:
        python -m backend.eval.distill_local --manifest videos.json --model-out
output/local_rally_model.json
    """
    from __future__ import annotations

    import json
    import argparse
    from typing import Dict, List, Optional, Tuple


    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
    from backend.pipeline.detectors import rally_gate
    from backend.eval.calibrate_wasb import load_golden_gts
    from backend.eval.training import label_candidates
    from backend.eval.classifier import LogisticRegression
    from backend.eval.metrics import score
    from backend.eval.gemini_refine import merge_overlaps
    from backend.eval import windowing as _windowing

    Interval = Tuple[float, float]

    FEATURE_NAMES = ["duration", "peak_velocity", "mean_velocity", "active_ratio",
"net_crossings"]
    # Recall-oriented proposal: deliberately permissive so real rallies are among the
    # candidates (the classifier removes the junk). (velocity_thresh, merge_gap, min_dur).
    # Single-sourced from backend.eval.windowing so eval and serve can never drift ? see
that
    # module's docstring for the eval?serve bug these named presets close. The bare tuples
are
    # preserved (byte-identical: PROPOSAL == (0.005, 1.0, 0.5), BASELINE_LOCAL == SERVED ==
    # (0.02, 2.0, 2.0)) so every existing call site / golden JSON stays valid.
    PROPOSAL = _windowing.PROPOSAL.as_tuple()
    BASELINE_LOCAL = _windowing.SERVED.as_tuple()   # the served (production) windowing, no
learning


    def build_candidates(trajectory_csv: str, fps: float, frame_width: float,
                         proposal: Tuple[float, float, float] = PROPOSAL) ->
Tuple[List[Interval], List[List[float]]]:
        """WASB trajectory -> (candidate intervals, fps/resolution-invariant feature
rows)."""
        points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
        vt, mg, mwd = proposal
        wins = TrackNetRunner.trajectory_to_action_windows(
            points, fps=fps, frame_width=frame_width,
            velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
        intervals = [(w["start"], w["end"]) for w in wins]
        gated = rally_gate.gate_windows(points, intervals, fps, min_crossings=0)  # for net-
```

```
crossings
62          X: List[List[float]] = []
63          for w, g in zip(wins, gated):
64              dur = max(1e-6, w["end"] - w["start"])
65              win_frames = max(1.0, dur * fps)
66              X.append([
67                  dur,                                            # rally length (sec) ?
fps-invariant
68                  float(w["peak_velocity"]),                      # already normalized by
frame_width
69                  float(w["mean_velocity"]),
70                  float(w["active_frame_count"]) / win_frames,     # active-shuttle ratio ?
fps-invariant
71                  float(g.crossings),                             # net crossings (count) ?
the rally signal
72              ])
73          return intervals, X
74
75
76      def baseline_windows(trajectory_csv: str, fps: float, frame_width: float) ->
List[Interval]:
77          points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
78          vt, mg, mwd = BASELINE_LOCAL
79          wins = TrackNetRunner.trajectory_to_action_windows(
80              points, fps=fps, frame_width=frame_width,
81              velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
82          return [(w["start"], w["end"]) for w in wins]
83
84
85      def load_video(spec: Dict, iou: float = 0.5) -> Dict:
86          fps, fw = float(spec["fps"]), float(spec["frame_width"])
87          intervals, X = build_candidates(spec["trajectory"], fps, fw)
88          gts = load_golden_gts(spec["labels"])
89          y = label_candidates(intervals, gts, iou)
90          return {"name": spec.get("name", spec["trajectory"]), "intervals": intervals, "X":
X,
91                  "y": y, "gts": gts, "baseline": baseline_windows(spec["trajectory"], fps,
fw)}
92
93
94      def predict_rallies(model: LogisticRegression, X: List[List[float]], intervals:
List[Interval],
95                          threshold: float = 0.5) -> List[Interval]:
96          if not intervals:
97              return []
98          proba = model.predict_proba(X)
99          pos = [iv for iv, p in zip(intervals, proba) if p >= threshold]
100         return merge_overlaps(pos, gap_tol=1.0)
101
102
103     def run(specs: List[Dict], iou: float = 0.5, threshold: float = 0.5,
104             model_out: Optional[str] = None) -> dict:
105         videos = [load_video(s, iou) for s in specs]
106         print(f"=== Distilled local rally classifier vs Gemini silver-GT "
107               f"({len(videos)} videos, IoU>={iou}, prob>={threshold}) ===")
108         for v in videos:
109             ceil = score(v["intervals"], v["gts"], iou).recall
110             print(f"[{v['name']}] {len(v['intervals'])} candidates ({sum(v['y'])} pos) | "
111                   f"proposal recall ceiling {ceil:.3f} | {len(v['gts'])} Gemini rallies")
112
113         result: dict = {"videos": [v["name"] for v in videos]}
114         if len(videos) >= 2:
115             print("\n--- HONEST leave-one-video-out: learned vs threshold baseline (held-
out) ---")
116             clf, base = [], []
```

```
117             for i, held in enumerate(videos):
118                 others = [v for j, v in enumerate(videos) if j != i]
119                 X = [row for v in others for row in v["X"]]
120                 y = [t for v in others for t in v["y"]]
121                 model = LogisticRegression().fit(X, y, FEATURE_NAMES)
122                 preds = predict_rallies(model, held["X"], held["intervals"], threshold)
123                 sc = score(preds, held["gts"], iou)
124                 bsc = score(held["baseline"], held["gts"], iou)
125                 clf.append(sc)
126                 base.append(bsc)
127                 print(f"  held-out {held['name']}: LEARNED P={sc.precision:.3f}
R={sc.recall:.3f} F1={sc.f1:.3f}"
128                       f"  |  baseline P={bsc.precision:.3f} R={bsc.recall:.3f}
F1={bsc.f1:.3f}")
129             def mean(xs, a):
130                 return sum(getattr(s, a) for s in xs) / len(xs)
131             print(f"  MEAN held-out: LEARNED P={mean(clf,'precision'):.3f}
R={mean(clf,'recall'):.3f} "
132                   f"F1={mean(clf,'f1'):.3f}  |  baseline P={mean(base,'precision'):.3f} "
133                   f"R={mean(base,'recall'):.3f} F1={mean(base,'f1'):.3f}")
134             result["mean_learned_f1"] = mean(clf, "f1")
135             result["mean_baseline_f1"] = mean(base, "f1")
136
137         # Final model on ALL videos (for production / running on a fresh held-out video).
138         Xall = [row for v in videos for row in v["X"]]
139         yall = [t for v in videos for t in v["y"]]
140         final = LogisticRegression().fit(Xall, yall, FEATURE_NAMES)
141         assert final.w is not None  # set by fit()
142         weights = dict(zip(FEATURE_NAMES, [round(float(w), 3) for w in final.w]))
143         print(f"\nfinal model standardized weights (sign/size = influence): {weights}")
144         if model_out:
145             final.save(model_out)
146             print(f"final model (all videos) -> {model_out}")
147             result["model_out"] = model_out
148         return result
149
150
151     def main() -> None:
152         ap = argparse.ArgumentParser(description="Distill Gemini labels into a local rally
classifier.")
153         ap.add_argument("--manifest", required=True,
154                         help="JSON: [{name, trajectory, fps, frame_width, labels}, ...]
(labels = Gemini CSV)")
155         ap.add_argument("--iou", type=float, default=0.5)
156         ap.add_argument("--threshold", type=float, default=0.5, help="classifier probability
cutoff")
157         ap.add_argument("--model-out", default="output/local_rally_model.json")
158         args = ap.parse_args()
159         with open(args.manifest, encoding="utf-8") as f:
160             specs = json.load(f)
161         run(specs, iou=args.iou, threshold=args.threshold, model_out=args.model_out)
162
163
164     if __name__ == "__main__":
165         main()
166
```

48. assistant (2026-06-18T01:35:15.699Z)

`load_video` (line 85-88) reads `spec["fps"]`, `spec["frame_width"]`, `spec["trajectory"]`, `spec["labels"]` unguarded from manifest JSON ? same pattern as `fusion_golden.py`. These are operator-written manifests, KeyError on a malformed one. Same category as before (low-confidence operator-facing CLI).

Let me now check the segmenter hybrids: `fusion_hybrid.py`, `wasb_hybrid.py`, `yolo_hybrid.py`.

These read config dicts heavily and persisted candidate stats. Let me also check the `per_user_gate.py` and `served_gate_a.py`. Let me check the most likely defect spots ? config access and persisted-stats reads in fusion_hybrid.

49. user (2026-06-18T01:35:16.880Z)

```
1       """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3       Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4       TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5       do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6       shuttle-seeded window via two overridable seams the engine exposes:
7
8       - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9         GPU-free, computed from the trajectory we already have); and, only when the person cue
10        is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11        (`fusion_features`), persisting them into ``candidate_segments.stats`` for the learned
12        fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads
the
13        torchvision/supervision model ? so the default (person-OFF) path never loads a
detector
14        model nor touches the GPU. (torch/cv2 themselves are imported by the registry
regardless,
15        via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
16      - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
17        (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
18        dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19        **Persisted candidates remain the full superset** regardless, so the offline
experiment
20        harness can re-score any variant.
21
22      Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
23      nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
24      and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
25      segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
26      experiment harness (EXPERIMENT_HARNESS.md), never silently.
27      """
28      from typing import Any, Dict, List, Optional, Tuple
29
30      from backend.pipeline.segmenters.base import segmenter_registry
31      from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
32      from backend.pipeline.detectors.base import DetectorRunner
33      from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
34      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
35      from backend.pipeline.detectors import rally_gate
36      from backend.pipeline.detectors import fusion_features as ff
37
38
39      class FusionSegmenter(TrajectoryHybridSegmenter):
40          """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
B)."""
41
42          family = "fusion"
43          display_label = "Fusion"
44
45          def __init__(self, db: Any, config: Any):
46              super().__init__(db, config)
47              # Built lazily only if the person cue is enabled (no detector model loaded
otherwise).
```

```
48              self._person: Optional[Any] = None
49              # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-
estimate it
50              # over the whole trajectory once per candidate window (it depends only on
`points`).
51              self._axis_cache_key: Optional[int] = None
52              self._axis_cache: Optional[tuple] = None
53
54          @property
55          def name(self) -> str:
56              return "fusion_hybrid"
57
58          @property
59          def description(self) -> str:
60              return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
61                      "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the "
62                      "AI provider sets semantic boundaries.")
63
64          def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
65              substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
66              if substrate == "tracknet":
67                  cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
68                  return TrackNetRunner(cfg), {
69                      "weights_path": cfg.weights_path,
70                      "queue_length": cfg.queue_length,
71                      "fusion_substrate": "tracknet",
72                  }
73              wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
74              return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}
75
76          def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
77                                      frame_width: float, video_path: str,
78                                      idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
79              stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
80              # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
81              # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
82              # over the whole trajectory by rally_gate (camera-agnostic); reused across
windows.
83              gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
84                                              min_crossings=0,
estimate=self._axis_estimate(points))
85              stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
86
87              fcfg = idx_cfg.get("fusion", {})
88              if fcfg.get("cue_flags", {}).get("person", False):
89                  stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
90              return stats
91
92          # P3 (learned fusion) first-step landing note (per MULTI_SIGNAL_FUSION_PLAN +
NEXT_STEPS 2026-06-14):
93          # Person features (from _person_features when cue_flags.person) are now persisted
for the harness
94          # `compare` LOVO litmus on the 6 golden videos. Eager-person-compute optimisation
(compute only
95          # for shuttle-seeded windows + padding, not whole video) is already in place via the
lazy
96          # self._person + windowed detections_per_frame. Next: actual harness run + lift
```

```
measurement
97          # (owner/hardware) before any served promotion. Flag-gated (default person OFF). See
98          # docs/NEXT_STEPS.md "Rally-detection quality track" and EXPERIMENT_HARNESS.md for
discipline.
99
100         def _axis_estimate(self, points: List[Any]) -> tuple:
101             """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so
it's
102             computed once per video, not once per candidate window (it depends only on
points)."""
103             key = id(points)
104             if self._axis_cache_key != key or self._axis_cache is None:
105                 self._axis_cache_key = key
106                 self._axis_cache = rally_gate.estimate_play_axis(points)
107             est = self._axis_cache
108             assert est is not None
109             return est
110
111         def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
112                              frame_width: float, video_path: str,
113                              fcfg: Dict[str, Any]) -> Dict[str, float]:
114             """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
115             frame range and compute the scale/translation-invariant person features. Lazily
builds
116             ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
117             # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
118             from backend.pipeline.detectors.person_detector import PersonDetector
119             if self._person is None:
120                 self._person = PersonDetector(self.config)
121
122             frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
123             # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
124             # window-edge off-by-one that drops the boundary frame from the shuttle
alignment.
125             start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
126             det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
127             fw = det["frame_width"] or frame_width
128             fh = det["frame_height"]
129             # If we can't trust the frame dims, every normalised feature would silently
collapse
130             # to ~0 ? return an explicit all-zero person-feature row instead of a half-baked
one.
131             if fw <= 0 or fh <= 0:
132                 return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}
133
134             poly = fcfg.get("court_polygon")
135             polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
136             net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
137             shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
138             return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
139                                               court_polygon=polygon, net=net)
140
141         def _select_for_handoff(self, windows: List[Dict[str, Any]],
142                                 window_stats: List[Dict[str, Any]],
143                                 idx_cfg: Dict[str, Any]) -> List[int]:
144             """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
145             (0) keep every window ? identical to the substrate. Persisted candidates are
146             untouched (full superset).
147
148             NEVER-EMPTY safeguard: a video that had ?1 window must keep ?1 after gating. If
the
```

```
149             thresholds would drop EVERY window (e.g. a short, noisy or one-sided-feed
trajectory
150             where no segment clears ``min_crossings``), the gate falls back to the full
ungated
151             set rather than emit a video with zero rallies. This makes Gate A/B strictly
152             *pruning* ? it can shrink a candidate set but never zero it out ? so layering it
on
153             the substrate is "do no harm": the bounded-windowing / largest-internal-gap
split
154             below it always wins on an otherwise-empty video. Observed on real footage
155             (mahadevpura-1): a single mega-window split into pieces, none clearing the
crossings
156             gate ? without this guard the whole clip yielded 0 rallies (no reel)."""
157             fcfg = idx_cfg.get("fusion", {})
158             min_crossings = int(fcfg.get("min_crossings", 0))
159             min_players = int(fcfg.get("min_players", 0))
160             person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
161
162             keep: List[int] = []
163             for i, st in enumerate(window_stats):
164                 if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
165                     continue
166                 if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
167                     continue
168                 keep.append(i)
169
170             if not keep and window_stats:
171                 # Gating emptied a non-empty video ? fall back to the ungated set.
172                 return list(range(len(window_stats)))
173             return keep
174
175
176     segmenter_registry.register(FusionSegmenter)
177
```

50. assistant (2026-06-18T01:35:22.688Z)

Line 134-135: `poly = fcfg.get("court_polygon")` then `polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None`. `court_polygon` comes from config (persisted config.json under `indexing.fusion`). If a user mis-configures `court_polygon` as a list of scalars instead of pairs, `p[0]` would fail ? but this is config the operator controls, and the person cue is default-OFF. Low concern.

Line 127: `det["frame_width"]`, line 128: `det["frame_height"]`, line 138: `det["frames"]` ? these come from `detections_per_frame`, an internal detector method with a controlled return contract. Fine.

Let me check `wasb_runner.py` and `wasb_hybrid.py`, plus `person_detector.py` (which produces `det`). Let me also look at `per_user_gate.py` and `served_gate_a.py`. Let me look at the WASB runner which parses the WASB inference output.

51. user (2026-06-18T01:35:23.970Z)

```
1       """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3       Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4         1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5         2. copies wasb_infer.py into the WASB repo's src/ dir,
6         3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7         4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9       So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10      with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11      """
```

```
    import os
    import shlex
    import logging
    import subprocess
    from dataclasses import dataclass
    from typing import Dict, Any, Optional, Tuple

    # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
    # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
TrackNetRunner.
    from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
    from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote

    logger = logging.getLogger(__name__)

    # Windows path to the inference wrapper we stage into WSL.
    _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")


    @dataclass
    class WasbConfig:
        """Resolved settings for a WASB WSL run (built from config.json ->
indexing.wasb)."""
        distro: str = "Ubuntu"
        repo_dir: str = "~/models/WASB-SBDT"
        conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
        conda_env: str = "wasb"
        weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
        sport: str = "badminton"
        wsl_stage_dir: str = "~/clips"
        timeout_sec: int = 1800  # 30 min; a hung GPU/conda call is killed (0 = no timeout)
        keep_frames: bool = False  # retain staged video + frame cache after success (debug
only)
        min_free_gb: float = 0.0   # warn if WSL free space below this before a run (0 =
off)
        # FAST PATH (default OFF until verified): decode the staged video on the fly inside
        # WSL and feed frames straight to the detector, skipping the slow per-frame PNG
        # extraction that dominates a long clip. Output is bit-identical to the PNG path
        # (PNG is lossless), but this stays opt-in until the owner's side-by-side confirms
it.
        stream_video: bool = False

        @classmethod
        def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
            w = (idx_cfg or {}).get("wasb", {}) or {}
            return cls(
                distro=w.get("wsl_distro", "Ubuntu"),
                repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
                conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
                conda_env=w.get("conda_env", "wasb"),
                weights_path=w.get("weights_path",
                                   "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
                sport=w.get("sport", "badminton"),
                wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
                timeout_sec=int(w.get("timeout_sec", 1800)),
                keep_frames=bool(w.get("keep_frames", False)),
                min_free_gb=float(w.get("min_free_gb", 0.0)),
                stream_video=bool(w.get("stream_video", False)),
            )


    class WasbRunner(WslCommandMixin, DetectorRunner):
```

```
69          def __init__(self, cfg: WasbConfig):
70              self.cfg = cfg
71
72          def healthcheck(self) -> Tuple[bool, str]:
73              """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
74              cmd = (f"conda activate {self.cfg.conda_env} && "
75                     f"python -c \"import torch; print('torch', torch.__version__, "
76                     f"'cuda', torch.cuda.is_available())\"")
77              try:
78                  res = self._wsl_bash(cmd)
79              except FileNotFoundError:
80                  return False, "`wsl` not found ? Windows + WSL2 required."
81              except subprocess.TimeoutExpired:
82                  return False, "WSL healthcheck timed out."
83              out = (res.stdout or "").strip()
84              if res.returncode != 0:
85                  return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
86              return True, out
87
88          def _stage_video(self, video_win_path: str, dest_name: str) -> Optional[str]:
89              src_mnt = to_wsl_mnt_path(video_win_path)
90              reason = self.wsl_source_unreadable_reason(src_mnt)
91              if reason:
92                  logger.error("Cannot stage video into WSL: %s", reason)
93                  return None
94              dest = f"{self.cfg.wsl_stage_dir}/{dest_name}"
95              cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
96              try:
97                  res = self._wsl_bash(cmd)
98              except subprocess.TimeoutExpired:
99                  logger.error("Staging video into WSL timed out.")
100                 return None
101             if res.returncode != 0 or "OK" not in (res.stdout or ""):
102                 logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
103                 return None
104             return dest
105
106         def _stage_infer_script(self) -> bool:
107             """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
108             src_mnt = to_wsl_mnt_path(_INFER_WIN)
109             cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo
OK"
110             res = self._wsl_bash(cmd)
111             return res.returncode == 0 and "OK" in (res.stdout or "")
112
113         def run_predict(self, video_win_path: str, output_win_dir: str,
114                         video_id: Optional[str] = None) -> Optional[str]:
115             """Run WASB on a video. Returns the Windows path to the trajectory CSV, or None.
116
117             All WSL/cache/output artifacts are namespaced by ``video_id`` (the file MD5) so
118             two videos that share a filename (e.g. two ``match.mp4`` from different folders)
119             can never reuse each other's staged frames, resumable cache, or CSV.
120             """
121             # Resolve relative dirs (the hybrids pass "output/wasb") BEFORE the /mnt
122             # translation: to_wsl_mnt_path passes relative paths through unchanged, so
123             # WSL resolved them against the inference cwd (~/models/WASB-SBDT/src) and
124             # the CSV landed inside WSL while Windows polled the repo-relative path.
125             output_win_dir = os.path.abspath(output_win_dir)
126             os.makedirs(output_win_dir, exist_ok=True)
127             # Normalize for cross-platform basename (tests use Windows paths on Linux).
128             video_win_path = str(video_win_path).replace("\\", "/")
129             base = os.path.basename(video_win_path)
130             stem, ext = os.path.splitext(base)
```

```
131             # Unique, content-derived key: same filename + different bytes -> different key.
132             key = f"{stem}__{video_id[:12]}" if video_id else stem
133             wsl_out_dir = to_wsl_mnt_path(output_win_dir)
134             expected_csv_win = os.path.join(output_win_dir, f"{key}_wasb.csv")
135
136             if not self._stage_infer_script():
137                 logger.error("Could not stage wasb_infer.py into WSL.")
138                 return None
139             staged_video = self._stage_video(video_win_path, f"{key}{ext}")
140             if staged_video is None:
141                 return None
142             frames_dir = f"{self.cfg.wsl_stage_dir}/{key}_frames"
143
144             # Disk guard: a 4K clip can extract ~100 GB of PNG frames. Warn early if the
145             # WSL stage filesystem is already low so the user can clean up before it fills.
146             # (Streaming never materializes the PNGs, so the guard only matters off the fast
path.)
147             if self.cfg.min_free_gb > 0 and not self.cfg.stream_video:
148                 free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
149                 if free is not None and free < self.cfg.min_free_gb:
150                     logger.warning(
151                         "Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f). Frame
extraction "
152                         "may fill the disk ? consider running tools/cleanup_caches.py.",
153                         free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
154
155             # Fast path: stream-decode the video in-process (no PNG extraction). Output is
156             # bit-identical to the PNG round-trip but avoids writing ~46k PNGs for a 12-min
clip.
157             if self.cfg.stream_video:
158                 frame_args = "--stream-video "
159             else:
160                 frame_args = f"--frames_out_dir {wsl_tilde_quote(frames_dir)} "
161             cmd = (
162                 f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
163                 f"python wasb_infer.py "
164                 f"--video {wsl_tilde_quote(staged_video)} "
165                 f"{frame_args}"
166                 f"--weights {wsl_tilde_quote(self.cfg.weights_path)} "
167                 f"--sport {shlex.quote(self.cfg.sport)} "
168                 f"--out {wsl_tilde_quote(wsl_out_dir + '/' + key + '_wasb.csv')}"
169             )
170             logger.info("Running WASB inference on %s ...", base)
171             try:
172                 res = self._wsl_bash(cmd)
173             except subprocess.TimeoutExpired:
174                 logger.error("WASB inference timed out after %ss.", self.cfg.timeout_sec)
175                 return None
176             if res.returncode != 0:
177                 logger.error("WASB inference failed:\n%s", (res.stderr or
res.stdout)[-2000:])
178                 return None
179             if not os.path.exists(expected_csv_win):
180                 logger.error("WASB finished but expected CSV not found at %s",
expected_csv_win)
181                 return None
182             logger.info("WASB trajectory CSV: %s", expected_csv_win)
183
184             # Success ? the trajectory CSV is the durable output; the staged video copy and
185             # the (huge) decoded frame cache are now pure intermediates. Delete them unless
186             # explicitly retained. On any earlier failure we return above WITHOUT cleaning,
187             # so the frame cache + detector cache remain for a resumable re-run.
188             # (Streaming never writes a frame cache, so there's only the staged video to
drop.)
189             if not self.cfg.keep_frames:
```

```
190                 stale = [staged_video] if self.cfg.stream_video else [frames_dir,
staged_video]
191                 logger.info("Cache hygiene: removing staged video%s for %s "
192                             "(set indexing.wasb.keep_frames=true to retain).",
193                             "" if self.cfg.stream_video else " + frame cache", key)
194                 self._wsl_rm_rf(stale)
195             return expected_csv_win
196
197         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
198         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
199         trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
200
```

52. assistant (2026-06-18T01:35:28.620Z)

Config access uses `.get()` with defaults throughout. Clean. Let me check `person_detector.py` which produces `det` dicts consumed elsewhere, and the remaining eval modules `per_user_gate.py`, `served_gate_a.py`, `per_user_eval.py`, `serve_contrast.py`, `rally_seq.py`. Let me also verify the `eval/annotations.py` parse_timestamp and `metrics.py`. Let me batch a few high-value remaining files.

53. user (2026-06-18T01:35:30.018Z)

```
1       """Load hand-annotated ground-truth rally timestamps from CSV.
2
3       Annotation file format (one file per video):
4
5           start,end
6           00:01:12,00:01:39
7           102.5,131.0
8
9       - A header row `start,end` is optional (auto-detected and skipped).
10      - Each timestamp is either decimal seconds (`102.5`) or colon-separated
11        `MM:SS` / `HH:MM:SS` (`00:01:12`). Both forms may be mixed in one file.
12      - Blank lines and `#` comment lines are ignored.
13      """
14
15      import csv
16      import logging
17      from typing import List, Tuple
18
19      logger = logging.getLogger(__name__)
20
21      Interval = Tuple[float, float]
22
23
24      def parse_timestamp(value: str) -> float:
25          """Parse a timestamp string to float seconds.
26
27          Accepts decimal seconds or colon-separated MM:SS / HH:MM:SS.
28          Raises ValueError on anything unparseable so bad annotations fail loudly.
29          """
30          value = value.strip()
31          if not value:
32              raise ValueError("empty timestamp")
33          if ":" in value:
34              parts = value.split(":")
35              if len(parts) > 3:
36                  raise ValueError(f"too many ':' segments in timestamp '{value}'")
37              total = 0.0
38              for part in parts:                # most-significant first (H, M, S)
39                  total = total * 60 + float(part)
40              return total
```

```
        return float(value)


    def load_annotations(csv_path: str) -> List[Interval]:
        """Load and validate ground-truth rally intervals from a CSV file.

        Delegates to the canonical loader (gt_loader.py, ADR-008 prereq), which accepts BOTH
        golden conventions: positional/headered ``start,end`` (the collector export) and the
        rally-annotator ``rally_number,start_time,end_time,...`` schema. Strict: raises
        ValueError on malformed rows (bad timestamp, start >= end) so labeling mistakes
        surface immediately rather than silently skewing scores. Returns intervals sorted by
        start time.
        """
        from backend.eval.gt_loader import load_gt_intervals
        intervals = load_gt_intervals(csv_path, strict=True)
        logger.info("Loaded %d ground-truth rallies from %s", len(intervals), csv_path)
        return intervals


    def write_scaffold(csv_path: str) -> None:
        """Write an empty annotation file with just the header, for hand-filling."""
        with open(csv_path, "w", newline="") as f:
            writer = csv.writer(f)
            writer.writerow(["start", "end"])

```

54. user (2026-06-18T01:35:30.464Z)

```
    """Per-video calibration driver: WASB trajectory + golden rallies -> tuned thresholds +
"+X%".

    Wires the already-tested pieces together:
      WASB trajectory CSV (Frame,Visibility,X,Y)
        -> TrackNetRunner.trajectory_to_action_windows(cfg)   [predict_fn]
        -> backend.eval.calibration (improvement_report / cross_validate)

    Run (after a full-video WASB pass exists):
        python -m backend.eval.calibrate_wasb --trajectory full_traj.csv \
            --labels "golden_labels_badminton.csv" --fps 59.94 --frame-width 1920

    Honesty note: the grid is *selected* on the full GT, so `improvement_report` on that
    same GT is an **optimistic** upper bound. The trustworthy number is the
    cross-validated held-out **recall** (and, once a 2nd labelled video exists,
    `leave_one_video_out` for precision/F1 generalization).
    """
    from __future__ import annotations

    import argparse
    from typing import List, Tuple, Callable

    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
    from backend.eval.calibration import (
        select_best, improvement_report, cross_validate, evaluate,
    )

    Interval = Tuple[float, float]
    Config = Tuple[float, float, float]   # (velocity_thresh, merge_gap,
min_window_duration)

    BASELINE: Config = (0.02, 2.0, 2.0)   # the WasbConfig / trajectory_to_action_windows
defaults


    def load_golden_gts(csv_path: str) -> List[Interval]:
        """Rally [start,end] seconds from a golden CSV ? delegates to the canonical loader.
```

```
35
36          Accepts BOTH golden conventions now (start_time/end_time AND the collector export's
37          start,end ? the historical fork is closed; see gt_loader.py / ADR-008). Keeps this
38          driver's legacy lenient semantics (skip bad rows), but skipped rows are now logged.
39          """
40          from backend.eval.gt_loader import load_gt_intervals
41          return load_gt_intervals(csv_path, strict=False)
42
43
44      def make_predict_fn(trajectory_csv: str, fps: float, frame_width: float) ->
Callable[[Config], List[Interval]]:
45          """predict_fn(cfg) -> rally windows, from a cached WASB trajectory (parsed once)."""
46          points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
47
48          def predict_fn(cfg: Config) -> List[Interval]:
49              vt, mg, mwd = cfg
50              wins = TrackNetRunner.trajectory_to_action_windows(
51                  points, fps=fps, frame_width=frame_width,
52                  velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
53              )
54              return [(w["start"], w["end"]) for w in wins]
55
56          return predict_fn
57
58
59      def default_grid() -> List[Config]:
60          return [(vt, mg, mwd)
61                  for vt in (0.005, 0.01, 0.02, 0.03, 0.05)
62                  for mg in (1.0, 2.0, 3.0)
63                  for mwd in (1.0, 2.0, 3.0)]
64
65
66      def run(trajectory_csv: str, labels_csv: str, fps: float, frame_width: float,
67              iou: float = 0.5) -> dict:
68          gts = load_golden_gts(labels_csv)
69          if not gts:
70              raise SystemExit(f"no usable golden rallies in {labels_csv}")
71          predict_fn = make_predict_fn(trajectory_csv, fps, frame_width)
72          grid = default_grid()
73
74          base = evaluate(predict_fn(BASELINE), gts, iou)
75          best_cfg, best_f1 = select_best(predict_fn, grid, gts, metric="f1",
iou_threshold=iou)
76          fit = improvement_report(predict_fn, BASELINE, best_cfg, gts, iou)   # OPTIMISTIC
(fit on full GT)
77          cv = cross_validate(predict_fn, grid, gts, k=5)                      # HONEST held-
out recall
78
79          print("=== WASB rally-segmentation calibration ===")
80          print(f"golden rallies: {len(gts)}  | IoU thresh: {iou}")
81          print(f"baseline cfg {BASELINE}: precision={base.precision:.3f}
recall={base.recall:.3f} "
82                f"f1={base.f1:.3f} meanIoU={base.mean_iou:.3f}")
83          print(f"tuned cfg    {best_cfg}: f1={best_f1:.3f}")
84          print(f"  precision {fit['baseline']['precision']:.3f} ->
{fit['tuned']['precision']:.3f} "
85                f"({fit['precision_delta_pct']:+.1f}% , fit-on-full = optimistic)")
86          print(f"  f1        {fit['baseline']['f1']:.3f} -> {fit['tuned']['f1']:.3f} "
87                f"({fit['f1_delta_pct']:+.1f}%)")
88          print(f"HONEST (cross-val, {cv.get('folds','?')} folds): held-out recall "
89                f"{cv.get('mean_test',float('nan')):.3f} ± {cv.get('std_test',0):.3f}, "
90                f"generalization_gap {cv.get('generalization_gap',float('nan')):+.3f}")
91          print("(For precision/F1 generalization, add a 2nd labelled video ->
leave_one_video_out.)")
92          return {"baseline": base.as_dict(), "best_cfg": best_cfg, "fit": fit, "cv": cv}
```

```
93
94
95      def main():
96          ap = argparse.ArgumentParser()
97          ap.add_argument("--trajectory", required=True, help="WASB trajectory CSV
(Frame,Visibility,X,Y)")
98          ap.add_argument("--labels", required=True, help="golden labels CSV
(start_time/end_time cols)")
99          ap.add_argument("--fps", type=float, default=59.94)
100         ap.add_argument("--frame-width", type=float, default=1920.0)
101         ap.add_argument("--iou", type=float, default=0.5)
102         args = ap.parse_args()
103         run(args.trajectory, args.labels, args.fps, args.frame_width, args.iou)
104
105
106     if __name__ == "__main__":
107         main()
108
```

55. assistant (2026-06-18T01:35:36.010Z)

Both clean. Note line 84 `fit['baseline']['precision']` etc. ? but `fit` is built by `improvement_report` internally, controlled shape. Fine.

Let me look at the remaining per-user and gate modules plus `serve_contrast.py`, `served_gate_a.py`, `per_user_gate.py`, `per_user_eval.py`. These read golden feature JSONs (`*_golden_features.json`) produced by fusion_golden, which is external persisted data. Let me check those ? particularly anything that reads `c["person"]` or `c["shuttle"]` from those JSONs.

56. user (2026-06-18T01:35:37.239Z)

```
1       """Held-out-user LEARNING-CURVE eval ? "<K videos to superior quality" as a release
gate.
2
3       The personalization feature's promise is operational, not aspirational: *after the user
4       contributes K of their own videos, the personalized (candidate) recipe should beat the
5       generic (control) one on the user's STILL-UNSEEN footage.* This module turns that
promise
6       into a falsifiable criterion by sweeping the corpus size ``k`` and asking, at each
``k``,
7       **does the candidate beat control on a held-out split, honestly?**
8
9       It is the per-user analogue of `fusion_compare`: pure-CPU, no new scoring/stats math.
Every
10      fold reuses the shipped experiment primitives ?
11
12        - `experiment.predict`  ? variant ? held-out intervals (static variants need no fit);
13        - `experiment._train`   ? fit a learned variant on the corpus split (the same honest
14          train-on-others step `evaluate_variant`'s LOVO uses, here train-on-the-k-corpus);
15        - `metrics.score`       ? per-video held-out F1;
16        - `eval_stats.paired_delta_ci` ? the honest paired ?F1 (bootstrap CI + exact sign
test, C1).
17
18      **Split semantics (held-out USER split, NOT leave-one-out).** At each ``k`` the first
``k``
19      videos are the *personalization corpus* (fit/operating-point source) and the remaining
20      ``len(videos) - k`` are *held-out* (scored, never seen during fit). This mirrors the
product
21      reality ? the user has uploaded ``k`` videos and we want quality on their next one ? and
keeps
22      ``k`` and the evaluation set cleanly disjoint, so a measured lift can't be in-sample.
23
24      The headline ``smallest_k_with_lift`` is the smallest ``k`` at which the candidate beats
25      control with the **CI clearing 0 AND the sign test agreeing** (B wins on more held-out
videos
```

```
26      than it loses) ? the same promotion-grade bar `compare()`'s honest block uses, never a
bare
27      mean. ``None`` means the criterion was never met on this corpus (the honest negative).
28
29      Pure, offline, deterministic given ``seed``; no GPU/torch/cv2; no file I/O (callers pass
30      `VideoCandidates`). See `docs/EXPERIMENT_HARNESS.md` +
`docs/ANALYZERS_AND_RECONCILERS.md` §13/C1.
31      """
32      from __future__ import annotations
33
34      from typing import Any, Dict, List, Optional, Sequence
35
36      from backend.eval import eval_stats as _stats
37      from backend.eval import experiment
38      from backend.eval.classifier import LogisticRegression
39      from backend.eval.experiment import ExperimentError, Variant, VideoCandidates
40      from backend.eval.metrics import score
41
42
43      def _fit_model(variant: Variant, corpus: Sequence[VideoCandidates],
44                     iou: float) -> Optional[LogisticRegression]:
45          """The fitted model a variant needs to predict held-out videos, or None if it needs
none.
46
47          - **static variant** (no learned filter): None ? `predict` scores directly, no
leakage.
48          - **pinned model** (``classifier_model`` set): the shipped model, loaded as-is (no
per-fold
49            training ? reports "how the shipped artifact does on this user's held-out
footage").
50          - **learned recipe** (``feature_names``, no pinned model): fit on the ``corpus``
split via
51            the harness's own honest training step. The corpus and held-out sets are disjoint,
so the
52            held-out scores are genuinely out-of-sample.
53          """
54          if variant.classifier_model is not None:
55              return LogisticRegression.load(variant.classifier_model)
56          if variant.feature_names:
57              return experiment._train(variant, corpus, iou)
58          return None
59
60
61      def _heldout_f1s(variant: Variant, corpus: Sequence[VideoCandidates],
62                       heldout: Sequence[VideoCandidates], iou: float) -> List[float]:
63          """Per-held-out-video F1 for ``variant``, fit on ``corpus`` (if learned), scored on
64          ``heldout``. Held-out videos are video-aligned with the returned list (caller pairs
B?A)."""
65          model = _fit_model(variant, corpus, iou)
66          f1s: List[float] = []
67          for vc in heldout:
68              assert vc.gts is not None  # guarded in learning_curve before any scoring
69              preds = experiment.predict(variant, vc, model=model)
70              f1s.append(score(preds, vc.gts, iou).f1)
71          return f1s
72
73
74      def learning_curve(videos: Sequence[VideoCandidates],
75                         control_variant: Variant,
76                         candidate_variant: Variant,
77                         *,
78                         iou: float = 0.5,
79                         min_k: int = 2,
80                         seed: int = 0) -> Dict[str, Any]:
81          """Sweep corpus size ``k`` and report where the candidate first honestly beats
```

```
control.
82
83          For one user's ``videos``, for each ``k`` in ``min_k .. len(videos) - 1``: fit/score
both
84          variants on the first ``k`` videos (the personalization corpus) and evaluate on the
remaining
85          held-out videos. Per-held-out-video F1s are paired B?A and summarised with
86          ``eval_stats.paired_delta_ci`` (mean ?F1 + bootstrap CI + exact sign test).
87
88          Returns ``{n_videos, min_k, iou, per_k, smallest_k_with_lift}`` where each ``per_k``
record is::
89
90              {k, n_heldout, control_f1, candidate_f1,
91               mean_delta, ci_low, ci_high, ci_excludes_zero, sign_test}
92
93          ``control_f1`` / ``candidate_f1`` are the mean held-out F1 of each variant at that
``k``.
94          ``smallest_k_with_lift`` is the smallest ``k`` whose record shows a real lift ?
candidate
95          above control, the ?F1 CI clearing 0, AND the sign test agreeing (B wins on strictly
more
96          held-out videos than it loses) ? else ``None`` (the honest "never wins" answer).
97
98          Pure + deterministic given ``seed`` (the only randomness is the CI bootstrap).
Raises
99          `ExperimentError` on unlabeled videos or a too-small corpus (no valid ``k`` to
sweep).
100         """
101         n = len(videos)
102         for vc in videos:
103             if vc.gts is None:
104                 raise ExperimentError(
105                     f"{vc.name} has no golden labels; learning_curve needs labeled held-out
videos")
106         if min_k < 1:
107             raise ExperimentError(f"min_k must be >= 1 (got {min_k})")
108         if n <= min_k:
109             # Every k in [min_k, n-1] needs k >= min_k corpus videos AND >= 1 held-out
video, so
110             # n must exceed min_k. Too-few videos ? no sweepable k; report an empty, honest
curve.
111             return {
112                 "n_videos": n,
113                 "min_k": min_k,
114                 "iou": iou,
115                 "per_k": [],
116                 "smallest_k_with_lift": None,
117             }
118
119         per_k: List[Dict[str, Any]] = []
120         smallest_k_with_lift: Optional[int] = None
121         for k in range(min_k, n):
122             corpus = videos[:k]
123             heldout = videos[k:]
124             control_f1s = _heldout_f1s(control_variant, corpus, heldout, iou)
125             candidate_f1s = _heldout_f1s(candidate_variant, corpus, heldout, iou)
126
127             paired = _stats.paired_delta_ci(control_f1s, candidate_f1s, seed=seed)
128             sign_test = paired["sign_test"]
129             # Honest lift = the candidate is above control AND the CI clears 0 AND the sign
test
130             # agrees (B wins on strictly more held-out videos than it loses) ? the promotion
bar.
131             control_mean = sum(control_f1s) / len(control_f1s) if control_f1s else 0.0
132             candidate_mean = sum(candidate_f1s) / len(candidate_f1s) if candidate_f1s else
```

```
0.0
133             has_lift = (
134                 candidate_mean > control_mean
135                 and bool(paired["ci_excludes_zero"])
136                 and paired["mean_delta"] > 0.0
137                 and sign_test["wins"] > sign_test["losses"]
138             )
139             per_k.append({
140                 "k": k,
141                 "n_heldout": len(heldout),
142                 "control_f1": control_mean,
143                 "candidate_f1": candidate_mean,
144                 "mean_delta": paired["mean_delta"],
145                 "ci_low": paired["ci_low"],
146                 "ci_high": paired["ci_high"],
147                 "ci_excludes_zero": bool(paired["ci_excludes_zero"]),
148                 "sign_test": sign_test,
149             })
150             if has_lift and smallest_k_with_lift is None:
151                 smallest_k_with_lift = k
152 
153         return {
154             "n_videos": n,
155             "min_k": min_k,
156             "iou": iou,
157             "per_k": per_k,
158             "smallest_k_with_lift": smallest_k_with_lift,
159         }
160 
```

57. assistant (2026-06-18T01:35:42.783Z)

Clean ? `paired[...]` and `sign_test[...]` are from internal `paired_delta_ci`. Let me check the modules that read the external golden-features JSON (`served_gate_a.py`, `serve_contrast.py`, `per_user_gate.py`) ? these are the consumers of `fusion_golden`'s output JSON with `candidates[].shuttle`/`.person`. Let me read them.

58. user (2026-06-18T01:35:43.733Z)

```
1       """SERVED Gate-A verification: run the *production* ``FusionSegmenter`` decision seams
2       on a cached WASB trajectory and score the resulting AI-handoff windows vs golden labels.
3 
4       The offline litmus (``backend.eval.ablation`` / ``fusion_compare``) measures Gate A on
the
5       **permissive proposal windowing** (``distill_local.PROPOSAL`` = recall-oriented) re-
scored
6       through the experiment harness's ``_gate_mask``. That validates the *signal*. It does
NOT
7       exercise the **served path**: in production the windows come from
8       ``trajectory_to_action_windows`` at the production operating point
9       (``velocity_threshold=0.02, dead_time_merge_gap=2.0, min_action_window_duration=2.0``),
the
10      net-crossing count is computed by ``FusionSegmenter._enrich_candidate_stats``, and the
gate is
11      ``FusionSegmenter._select_for_handoff``. THIS module drives exactly those production
methods ?
12      no reimplementation of the windowing or the gate ? on a cached trajectory CSV, so the
13      verification reflects what ``process_video`` actually hands to Gemini.
14 
15      GPU-free and Gemini-free: it stops at the handoff-selection boundary (the windows that
WOULD
16      be sent to the AI). With the person cue OFF (the default) nothing here loads torch or
touches
17      the GPU ? it just re-scores cached WASB trajectories.
18 
```

```
    Run (owner box, golden trajectories present):
        python -m backend.eval.served_gate_a --manifest output/human_lovo_manifest.json
    """
    from __future__ import annotations

    import argparse
    import json
    from dataclasses import dataclass
    from typing import Any, Dict, List, Optional, Tuple

    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
    from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
    from backend.eval.calibrate_wasb import load_golden_gts
    from backend.eval.metrics import Interval, score, Score
    from backend.eval import segmentation_metrics as _segm

    # Production windowing operating point (config.json indexing defaults ? the values
    # TrajectoryHybridSegmenter.process_video reads for the served fusion run).
    PROD_VELOCITY_THRESH = 0.02
    PROD_MERGE_GAP = 2.0
    PROD_MIN_WINDOW = 2.0


    def _served_idx_cfg(min_crossings: int) -> Dict[str, Any]:
        """The ``indexing`` config dict the served FusionSegmenter seams read ? mirrors
        config.json's ``indexing.fusion`` block, with Gate A at ``min_crossings`` and the
        person cue OFF (so no GPU / torch is touched)."""
        return {
            "fusion": {
                "substrate": "wasb",
                "min_crossings": min_crossings,
                "min_players": 0,
                "cue_flags": {},
            },
            "wasb": {"velocity_threshold": PROD_VELOCITY_THRESH},
            "dead_time_merge_gap": PROD_MERGE_GAP,
            "min_action_window_duration": PROD_MIN_WINDOW,
        }


    def served_handoff_windows(trajectory_csv: str, fps: float, frame_width: float,
                               min_crossings: int) -> List[Interval]:
        """Run the PRODUCTION FusionSegmenter decision path on a cached trajectory and
return the
        windows that would reach the AI handoff (i.e. the served rally candidates).

        Uses the real ``FusionSegmenter`` seam methods ? ``_enrich_candidate_stats``
(computes
        ``net_crossings``) and ``_select_for_handoff`` (Gate A) ? over windows from the
production
        ``trajectory_to_action_windows`` operating point. No GPU, no Gemini: the person cue
is OFF.
        """
        points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
        windows = TrackNetRunner.trajectory_to_action_windows(
            points, fps=fps, frame_width=frame_width,
            velocity_thresh=PROD_VELOCITY_THRESH, merge_gap=PROD_MERGE_GAP,
            min_window_duration=PROD_MIN_WINDOW,
        )
        idx_cfg = _served_idx_cfg(min_crossings)
        # Construct the real served segmenter (db=None, config carries the indexing block).
We only
        # call the pure decision seams ? no process_video, no DB, no provider ? exactly as
the
        # registry constructs it for `name`/`description`.
```

```
        seg = FusionSegmenter(None, {"indexing": idx_cfg, "sport": "badminton"})
        # The served per-window stats (incl. net_crossings) via the production enrichment
hook.
        window_stats = [
            seg._enrich_candidate_stats(cw, points, fps, frame_width, video_path="",
                                        idx_cfg=idx_cfg)
            for cw in windows
        ]
        keep = seg._select_for_handoff(windows, window_stats, idx_cfg)
        return [(windows[i]["start"], windows[i]["end"]) for i in keep]


    @dataclass
    class ServedVideoResult:
        name: str
        num_gt: int
        on: Score                # Gate-A ON  (min_crossings=2)
        off: Score               # Gate-A OFF (min_crossings=0, control)
        seg_ratio_on: float      # |preds| / |gts| with Gate-A ON  (over-segmentation)
        seg_ratio_off: float
        n_handoff_on: int
        n_handoff_off: int

        @property
        def delta_f1(self) -> float:
            return self.on.f1 - self.off.f1


    def evaluate_video(spec: Dict[str, Any], iou: float = 0.5,
                       on_crossings: int = 2) -> ServedVideoResult:
        """Score the served Gate-A ON (min_crossings=on_crossings) vs OFF (0) handoff
windows for
        one golden video against its golden labels."""
        fps, fw = float(spec["fps"]), float(spec["frame_width"])
        traj, labels = str(spec["trajectory"]), str(spec["labels"])
        gts = load_golden_gts(labels)
        preds_on = served_handoff_windows(traj, fps, fw, min_crossings=on_crossings)
        preds_off = served_handoff_windows(traj, fps, fw, min_crossings=0)
        return ServedVideoResult(
            name=str(spec["name"]),
            num_gt=len(gts),
            on=score(preds_on, gts, iou),
            off=score(preds_off, gts, iou),
            seg_ratio_on=_segm.segment_count_ratio(preds_on, gts),
            seg_ratio_off=_segm.segment_count_ratio(preds_off, gts),
            n_handoff_on=len(preds_on),
            n_handoff_off=len(preds_off),
        )


    def evaluate_manifest(specs: List[Dict[str, Any]], iou: float = 0.5,
                          on_crossings: int = 2) -> List[ServedVideoResult]:
        return [evaluate_video(s, iou, on_crossings) for s in specs]


    def aggregate(results: List[ServedVideoResult]) -> Dict[str, Any]:
        """Mean ON/OFF F1 + over-seg ratio, the paired ?F1, and the per-video win count."""
        n = len(results)

        def _mean(getter) -> float:
            return sum(getter(r) for r in results) / n if n else 0.0

        deltas = [r.delta_f1 for r in results]
        wins = sum(1 for d in deltas if d > 1e-9)
        losses = sum(1 for d in deltas if d < -1e-9)
```

```
        return {
            "num_videos": n,
            "mean_f1_on": _mean(lambda r: r.on.f1),
            "mean_f1_off": _mean(lambda r: r.off.f1),
            "mean_delta_f1": _mean(lambda r: r.delta_f1),
            "mean_precision_on": _mean(lambda r: r.on.precision),
            "mean_precision_off": _mean(lambda r: r.off.precision),
            "mean_recall_on": _mean(lambda r: r.on.recall),
            "mean_recall_off": _mean(lambda r: r.off.recall),
            "mean_seg_ratio_on": _mean(lambda r: r.seg_ratio_on),
            "mean_seg_ratio_off": _mean(lambda r: r.seg_ratio_off),
            "wins": wins,
            "losses": losses,
        }


    def format_table(results: List[ServedVideoResult], agg: Dict[str, Any], iou: float) ->
str:
        lines = [
            f"=== SERVED Gate-A verification (production FusionSegmenter path, IoU>={iou})
===",
            f"  {'video':<24s} {'gt':>3s} {'F1_off':>7s} {'F1_on':>7s} {'?F1':>7s} "
            f"{'segR_off':>9s} {'segR_on':>8s} {'hand_off':>9s} {'hand_on':>8s}",
        ]
        for r in results:
            lines.append(
                f"  {r.name:<24s} {r.num_gt:>3d} {r.off.f1:>7.3f} {r.on.f1:>7.3f} "
                f"{r.delta_f1:>+7.3f} {r.seg_ratio_off:>9.2f} {r.seg_ratio_on:>8.2f} "
                f"{r.n_handoff_off:>9d} {r.n_handoff_on:>8d}")
        lines += [
            f"  {'-' * 92}",
            f"  MEAN  F1 {agg['mean_f1_off']:.3f} (off) -> {agg['mean_f1_on']:.3f} (on)  "
            f"?F1={agg['mean_delta_f1']:+.3f}   "
            f"P {agg['mean_precision_off']:.3f}->{agg['mean_precision_on']:.3f}  "
            f"R {agg['mean_recall_off']:.3f}->{agg['mean_recall_on']:.3f}",
            f"  over-seg ratio {agg['mean_seg_ratio_off']:.2f} (off) -> "
            f"{agg['mean_seg_ratio_on']:.2f} (on)  | wins {agg['wins']}/{agg['num_videos']}
"
            f"(losses {agg['losses']})",
        ]
        return "\n".join(lines)


    def run(manifest_path: str, iou: float = 0.5) -> Tuple[List[ServedVideoResult],
Dict[str, Any]]:
        with open(manifest_path, encoding="utf-8") as f:
            specs = json.load(f)
        results = evaluate_manifest(specs, iou=iou)
        return results, aggregate(results)


    def main(argv: Optional[List[str]] = None) -> None:
        ap = argparse.ArgumentParser(
            description="Verify served Gate-A on real footage via the production
FusionSegmenter path.")
        ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
        ap.add_argument("--iou", type=float, default=0.5)
        args = ap.parse_args(argv)
        results, agg = run(args.manifest, iou=args.iou)
        print(format_table(results, agg, args.iou))


    if __name__ == "__main__":
        main()

```

59. assistant (2026-06-18T01:35:56.534Z)

Same manifest
`spec["fps"]`/`spec["frame_width"]`/`spec["trajectory"]`/`spec["labels"]`/`spec["name"]` pattern
(line 109-110, 115) ? operator manifest, KeyError on malformed. Same category.

Now I have a recurring pattern across eval CLI tools (manifest spec access). These are all
operator-facing eval tools where a KeyError on a hand-written manifest is loud-and-acceptable.
The prompt asks for defects a maintainer would call real, where renamed/typed-differently data
triggers it. The manifest readers are borderline but consistent with the repo's own convention
(e.g. `_derive_video_path` deliberately does `spec.get("name")` in the error message but
`spec["labels"]` for the value).

Let me focus on the highest-value remaining target: code that reads data where the schema
actually varies or has been renamed. Let me check `serve_contrast.py`, `per_user_gate.py`, and
`score_calibration.py`. Also let me check `metrics.py` and `segmentation_metrics.py` briefly.
Let me look at the person_detector det-dict contract and `fusion_features.py`.

60. user (2026-06-18T01:35:57.850Z)

```
1       """The eval?serve worked example: score a cue at BOTH windowings and run the promotion
guard.
2
3       This is the reproducible proof of the foundational fix. For the Gate-A held-shuttle
guard it
4       builds candidate windows directly from each golden trajectory CSV under BOTH named
windowings
5       (:data:`backend.eval.windowing.PROPOSAL` and :data:`~backend.eval.windowing.SERVED`),
scores
6       CONTROL vs Gate-A (``net_crossings >= 2``) at each, and feeds the per-video held-out ?F1
into
7       :func:`backend.eval.promotion.promote_if_served_safe`.
8
9       It demonstrates the bug *and* the fix in one run:
10        - PROPOSAL windowing: Gate-A wins (historically **?F1 ? +0.074**, CI clears 0, 6/0
sign test).
11        - SERVED windowing:   Gate-A does **not** win (historically **?F1 ? ?0.008**, real-
rally loss).
12        - The promotion guard therefore **vetoes** a promotion that would otherwise pass on
the
13          proposal numbers ? a cue cannot be promoted on a windowing production doesn't serve.
14
15      Unlike the persisted ``*_golden_features.json`` (which are PROPOSAL-windowed only), this
16      re-windows from the raw trajectory CSVs so the SERVED candidate set is genuinely
different.
17      Pure CPU, GPU-free, no Gemini, no DB. Reuses only shipped pieces (windowing, rally_gate,
18      metrics, promotion). Owner runs it against the real golden CSVs; tests exercise it on
synthetic
19      trajectories.
20
21      CLI::
22
23          python -m backend.eval.serve_contrast --manifest output/human_lovo_manifest.json
24      """
25      from __future__ import annotations
26
27      import argparse
28      import json
29      from typing import Any, Dict, List, Sequence, Tuple
30
31      from backend.eval import windowing as W
32      from backend.eval.calibrate_wasb import load_golden_gts
33      from backend.eval.gemini_refine import merge_overlaps
34      from backend.eval.metrics import score
35      from backend.eval.promotion import promote_if_served_safe
```

```
    from backend.pipeline.detectors import rally_gate
    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

    Interval = Tuple[float, float]
    IOU = 0.5
    MIN_CROSSINGS = 2


    def gate_a_scores_at(points: List[Any], gts: List[Interval], fps: float, frame_width:
float,
                         preset: W.WindowingPreset, iou: float = IOU,
                         min_crossings: int = MIN_CROSSINGS, merge_gap: float = 1.0
                         ) -> Tuple[float, float]:
        """One video's (CONTROL F1, Gate-A F1) at ``preset``'s windowing.

        Builds candidates under ``preset``, then scores two static variants vs ``gts``:
          - CONTROL  ? every candidate window, overlap-merged (the un-gated baseline).
          - Gate-A   ? keep only windows whose shuttle net-crossing count >=
``min_crossings``.

        ``merge_gap`` is the post-keep overlap-merge gap (the harness's ``merge_overlaps``
default),
        independent of the windowing preset's own coalescing gap. Returns the two F1
scores."""
        windows = W.build_windows(points, fps, frame_width, preset)
        intervals = W.windows_to_intervals(windows)
        if not intervals:
            return (0.0, 0.0)
        est = rally_gate.estimate_play_axis(points)
        gated = rally_gate.gate_windows(points, intervals, fps, min_crossings=0,
estimate=est)
        crossings = [g.crossings for g in gated]

        control = merge_overlaps(intervals, gap_tol=merge_gap)
        kept = [iv for iv, c in zip(intervals, crossings) if c >= min_crossings]
        gate_a = merge_overlaps(kept, gap_tol=merge_gap)
        return (score(control, gts, iou).f1, score(gate_a, gts, iou).f1)


    def contrast_video(spec: Dict[str, Any], iou: float = IOU,
                       min_crossings: int = MIN_CROSSINGS) -> Dict[str, Any]:
        """Per-video CONTROL/Gate-A F1 at BOTH the proposal and served windowings."""
        fps, fw = float(spec["fps"]), float(spec["frame_width"])
        points = TrackNetRunner.parse_trajectory_csv(str(spec["trajectory"]))
        gts = load_golden_gts(str(spec["labels"]))
        p_control, p_gate = gate_a_scores_at(points, gts, fps, fw, W.PROPOSAL, iou,
min_crossings)
        s_control, s_gate = gate_a_scores_at(points, gts, fps, fw, W.SERVED, iou,
min_crossings)
        return {
            "name": spec["name"],
            "proposal": {"control_f1": p_control, "gate_a_f1": p_gate, "delta_f1": p_gate -
p_control},
            "served": {"control_f1": s_control, "gate_a_f1": s_gate, "delta_f1": s_gate -
s_control},
        }


    def run_contrast(specs: Sequence[Dict[str, Any]], iou: float = IOU,
                     min_crossings: int = MIN_CROSSINGS, min_n: int = 5,
                     served_regress_eps: float = 0.0) -> Dict[str, Any]:
        """Full corpus: per-video contrast + the served-guarded promotion decision for
Gate-A.

        Gate-A is a *structural* guard (held-shuttle FP killer), so the promotion gate keeps
```

```
it ON
91          regardless of measured lift ? but it is only auto-promoted to the SERVED default
when it is
92          served-safe. This run shows it ISN'T (it regresses served), so the guard vetoes the
93          eval-only proposal win exactly as the foundation demands."""
94          per_video = [contrast_video(s, iou, min_crossings) for s in specs]
95          served_a = [v["served"]["control_f1"] for v in per_video]
96          served_b = [v["served"]["gate_a_f1"] for v in per_video]
97          proposal_a = [v["proposal"]["control_f1"] for v in per_video]
98          proposal_b = [v["proposal"]["gate_a_f1"] for v in per_video]
99
100         decision = promote_if_served_safe(
101             served_a, served_b,
102             proposal_a=proposal_a, proposal_b=proposal_b,
103             structural=True, min_n=min_n, served_regress_eps=served_regress_eps,
104         )
105         return {
106             "iou": iou,
107             "min_crossings": min_crossings,
108             "num_videos": len(per_video),
109             "per_video": per_video,
110             "decision": decision.as_dict(),
111         }
112
113
114     def format_contrast(result: Dict[str, Any]) -> str:
115         d = result["decision"]
116         prop, serv = d["proposal"], d["served"]
117         lines = [
118             f"=== Gate-A served-vs-proposal windowing contrast (n={result['num_videos']}, "
119             f"IoU>={result['iou']}, min_crossings={result['min_crossings']}) ===",
120             f"  {'video':<24}{'PROPOSAL ?F1':>14}{'SERVED ?F1':>14}",
121         ]
122         for v in result["per_video"]:
123             lines.append(f"  {v['name']:<24}{v['proposal']['delta_f1']:>+14.3f}"
124                          f"{v['served']['delta_f1']:>+14.3f}")
125         lines += [
126             "  " + "-" * 50,
127             f"  PROPOSAL: mean ?F1 {prop['mean_delta']:+.4f}  CI [{prop['ci_low']:+.4f}, "
128             f"{prop['ci_high']:+.4f}]  {int(prop['sign_test']['wins'])}W/"
129             f"{int(prop['sign_test']['losses'])}L  regresses={prop['regresses']}",
130             f"  SERVED  : mean ?F1 {serv['mean_delta']:+.4f}  CI [{serv['ci_low']:+.4f}, "
131             f"{serv['ci_high']:+.4f}]  {int(serv['sign_test']['wins'])}W/"
132             f"{int(serv['sign_test']['losses'])}L  regresses={serv['regresses']}",
133             "",
134             f"  promote={d['promote']}  served_safe={d['served_safe']}  "
135             f"vetoed_by_served={d['vetoed_by_served']}",
136             f"  -> {d['reason']}",
137         ]
138         return "\n".join(lines)
139
140
141     def main() -> None:
142         ap = argparse.ArgumentParser(
143             description="Gate-A eval?serve worked example: score CONTROL vs Gate-A at the
PROPOSAL "
144                         "and SERVED windowings, then run the served-guarded promotion
gate.")
145         ap.add_argument("--manifest", default="output/human_lovo_manifest.json",
146                         help="JSON: [{name, trajectory, fps, frame_width, labels}, ...]")
147         ap.add_argument("--iou", type=float, default=IOU)
148         ap.add_argument("--min-crossings", type=int, default=MIN_CROSSINGS)
149         ap.add_argument("--min-n", type=int, default=5, help="promotion min-N floor")
150         ap.add_argument("--served-regress-eps", type=float, default=0.0,
151                         help="served regression tolerance (0.0 = no served-mean drop
```

```
allowed)")
152         ap.add_argument("--json", default=None, help="also write the structured result JSON
here")
153         args = ap.parse_args()
154
155         with open(args.manifest, encoding="utf-8") as f:
156             specs = json.load(f)
157         result = run_contrast(specs, iou=args.iou, min_crossings=args.min_crossings,
158                               min_n=args.min_n, served_regress_eps=args.served_regress_eps)
159         print(format_contrast(result))
160         if args.json:
161             with open(args.json, "w", encoding="utf-8") as f:
162                 json.dump(result, f, indent=2)
163             print(f"\nwrote {args.json}")
164
165
166     if __name__ == "__main__":
167         main()
168
```

61. assistant (2026-06-18T01:36:02.737Z)

Same manifest pattern. Consistent.

Let me now check the validators which read ffprobe/external metadata ? `format.py`, `pts_continuity.py`, `resolution_fps.py`, `scene_cut.py` ? these parse ffprobe JSON. The `format.py` showed up in the JSON-reading grep. Let me check the validators and the `utils/video.py` (ffprobe parsing).

62. user (2026-06-18T01:36:04.158Z)

```
1       import subprocess
2       import json
3       import os
4       from backend.pipeline.validators.base import VideoValidator, ValidationResult
5       from typing import Dict, Any
6
7       class FormatValidator(VideoValidator):
8           @property
9           def name(self) -> str:
10              return "format_check"
11
12          @property
13          def description(self) -> str:
14              return "Verifies the video is a valid, uncorrupted MP4 container and extracts
metadata."
15
16          def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
17              if not os.path.exists(video_path):
18                  return ValidationResult(
19                      validator_name=self.name,
20                      passed=False,
21                      message=f"File not found: {video_path}"
22                  )
23
24              cmd = [
25                  "ffprobe",
26                  "-v", "error",
27                  "-select_streams", "v:0",
28                  "-show_format",
29                  "-show_streams",
30                  "-of", "json",
31                  video_path
32              ]
```

```
33
34              try:
35                  result = subprocess.run(cmd, capture_output=True, text=True, check=True)
36                  info = json.loads(result.stdout)
37
38                  if not info.get("streams") or not info.get("format"):
39                      return ValidationResult(
40                          validator_name=self.name,
41                          passed=False,
42                          message="Failed to retrieve valid video streams from the file."
43                      )
44
45                  # Store the metadata in context for subsequent validators to utilize
46                  context["ffprobe_meta"] = info
47
48                  stream = info["streams"][0]
49                  fmt = info["format"]
50
51                  details = {
52                      "format_name": fmt.get("format_name"),
53                      "duration": float(fmt.get("duration", 0)),
54                      "size_bytes": int(fmt.get("size", 0)),
55                      "codec_name": stream.get("codec_name"),
56                  }
57
58                  # Check if container name includes mp4
59                  if "mp4" not in fmt.get("format_name", "").lower():
60                      return ValidationResult(
61                          validator_name=self.name,
62                          passed=False,
63                          message=f"Invalid format: Found '{fmt.get('format_name')}', but MP4
is required.",
64                          details=details
65                      )
66
67                  return ValidationResult(
68                      validator_name=self.name,
69                      passed=True,
70                      message="Video format is valid MP4 container.",
71                      details=details
72                  )
73
74              except subprocess.CalledProcessError as e:
75                  return ValidationResult(
76                      validator_name=self.name,
77                      passed=False,
78                      message="Video stream is corrupted or unreadable by ffprobe.",
79                      details={"error": e.stderr}
80                  )
81              except Exception as e:
82                  return ValidationResult(
83                      validator_name=self.name,
84                      passed=False,
85                      message=f"Unexpected error running ffprobe: {str(e)}"
86                  )
87
```

63. user (2026-06-18T01:36:04.438Z)

```
1       import subprocess
2       import logging
3       from typing import Optional
4
5       logger = logging.getLogger(__name__)
6
```

```
7       def get_video_duration(video_path: str) -> float:
8           """Returns the exact duration of a video file in seconds using ffprobe.
9           Returns 0.0 on failure."""
10          try:
11              duration_output = subprocess.check_output([
12                  "ffprobe", "-v", "error", "-show_entries", "format=duration",
13                  "-of", "default=noprint_wrappers=1:nokey=1", video_path
14              ]).decode().strip()
15              return float(duration_output)
16          except Exception as e:
17              logger.error(f"Could not detect video duration for {video_path}: {e}")
18              return 0.0
19
20      def extract_video_chunk(video_path: str, start_time: float, duration: float,
21                              output_path: str, reencode: bool = False,
22                              scale_width: Optional[int] = None) -> bool:
23          """Extracts a chunk from a video file.
24
25          Args:
26              reencode: If True, re-encodes with libx264 (slower but frame-accurate).
27                        If False, uses stream-copy (fast but cuts at nearest keyframe).
28                        Use True for short clips where keyframe drift matters.
29              scale_width: If set (requires reencode), downscale to this width (height auto,
30                        even). Useful for high-res sources headed to an API that analyses at
31                        low res anyway ? shrinks uploads with no analysis loss.
32          """
33          cmd = [
34              "ffmpeg", "-y",
35              "-ss", str(start_time),
36              "-t", str(duration),
37              "-i", video_path,
38          ]
39          if reencode:
40              if scale_width:
41                  cmd += ["-vf", f"scale={int(scale_width)}:-2"]
42              cmd += ["-c:v", "libx264", "-preset", "ultrafast", "-crf", "23", "-c:a", "aac"]
43          else:
44              cmd += ["-c", "copy"]
45          cmd.append(output_path)
46          try:
47              subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
48              return True
49          except Exception as e:
50              logger.error(f"Failed to extract chunk {start_time}s - {start_time+duration}s:
{e}")
51              return False
52
```

64. assistant (2026-06-18T01:36:09.783Z)

`format.py` line 48 `info["streams"][0]` is guarded by the `not info.get("streams")` check at line 38 (empty list/None is falsy). Fine.

The other validators read `context["ffprobe_meta"]` which `format.py` populates. But `format.py` only puts it in context AFTER passing. If a later validator reads `context["ffprobe_meta"]` directly without a guard, and format validation failed (so the key was never set), that would KeyError. Let me check `pts_continuity.py`, `resolution_fps.py`, `scene_cut.py`, `relevance.py`, `blur.py` to see how they read context. This is a real cross-validator data-dependency that could trigger.

65. user (2026-06-18T01:36:11.224Z)

```
1       from backend.pipeline.validators.base import VideoValidator, ValidationResult
2       from typing import Dict, Any
```

```

      class ResolutionFPSValidator(VideoValidator):
          @property
          def name(self) -> str:
              return "resolution_fps_check"

          @property
          def description(self) -> str:
              return "Checks if the video meets minimum resolution (e.g. 720p) and frame rate
(e.g. 30fps) requirements."

          def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
              meta = context.get("ffprobe_meta")
              if not meta or not meta.get("streams"):
                  return ValidationResult(
                      validator_name=self.name,
                      passed=False,
                      message="Video metadata is missing. Run format_check validator first."
                  )

              stream = meta["streams"][0]
              width = int(stream.get("width", 0))
              height = int(stream.get("height", 0))

              # Parse frame rate (e.g. "30/1" or "30000/1001")
              fps_str = stream.get("r_frame_rate", "0/1")
              try:
                  num, den = map(int, fps_str.split("/"))
                  fps = num / den if den != 0 else 0
              except ValueError:
                  fps = 0.0

              details = {
                  "width": width,
                  "height": height,
                  "fps": round(fps, 2)
              }

              # Check bounds from config
              val_cfg = config.get("validation", {})
              min_w = val_cfg.get("min_resolution_width", 1280)
              min_h = val_cfg.get("min_resolution_height", 720)
              min_fps = val_cfg.get("min_fps", 29.9)

              if width < min_w or height < min_h:
                  return ValidationResult(
                      validator_name=self.name,
                      passed=False,
                      message=f"Low resolution: Found {width}x{height}, but minimum is
{min_w}x{min_h}.",
                      details=details
                  )

              if fps < min_fps:
                  return ValidationResult(
                      validator_name=self.name,
                      passed=False,
                      message=f"Low frame rate: Found {round(fps, 2)} FPS, but minimum is
{min_fps} FPS.",
                      details=details
                  )

              return ValidationResult(
                  validator_name=self.name,
```

```
64                  passed=True,
65                  message="Video resolution and frame rate meet minimum standards.",
66                  details=details
67              )
68
```

66. user (2026-06-18T01:36:11.760Z)

```
1       import subprocess
2       from backend.pipeline.validators.base import VideoValidator, ValidationResult
3       from typing import Dict, Any
4
5       class PTSContinuityValidator(VideoValidator):
6           @property
7           def name(self) -> str:
8               return "pts_continuity_check"
9
10          @property
11          def description(self) -> str:
12              return "Analyzes Presentation Time Stamps (PTS) of keyframes to check for
timeline gaps, jumps, or editing splices."
13
14          def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
15              # Command to extract packet PTS and flags (extremely fast, works on H.264 &
H.265)
16              cmd = [
17                  "ffprobe",
18                  "-v", "error",
19                  "-select_streams", "v:0",
20                  "-show_entries", "packet=pts_time,flags",
21                  "-of", "csv=p=0",
22                  video_path
23              ]
24
25              try:
26                  result = subprocess.run(cmd, capture_output=True, text=True, check=True)
27                  output = result.stdout.strip()
28
29                  if not output:
30                      return ValidationResult(
31                          validator_name=self.name,
32                          passed=False,
33                          message="No packet timestamps could be extracted from the video
stream."
34                      )
35
36                  timestamps = []
37                  for line in output.split("\n"):
38                      line = line.strip()
39                      if line:
40                          parts = line.split(",")
41                          if len(parts) == 2:
42                              pts_str, flags = parts
43                              if "K" in flags: # Check if keyframe
44                                  try:
45                                      timestamps.append(float(pts_str))
46                                  except ValueError:
47                                      continue
48
49                  if len(timestamps) < 2:
50                      return ValidationResult(
51                          validator_name=self.name,
52                          passed=True,
53                          message="Video is too short to perform a PTS continuity check.",
```

```
                        details={"keyframes_count": len(timestamps)}
                    )

                # Analyze gaps
                max_gap = 0.0
                backward_jumps = 0
                gaps = []

                for i in range(1, len(timestamps)):
                    gap = timestamps[i] - timestamps[i-1]
                    gaps.append(gap)
                    if gap < 0:
                        backward_jumps += 1
                    if gap > max_gap:
                        max_gap = gap

                # An continuous GoPro recording usually has keyframe intervals (GOP size) of 1.0 to 4.0 seconds.
                # If there's a keyframe gap larger than 8.0 seconds or a negative gap (backward jump), it's highly likely spliced.
                details = {
                    "keyframes_count": len(timestamps),
                    "max_keyframe_gap_seconds": round(max_gap, 2),
                    "backward_jumps_detected": backward_jumps,
                    "first_keyframe": timestamps[0],
                    "last_keyframe": timestamps[-1]
                }

                if backward_jumps > 0:
                    return ValidationResult(
                        validator_name=self.name,
                        passed=False,
                        message="Video integrity compromise: Backward timeline jumps (splicing) detected.",
                        details=details
                    )

                # Threshold for sudden gap (e.g. 10.0 seconds)
                if max_gap > 10.0:
                    return ValidationResult(
                        validator_name=self.name,
                        passed=False,
                        message=f"Video integrity compromise: Detected a timestamp gap of {round(max_gap, 2)}s, indicating edited or missing content.",
                        details=details
                    )

                return ValidationResult(
                    validator_name=self.name,
                    passed=True,
                    message="PTS timeline is continuous with no abnormal gaps or backward jumps.",
                    details=details
                )

            except subprocess.CalledProcessError as e:
                return ValidationResult(
                    validator_name=self.name,
                    passed=False,
                    message="Failed to execute ffprobe PTS analysis.",
                    details={"error": e.stderr}
                )
            except Exception as e:
                return ValidationResult(
                    validator_name=self.name,
```

```
114                     passed=False,
115                     message=f"Unexpected error in PTS validation: {str(e)}"
116                 )
117
```

67. assistant (2026-06-18T01:36:20.993Z)

`resolution_fps.py` line 14-15 properly guards `context.get("ffprobe_meta")` and `meta.get("streams")`. Clean. `pts_continuity.py` doesn't use context. These validators are defensively written.

I've now examined the core external/persisted-data readers across the API, DB, providers, config, eval, validators, reporting, segmenters, and runners. The codebase is consistently defensive: `.get()` with defaults, guards before indexing, try/except around parses. The one recurring unguarded pattern is `spec["fps"]`/`spec["labels"]` etc. in operator-written eval manifests ? but those are consistent with the repo's loud-fail convention for CLI tools and a maintainer would likely consider a KeyError on a hand-malformed manifest acceptable (and the actual KeyError message names the missing key clearly).

Let me do a final targeted check on `eval/regression.py` and `metrics.py`/`segmentation_metrics.py` plus `rally_seq.py` and `eval_partial_labels.py` ? the regression baseline reader (`regression_baseline.json`) is persisted data with version/fingerprint fields that could be accessed unguarded. Let me check regression.py.

68. user (2026-06-18T01:36:21.953Z)

```
1       """Regression runner ? verify the production tool against golden-set ground truth.
2
3       Implements the explicit ask: *"given a video, get a golden set created and verify
4       its precision and recall against the production tool."* This is deliberately a thin
5       wrapper over machinery that already exists:
6
7           ground truth  ? AnnotationProvider (file / vendor / oracle)  [backend.annotations]
8           predictions   ? the production pipeline's stored DB rows     [backend.eval.harness]
9           comparison    ? interval matching + P/R/F1                   [backend.eval.metrics]
10          baseline      ? a per-video snapshot of "known good" numbers [this module]
11
12      A "regression" is a precision/recall drop beyond `tolerance` versus the snapshotted
13      baseline. Tiers map to annotation providers: e.g. tier "file" ? FileProvider (CI),
14      later tier "vendor" ? HumanVendorProvider (authoritative), tier "auto" ? an
15      independent-lineage oracle (fast smoke alarm). See docs/GOLDEN_SET_IMPLEMENTATION.md.
16      """
17      import os
18      import json
19      import hashlib
20      import logging
21      from dataclasses import dataclass, asdict
22      from typing import Dict, List, Optional
23
24      from backend.infrastructure.database import Database
25      from backend.eval import metrics
26      from backend.eval.harness import get_predictions
27      from backend.annotations import annotation_registry
28
29      logger = logging.getLogger(__name__)
30
31      # Tracked location (NOT under the gitignored output/), so a fresh checkout / CI run can
32      # actually load a committed baseline to compare against.
33      DEFAULT_BASELINE_PATH = os.path.join("eval_baselines", "regression_baseline.json")
34      DEFAULT_TOLERANCE = 0.05
35
36
37      def gt_hash(ground_truth: List[metrics.Interval]) -> str:
38          """Stable short hash of the GT intervals ? detects a baseline taken against
different labels."""
```

```
        norm = sorted((round(float(s), 3), round(float(e), 3)) for s, e in ground_truth)
        return hashlib.sha1(json.dumps(norm).encode()).hexdigest()[:12]


    @dataclass
    class RegressionResult:
        video_id: str
        stage: str
        iou_threshold: float
        precision: float
        recall: float
        f1: float
        # Baseline values compared against (None when no baseline existed yet).
        baseline_precision: Optional[float]
        baseline_recall: Optional[float]
        tolerance: float
        regressed: bool
        reasons: List[str]
        # True only when a baseline existed to compare against. Distinguishes a genuine PASS
        # from "nothing to compare" so the CLI can refuse to silently green-light the
latter.
        has_baseline: bool = False
        gt_hash: Optional[str] = None

        def as_dict(self) -> dict:
            return asdict(self)


    # ---------------------------------------------------------------- baseline store

    def load_baselines(path: str = DEFAULT_BASELINE_PATH) -> Dict[str, dict]:
        """Load the per-video baseline snapshot file (returns {} if absent)."""
        if not os.path.isfile(path):
            return {}
        with open(path) as f:
            return json.load(f)


    def _baseline_key(video_id: str, stage: str) -> str:
        return f"{video_id}::{stage}"


    def save_baseline(video_id: str, stage: str, precision: float, recall: float,
                      f1: float, path: str = DEFAULT_BASELINE_PATH,
                      iou_threshold: Optional[float] = None, detector: Optional[str] = None,
                      gt_fingerprint: Optional[str] = None) -> None:
        """Snapshot a video/stage's current numbers as the new 'known good' baseline.

        Records the iou_threshold, detector, and a GT fingerprint alongside the metrics so a
        later run computed under different settings (or against different labels) is
detected
        as incommensurable rather than silently compared.
        """
        baselines = load_baselines(path)
        baselines[_baseline_key(video_id, stage)] = {
            "video_id": video_id, "stage": stage,
            "precision": precision, "recall": recall, "f1": f1,
            "iou_threshold": iou_threshold, "detector": detector, "gt_hash": gt_fingerprint,
        }
        os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
        with open(path, "w") as f:
            json.dump(baselines, f, indent=2, sort_keys=True)
        logger.info("Saved baseline for %s (precision=%.3f recall=%.3f)",
_baseline_key(video_id, stage), precision, recall)

```

```
101
102     # ------------------------------------------------------------------ the runner
103
104     def regress(db: Database, video_id: str, ground_truth: List[metrics.Interval],
105                 stage: str = "final", iou_threshold: float = 0.5,
106                 detector: Optional[str] = None,
107                 tolerance: float = DEFAULT_TOLERANCE,
108                 baseline_path: str = DEFAULT_BASELINE_PATH) -> RegressionResult:
109         """Score stored predictions for one video against ground truth and compare to
baseline.
110
111         `ground_truth` is a list of (start, end) intervals ? typically obtained from an
112         AnnotationProvider (see `regress_with_provider`). A regression is flagged when
113         precision OR recall falls more than `tolerance` below the snapshotted baseline.
114         If no baseline exists yet, `regressed` is False (nothing to compare to) and the
115         caller can choose to snapshot the current numbers via `save_baseline`.
116         """
117         preds = get_predictions(db, video_id, stage, detector=detector)
118         s = metrics.score(preds, ground_truth, iou_threshold)
119         fp = gt_hash(ground_truth)
120
121         baselines = load_baselines(baseline_path)
122         b = baselines.get(_baseline_key(video_id, stage))
123
124         reasons: List[str] = []
125         regressed = False
126         has_baseline = b is not None
127         base_p = base_r = None
128         if b is not None:
129             base_p, base_r = b["precision"], b["recall"]
130             # Incommensurable comparison guard: a baseline taken at a different IoU/detector
or
131             # against different labels is not a valid reference ? flag it instead of
trusting it.
132             b_iou, b_det, b_hash = b.get("iou_threshold"), b.get("detector"),
b.get("gt_hash")
133             if b_iou is not None and abs(float(b_iou) - iou_threshold) > 1e-9:
134                 regressed = True
135                 reasons.append(f"baseline IoU {b_iou} != run IoU {iou_threshold}
(incommensurable)")
136             if b_det is not None and b_det != detector:
137                 regressed = True
138                 reasons.append(f"baseline detector {b_det!r} != run detector {detector!r}
(incommensurable)")
139             if b_hash is not None and b_hash != fp:
140                 regressed = True
141                 reasons.append(f"baseline GT hash {b_hash} != run GT hash {fp} (labels
changed)")
142             if s.precision < base_p - tolerance:
143                 regressed = True
144                 reasons.append(f"precision {s.precision:.3f} < baseline {base_p:.3f} -
{tolerance}")
145             if s.recall < base_r - tolerance:
146                 regressed = True
147                 reasons.append(f"recall {s.recall:.3f} < baseline {base_r:.3f} -
{tolerance}")
148         else:
149             reasons.append("no baseline recorded for this video/stage ? nothing to compare")
150
151         return RegressionResult(
152             video_id=video_id, stage=stage, iou_threshold=iou_threshold,
153             precision=s.precision, recall=s.recall, f1=s.f1,
154             baseline_precision=base_p, baseline_recall=base_r,
155             tolerance=tolerance, regressed=regressed, reasons=reasons,
156             has_baseline=has_baseline, gt_hash=fp,
```

```
157         )
158 
159 
160     def regress_with_provider(db: Database, video_id: str, video_ref: str,
161                               tier: str = "file", stage: str = "final",
162                               iou_threshold: float = 0.5,
163                               detector: Optional[str] = None,
164                               tolerance: float = DEFAULT_TOLERANCE,
165                               baseline_path: str = DEFAULT_BASELINE_PATH,
166                               guideline: Optional[str] = None,
167                               sport: str = "badminton",
168                               provider_kwargs: Optional[dict] = None) -> RegressionResult:
169         """End-to-end: obtain ground truth from the `tier` annotation provider, then
regress.
170 
171         `tier` is an annotation-provider registry key ("file" today; "vendor"/"auto"
172         once those providers land). `video_ref` is whatever that provider expects (a CSV
173         path for FileProvider, a video_id/URL for a vendor). `video_id` keys the stored
174         predictions in the DB (the file MD5).
175         """
176         provider = annotation_registry.get(tier, **(provider_kwargs or {}))
177         gt = provider.annotate(video_ref, guideline=guideline, sport=sport)
178         return regress(db, video_id, gt, stage=stage, iou_threshold=iou_threshold,
179                        detector=detector, tolerance=tolerance, baseline_path=baseline_path)
180 
181 
182     def format_result(r: RegressionResult) -> str:
183         verdict = "REGRESSED" if r.regressed else "ok"
184         head = (f"[{verdict}] {r.video_id} ({r.stage}) "
185                 f"precision={r.precision:.3f} recall={r.recall:.3f} f1={r.f1:.3f}")
186         if r.baseline_precision is not None:
187             head += f" | baseline p={r.baseline_precision:.3f} r={r.baseline_recall:.3f}
tol={r.tolerance}"
188         return head + "".join(f"\n   - {why}" for why in r.reasons)
189 
```

69. assistant (2026-06-18T01:36:31.627Z)

Found a real one. Line 129: `base_p, base_r = b["precision"], b["recall"]`.

Here `b = baselines.get(_baseline_key(video_id, stage))` where `baselines = load_baselines(baseline_path)` reads the persisted `regression_baseline.json` (line 121-122, 68-73). `b` is a baseline record dict loaded from that committed JSON file. Line 129 accesses `b["precision"]` and `b["recall"]` **unconditionally** with bracket indexing ? while every OTHER field on the same record is read defensively with `.get()` (line 132: `b.get("iou_threshold"), b.get("detector"), b.get("gt_hash")`).

`save_baseline` always writes both `precision` and `recall`, so a baseline file written by this code has them. But the file is a tracked, hand-editable JSON artifact in `eval_baselines/` (committed to the repo, explicitly described as "tracked location ... so CI can load a committed baseline"). A baseline record that is missing `precision` or `recall` ? e.g. a hand-edited entry, a partial/older-schema record, or one merged from another source ? would raise `KeyError: 'precision'` at line 129, crashing the regression run, instead of the graceful incommensurable-comparison handling the surrounding code is clearly designed for. The inconsistency between `.get()` for the metadata fields and `[...]` for the metric fields is the tell.

This is real but the trigger requires a malformed/hand-edited baseline file. Severity low-to-medium, moderate confidence. Let me verify `get_predictions` in harness.py and check `metrics.score` is robust, then also confirm `eval_partial_labels.py` and `rally_seq.py` don't have a stronger defect. Let me check harness.py.

70. user (2026-06-18T01:36:32.461Z)

```
1       """Score the pipeline's detected rallies against ground-truth annotations.
2 
```

```
    Reads predictions straight from the SQLite DB (no pipeline re-run, no API cost),
    so scoring is fast, deterministic, and free. Two prediction "stages" can be
    scored:

      - ``candidates`` ? raw detector windows (``candidate_segments`` table). What
        the shuttle/person detector found, before AI confirmation.
      - ``final``      ? confirmed play segments (``play_segments`` table). After the
        AI handoff trimmed/validated boundaries.

    Comparing the two tells you whether the weak link is *detection* (candidates
    already miss rallies) or *segmentation* (candidates catch them but final is
    wrong) ? see docs/EVALUATION.md.
    """

    import logging
    from dataclasses import dataclass, field
    from typing import Any, List, Dict, Optional

    from backend.infrastructure.database import Database
    from backend.eval import metrics
    from backend.eval.metrics import Interval, Score, MatchResult

    logger = logging.getLogger(__name__)

    STAGES = ("candidates", "final")


    @dataclass
    class StageReport:
        stage: str
        pred_intervals: List[Interval]
        score: Score
        match_result: MatchResult
        pr_curve: List[Score] = field(default_factory=list)


    @dataclass
    class EvalReport:
        video_id: str
        iou_threshold: float
        gt_intervals: List[Interval]
        stages: Dict[str, StageReport] = field(default_factory=dict)


    def _rows_to_intervals(rows: List[dict]) -> List[Interval]:
        """Pull (start_time, end_time) out of DB segment rows, sorted by start."""
        ivs = [(float(r["start_time"]), float(r["end_time"])) for r in rows]
        ivs.sort(key=lambda iv: iv[0])
        return ivs


    def get_predictions(db: Database, video_id: str, stage: str,
                        detector: Optional[str] = None) -> List[Interval]:
        """Fetch predicted rally intervals for one stage from the DB."""
        if stage == "candidates":
            rows = db.query_candidate_segments(video_id, detector=detector)
        elif stage == "final":
            rows = db.query_play_segments(video_id, {})
        else:
            raise ValueError(f"unknown stage {stage!r}; expected one of {STAGES}")
        return _rows_to_intervals(rows)


    def evaluate(db: Database, video_id: str, gt_intervals: List[Interval],
                 stages: List[str], iou_threshold: float = 0.5,
```

```
                 detector: Optional[str] = None,
                 with_pr_curve: bool = False) -> EvalReport:
        """Score the requested stages for one video against ground truth."""
        report = EvalReport(video_id=video_id, iou_threshold=iou_threshold,
                            gt_intervals=gt_intervals)
        for stage in stages:
            preds = get_predictions(db, video_id, stage, detector=detector)
            mr = metrics.match_intervals(preds, gt_intervals, iou_threshold)
            s = metrics.score(preds, gt_intervals, iou_threshold, match_result=mr)
            curve = metrics.pr_curve(preds, gt_intervals) if with_pr_curve else []
            report.stages[stage] = StageReport(stage=stage, pred_intervals=preds,
                                               score=s, match_result=mr, pr_curve=curve)
        return report


    # ---------------------------------------------------------------------- reporting

    def _fmt_ts(seconds: float) -> str:
        seconds = max(0, int(seconds))
        h, rem = divmod(seconds, 3600)
        m, s = divmod(rem, 60)
        return f"{h:02d}:{m:02d}:{s:02d}"


    def report_to_dict(report: EvalReport) -> dict:
        """JSON-serialisable view of an EvalReport."""
        out: Dict[str, Any] = {
            "video_id": report.video_id,
            "iou_threshold": report.iou_threshold,
            "num_ground_truth": len(report.gt_intervals),
            "stages": {},
        }
        for stage, sr in report.stages.items():
            out["stages"][stage] = {
                "score": sr.score.as_dict(),
                "pr_curve": [s.as_dict() for s in sr.pr_curve],
            }
        return out


    def format_report_text(report: EvalReport, show_failures: bool = False) -> str:
        """Human-readable console report for one video."""
        lines = [
            "=" * 64,
            f" Rally evaluation ? video '{report.video_id}'",
            f" Ground-truth rallies: {len(report.gt_intervals)} | IoU threshold:
{report.iou_threshold}",
            "=" * 64,
            f" {'Stage':<12} {'Pred':>5} {'TP':>4} {'FP':>4} {'FN':>4} "
            f"{'Prec':>6} {'Rec':>6} {'F1':>6} {'mIoU':>6} {'dStart':>7} {'dEnd':>7}",
            "-" * 64,
        ]
        for stage in report.stages:
            s = report.stages[stage].score
            lines.append(
                f" {stage:<12} {s.num_pred:>5} {s.true_positives:>4} {s.false_positives:>4}
"
                f"{s.false_negatives:>4} {s.precision:>6.2f} {s.recall:>6.2f} {s.f1:>6.2f} "
                f"{s.mean_iou:>6.2f} {s.mean_start_error:>6.1f}s {s.mean_end_error:>6.1f}s"
            )
        lines.append("=" * 64)

        if show_failures:
            for stage, sr in report.stages.items():
                misses = [report.gt_intervals[i] for i in
```

```
sr.match_result.unmatched_gt_indices]
131                 fps = [sr.pred_intervals[i] for i in sr.match_result.unmatched_pred_indices]
132                 lines.append(f"\n [{stage}] missed rallies (false negatives):
{len(misses)}")
133                 for iv in misses:
134                     lines.append(f"   - {_fmt_ts(iv[0])}?{_fmt_ts(iv[1])}")
135                 lines.append(f" [{stage}] spurious detections (false positives):
{len(fps)}")
136                 for iv in fps:
137                     lines.append(f"   - {_fmt_ts(iv[0])}?{_fmt_ts(iv[1])}")
138
139         return "\n".join(lines)
140
```

71. assistant (2026-06-18T01:36:43.201Z)

`_rows_to_intervals` (line 49) accesses `r["start_time"]`, `r["end_time"]` ? these are DB columns that are `NOT NULL`, always present. Fine.

I've now thoroughly cross-cut the bug class across the codebase. The repo is exceptionally defensively coded. My findings:

1. **`regression.py:129`** ? `b["precision"]`/`b["recall"]` unguarded on a baseline record loaded from the committed/hand-editable `regression_baseline.json`, while every sibling field uses `.get()`. The clearest defect ? a real inconsistency where a malformed/partial baseline entry KeyErrors instead of being handled.

2. The eval-CLI manifest readers (`fusion_golden.py`, `distill_local.py`, `served_gate_a.py`, `serve_contrast.py`, `_derive_video_path`) ? `spec["fps"]`/`spec["labels"]` etc. unguarded from operator JSON manifests. These are consistent across the repo and arguably loud-fail-acceptable for CLI tools, so lower confidence as "defects."

Let me do one final verification of the `_baseline_key` and confirm there's no upstream schema-validation that would prevent finding #1. The `load_baselines` returns raw `json.load` with no validation, and `save_baseline` always writes both fields. The trigger is a hand-edited or externally-provided baseline file. Given the surrounding code's clear intent to handle baselines gracefully (the `b.get()` for metadata + the `has_baseline`/`no baseline recorded` paths), accessing the metric fields with `[...]` is an inconsistency a maintainer would recognize.

Let me finalize. I'll report finding #1 as the primary (medium confidence, low severity), and the manifest pattern as a single low-confidence finding representing the family.

72. user (2026-06-18T01:36:52.280Z)

Structured output provided successfully

73. assistant (2026-06-18T01:37:03.785Z)

I completed an adversarial cross-cut of the **unguarded access on external/persisted data** bug class across the codebase. I traced every reader of JSON/DB rows/config/API responses I could find: the FastAPI layer (`main.py`), the Gemini provider response parsing, the SQLite row readers (`database.py`), the typed config loader, the eval label/fixture/manifest loaders (`gt_loader`, `golden_fixtures`, `distill_local`, `fusion_golden`, `served_gate_a`, `serve_contrast`), the per-user/promotion stores, the classifier JSON (de)serializer, the validators that parse ffprobe JSON, the WASB/TrackNet runners that parse trajectory CSVs, and the reporting/export modules.

The codebase is unusually defensive ? `.get()` with defaults, guards before indexing, and try/except around every parse are the norm.

I found one concrete defect worth reporting:

`backend/eval/regression.py:129` ? `base_p, base_r = b["precision"], b["recall"]` reads two metric fields with bare bracket indexing on a baseline record `b` loaded from the committed,

hand-editable `eval_baselines/regression_baseline.json`. Every sibling field on the same record (lines 132) is read with `.get()`, and the surrounding code is clearly built to degrade gracefully around baselines (`has_baseline` flag, "no baseline recorded" branch, incommensurable-comparison reasons). A malformed/partial/older-schema baseline entry missing either key raises an uncaught `KeyError` and aborts the regression run instead of being handled. Real but trigger-dependent (requires a malformed baseline file), so I rated it low severity / 0.5 confidence.

I deliberately did **not** report the recurring `spec["fps"]` / `spec["labels"]` unguarded-access pattern in the operator-facing eval CLIs (`fusion_golden.py`, `distill_local.py`, `served_gate_a.py`, `serve_contrast.py`): it's consistent with the repo's loud-fail convention for hand-written manifests, the KeyError names the missing key, and a maintainer would likely consider it acceptable rather than a defect ? it didn't clear the precision bar.